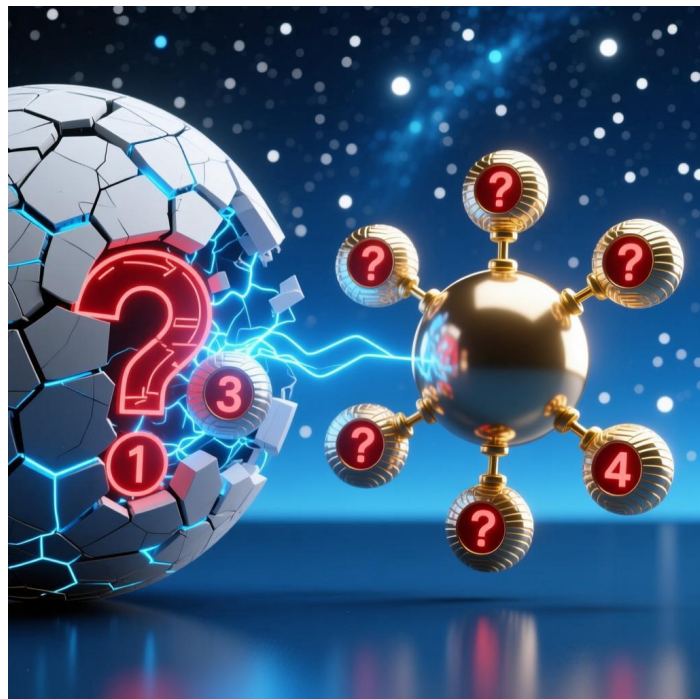




分治法：从入门到精通

王晓茹

北京邮电大学





目录

CONTENT

01 | 初识分治

02 | 扩展疆域

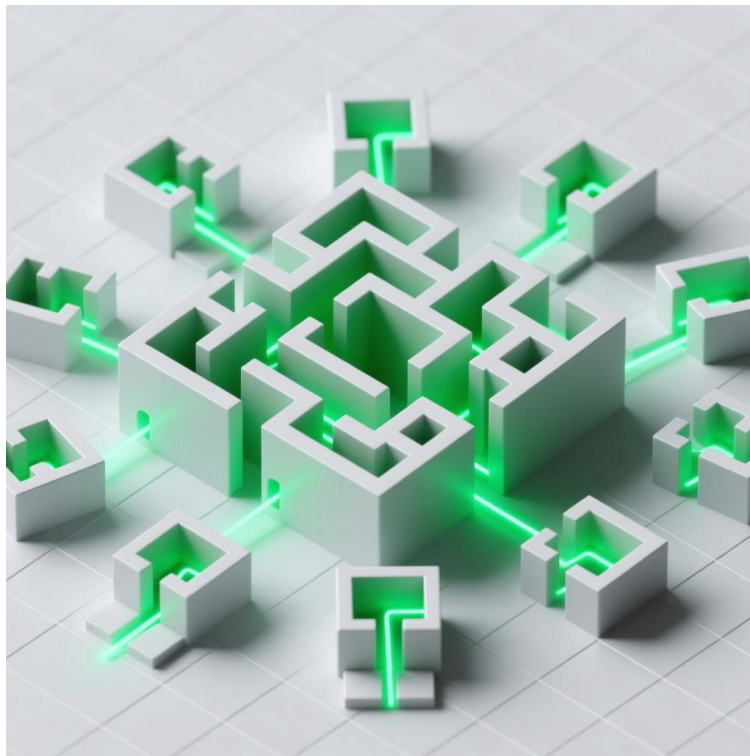
03 | 深度剖析

04 | 实战进阶





PART 01



初识分治



分治法：分而治之的艺术

Divide and Conquer: The Art of Problem Decomposition



分解 (Divide)

将复杂问题拆分为
规模更小的子问题



解决 (Conquer)

递归地解决
每一个独立的子问题



合并 (Combine)

将子问题的解
合并为原问题的解

分治法通过“**分解-解决-合并**”的策略，将复杂问题化繁为简，是提升**思维清晰度**与**算法效率**的强大工具。

历史脉络与思想起源



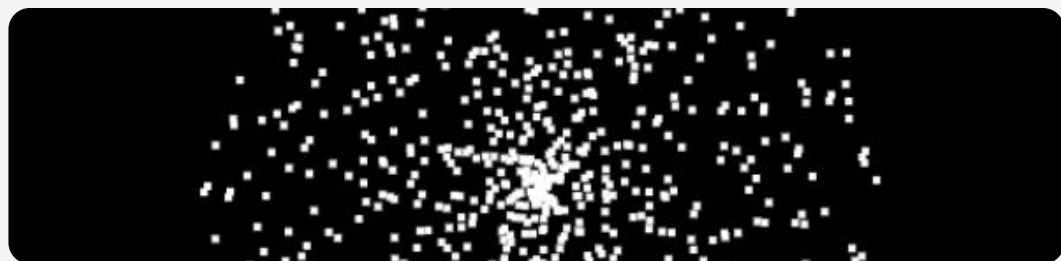
古代智慧

源于兵法“各个击破”与政治哲学“分而治之”的古老策略。



现代算法

20世纪中期，归并排序与快速排序的诞生，标志着其成为核心算法范式。



用分治八例透视递归之美

八例并举，揭示“分而治之”的算法精髓



二分搜索

不断对半分搜索区间



大整数乘法

将大数分解为较小部分



Strassen矩阵乘法

将矩阵分块递归计算



合并排序

将数组一分为二后归并



快速排序

按基准分区后递归



线性时间选择

利用划分缩小选择范围



最接近点对

将平面分割后求解



循环赛日程表

将选手分组安排赛程



PART 02



二分搜索



二分搜索：分治思想的最小化模型

在有序序列里，每次排除一半

线性扫描

$O(n)$

最坏情况需遍历全部
 n 个元素

二分搜索

$O(\log n)$

32个元素最多 5 次
1024个元素最多 10 次



核心前提

数组必须有序
这是分治决策的基础

Find an element in a sorted array:

Example: Find 9

3 5 7 8 9 12 15

“折半”抽象：分、治、合三步解构



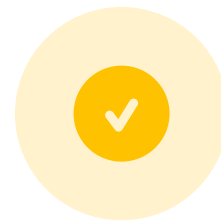
分 (Divide)

取中间元素，将区间一分为二。



治 (Conquer)

根据比较，仅选一侧继续，另一侧丢弃。



合 (Combine)

无需额外合并，答案即最终下标。

与传统归并排序不同，二分搜索的合并成本为零，其速度瓶颈仅取决于递归深度。

3 5 7 8 9 12 15

Python实现与边界陷阱

标准实现

```
1. def binary_search(arr, target):
2.     low, high = 0, len(arr) - 1
3.
4.     while low <= high:
5.         mid = (low + high) // 2
6.         if arr[mid] == target:
7.             return mid # 找到目标, 返回索引
8.         elif arr[mid] < target:
9.             low = mid + 1 # 目标在右半部分
10.        else:
11.            high = mid - 1 # 目标在左半部分
12.
13.    return -1 # 未找到目标
14.
15. # 示例
16. arr = [1, 3, 5, 7, 9]
17. target = 5
18. result = binary_search(arr, target)
19. print(f"元素 {target} 的索引是: {result}") # 输出: 元素 5 的索引是: 2
```

算法步骤

1. **初始化**: 设定两个指针, `low` 和 `high`, 分别指向数组的起始和末尾。
2. **循环查找**:
 - 计算中间位置: `mid = (low + high) // 2`。
 - 比较中间元素与目标值:
 - 如果中间元素等于目标值, 返回 `mid`。
 - 如果中间元素小于目标值, 调整 `low = mid + 1`。
 - 如果中间元素大于目标值, 调整 `high = mid - 1`。
3. **终止条件**: 当 `low` 超过 `high` 时, 说明目标值不在数组中, 返回不存在信号 (如 `-1`)

常见边界 Bug

死循环: `'low < high'`
漏掉 `'='` 可能漏掉单元素区间。

溢出: `'(low+high)'`
其他语言中可能溢出, 用 `'low+(high-low)//2'`。

返回值不确定
重复元素时, 返回哪个下标未定义。

边界是二分搜索的第一坑, 务必通过 **单步调试** 和 **单元测试** 确保正确性。

复杂度分析与并行化视角

复杂度总结

时间复杂度: $O(\log n)$

每次迭代都将搜索范围减半。

空间复杂度: $O(1)$

仅需常数个变量存储指针。

对比分析

优于线性扫描 $O(n)$ ，但在频繁插入/删除场景下，哈希表 $O(1)$ 查找可能更优。

$$T(n) = \underset{\substack{\text{\# subproblems} \\ \text{subproblem size}}}{1} T(\underset{\text{subproblem size}}{n/2}) + \underset{\text{work dividing and combining}}{\Theta(1)}$$

$$n^{\log_b a} = n^{\log_2 1} = n^0 = 1 \Rightarrow \text{CASE 2 } (k=0) \\ \Rightarrow T(n) = \Theta(\lg n).$$

并行化思考

虽然经典二分搜索是串行的，但其思想可扩展至并行场景：

 将大数据集分块，并行在各块内进行二分搜索。

 汇总各块结果，进行最终筛选。

 总体仍能保持对数级深度，充分利用多核优势。

跳出数组：二分思想的三大工程应用



代码调试

用二分法在版本历史中快速定位首次引入错误的 commit。

前提：错误出现时间是单调的。



数值计算

在单调函数 $f(x)=0$ 求解中，用二分法逼近根至指定精度。

前提：函数在区间内有单根且单调。



资源分配

在带宽受限场景下，用二分法找到最大可分配流量。

前提：分配方案可行性是单调的。

从二分搜索到分治算法体系



二分搜索

零合并成本



归并/快排

需合并/划分



高维分治

矩阵/几何

掌握二分搜索，不仅是掌握高效查找，更是获得了“**对数级缩小问题规模**”的通用思维钥匙，为学习更复杂的分治算法奠定坚实基础。



PART 03



快速幂：让指数爆炸瞬间降温



逐次相乘为何撑不住大指数？

【Example】 Exponentiation: computing X^N .

```
long int Pow ( long int X, unsigned int N )  
{  
    long int P = 1;  
    while ( N -- ) P *= X;  
    return P;  
}
```

$$X^N = \begin{cases} (X^2)^{N/2} & \text{if } N \text{ is even} \\ \text{or} \\ (X^2)^{(N-1)/2} \times X & \text{if } N \text{ is odd} \end{cases}$$

朴素算法 $O(n)$

99次

计算 2^{100} 需要进行99次乘法运算。

延迟: $\sim 100\mu\text{s}$

快速幂 $O(\log n)$

7次

计算 2^{100} 仅需7次乘法运算。

延迟: $\sim \text{ns级}$

当指数爆炸时， $O(n)$ 的朴素算法将面临严峻的性能挑战。

在密码学等领域，处理 a^{109} 量级的运算是家常便饭。快速幂算法能将CPU从 **马拉松选手** 变为 **短跑冠军**。

二进制拆分思想

1. 分解指数：把指数拆成2的幂次和

快速幂的第一把钥匙：任何正整数都能表示为二进制位权之和，从而将大指数运算拆解为一系列平方运算的乘积。

$$\begin{array}{ccccc} \text{指数 } n & & \text{2的幂次} & & \text{基底序列} \\ \text{13} & = & 8 + 4 + 1 & = & a^8 * a^4 * a^1 \\ (1101)_2 & & (2^3 + 2^2 + 2^0) & & \end{array}$$

边平方边筛选的行走规则

通过迭代，我们一边生成基底的平方序列，一边根据指数的二进制位决定是否将其乘入结果。

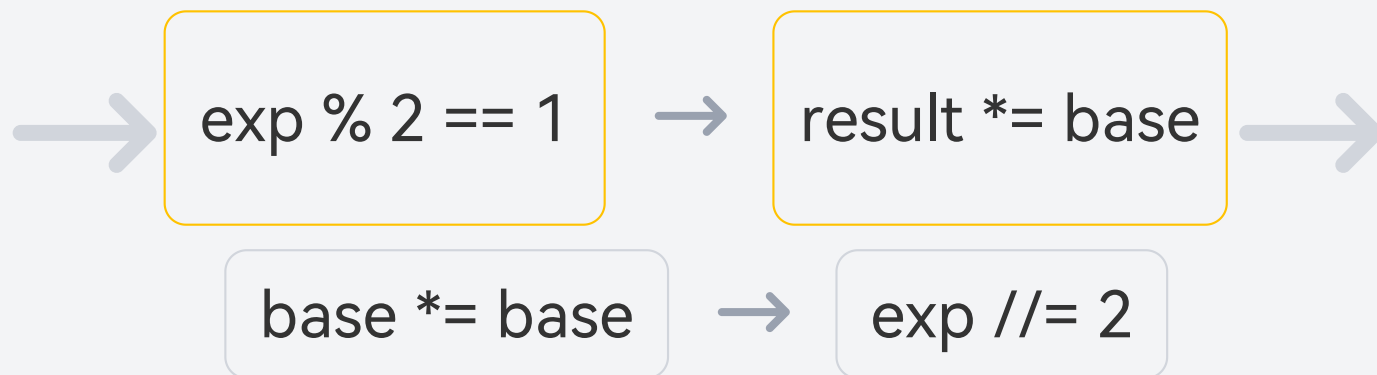
复杂度被牢牢锁死在 $O(\log n)$ 。

1. 初始化：设 `result = 1`，`base = a`，`exponent = n`。
2. 循环分解：当 `exponent > 0` 时，执行以下步骤：
 - 如果 `exponent` 是奇数，将 `base` 乘到 `result` 中。
 - 将 `base` 平方，`exponent` 右移一位（即 `exponent //= 2`）。
3. 返回结果：最终 `result` 即为 a^n 的值。

1. 初始化

`result = 1`
`base = a`
`exp = n`

2. 迭代 (当 `exp > 0`)



3. 完成

返回 `result`

简洁的Python实现

核心代码仅需一个循环，清晰展示了“平方乘”算法的精髓。

```
1. def fast_exponentiation(base, exponent):
2.     result = 1
3.     while exponent > 0:
4.         if exponent % 2 == 1:
5.             result *= base
6.             base *= base
7.             exponent //= 2 # 或者使用 exponent >>= 1
8.     return result
9.
10. # 示例
11. print(fast_exponentiation(3, 13)) # 输出: 1594323
```

边界与优化警示



位运算提速: 在C++/Java等语言中, 使用 `n & 1` 和 `n >>= 1` 代替取模和除法。



模运算防溢出: 计算 `a^n % mod` 时, 每次乘法后立即取模: `result = (result * a) % mod`。



竞赛模板: 掌握 `fast_exp_mod(base, exp, mod)` 函数, 可直接用于数论题目。

1. 初始化: 设 `result = 1`, `base = a`, `exponent = n`。
2. 循环分解: 当 `exponent > 0` 时, 执行以下步骤:
 - 如果 `exponent` 是奇数, 将 `base` 乘到 `result` 中。
 - 将 `base` 平方, `exponent` 右移一位 (即 `exponent //= 2`)。
3. 返回结果: 最终 `result` 即为 `a^n` 的值。

变体与扩展

超越整数：矩阵快速幂与模运算

“平方乘”思想的应用远不止于整数，它是解决线性递推和加密问题的核心加速器。

矩阵快速幂

将基底从整数 a 替换为矩阵 A ，可在 $O(\log n)$ 时间内计算出 A^n 。

典型应用：

- 计算斐波那契数列第 n 项。
- 求解任何常系数线性齐次递推关系。

模意义下快速幂

在模运算下， $(a * b) \% \text{mod}$ 的结果不会溢出，是密码学的基础。

典型应用：

- RSA、Diffie-Hellman等加密协议的核心运算。
- 计算大组合数取模。

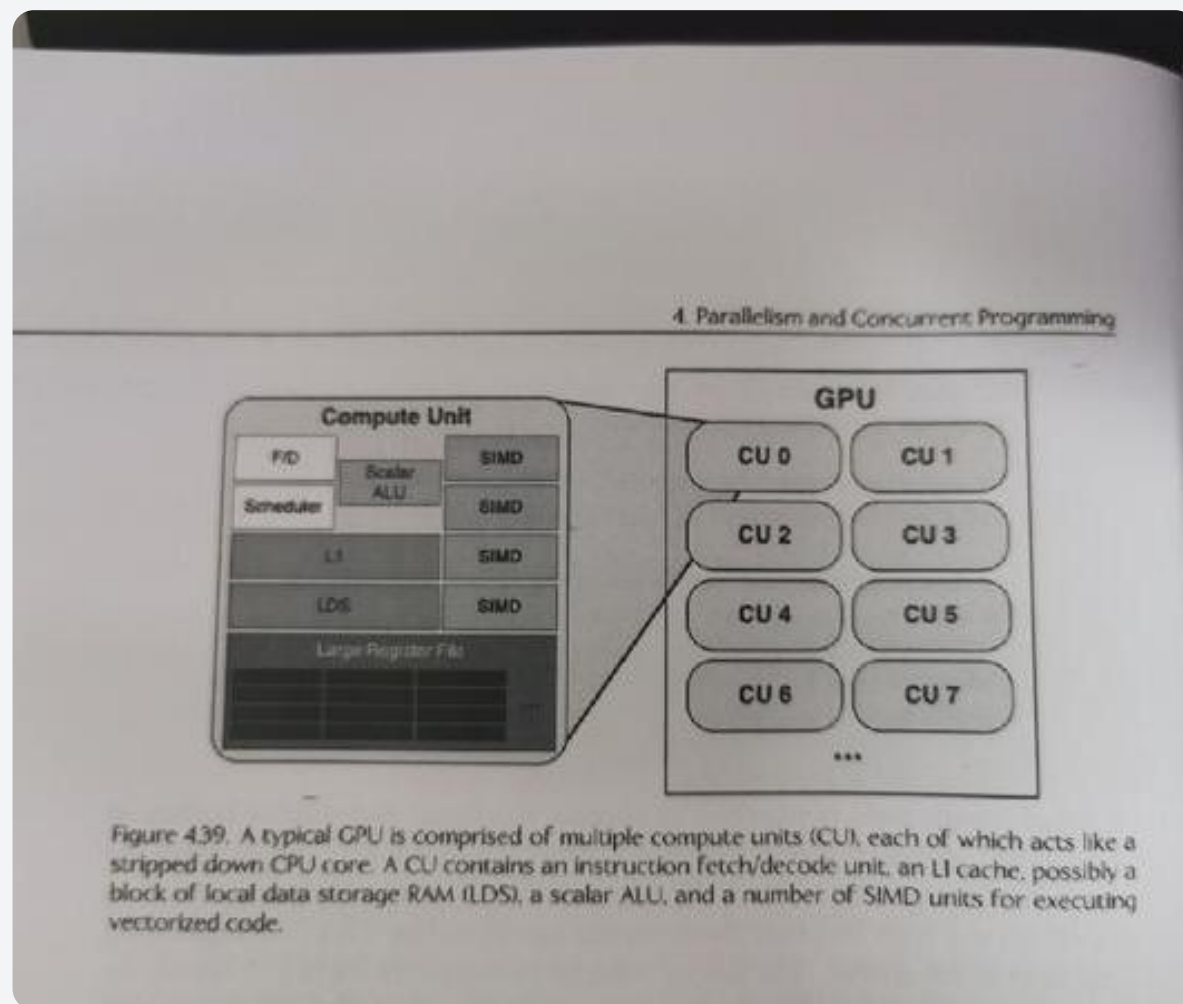
复杂度与并行化潜力

快速幂的核心优势在于其 $O(\log n)$ 的时间复杂度，空间复杂度仅为 $O(1)$ 。

虽然算法本身是串行的，但其底层的平方和乘法操作在现代硬件上仍有加速空间。

 GPU并行: 并行位扫描和SIMD批量平方可进一步降低常数因子。

 分布式计算: 在分块有序的大数据场景下，可并行处理后再汇总。



回顾与升华

平方乘思想：分治法的精髓

快速幂是“零合并成本”分治模型的典范。它揭示了分治法的核心：通过不断对折，将指数级增长的问题，转化为对数级规模的子问题。

二分搜索

区间折半

+

归并排序

任务折半

+

快速幂

值域折半

掌握快速幂，等于获得了“对数级缩小问题规模”的通用思维钥匙。



PART 04



大整数乘法的分治艺术



大整数乘法的效率瓶颈

当传统“竖式乘法”遭遇性能天花板



金融加密: 1024位密钥的生成与验证，耗时巨大。



天文计算: 高精度轨道模拟，乘法效率是核心瓶颈。



密码学: 大数分解和质数测试，速度至关重要。

性能鸿沟：1024位整数乘法对比

$O(n^2)$



传统算法

$O(n^{1.585})$



分治算法

随着位数增长，传统算法的耗时呈**爆炸式**增长，
而分治法则展现出巨大的性能优势。

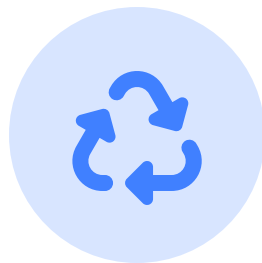
分治范式核心框架

“拆—治—合”三步流程



拆

将大整数按位权分裂为规模减半的子问题。



治

递归求解结构一致的子问题，直至基例。



合

将子问题解在线性时间内合并，注意处理进位。

分治法的正确性依赖于子问题与原问题结构一致，其复杂度由主定理精确刻画。

整数分裂与多项式视角

将数值运算映射到系数运算，为减少乘法次数提供代数基础

将n位整数X、Y各拆成高m位与低m位：

$$X = A \cdot 10^m + B$$

$$Y = C \cdot 10^m + D$$

这相当于把乘法转化为二次多项式求值。

关键点：

m的选取对平衡子问题规模至关重要。

- 需处理位数n为奇偶的不同拆分策略。

Karatsuba公式推导

$$X = \begin{array}{|c|c|} \hline A & B \\ \hline \end{array}$$

$$Y = \begin{array}{|c|c|} \hline C & D \\ \hline \end{array}$$

假设我们有两个8位数：X = 12345678 和 Y = 87654321。

• 分解：

- $X = 1234 * 10^4 + 5678$
- $Y = 8765 * 10^4 + 4321$

• 计算：

- $AC = 1234 \ 8765$
- $BD = 5678 \ 4321$
- $AD = 1234 \ 4321$
- $BC = 5678 \ 8765$

• 合并结果（这里简化，实际需要计算各项和并移位）：

- 最终结果 = $AC \ 10^8 + (AD + BC) \ 10^4 + BD$

三次乘法巧算四项积

用代数恒等式实现计算量的关键突破。

直接计算

展开XY得到四项:

$$AC \cdot 10^{2m} + (AD + BC) \cdot 10^m + BD$$

需要 4 次乘法



Karatsuba 技巧

利用恒等式:

$$(A + B)(C + D) = AC + AD + BC + BD$$

通过加减拼出 AD+BC

仅需 3 次乘法

核心洞察: 一次额外的加法代价远低于一次乘法, 这是复杂度突破的关键。

Karatsuba公式推导

递归深度与基例设计

递归基例 (Base Case)

当整数位数小于阈值T(如单字长) 时:

退化为硬件乘法或竖式乘法。

避免函数调用开销。

阈值T通过实验调优, 与缓存行、字长匹配。

递归深度与优化

递归深度为 $\log_2 n$:

需考虑栈空间占用。

尾递归不可优化。

可依赖显式栈或迭代版本控制内存峰值。

Python参考实现

合理的基例是递归算法的效率保障。

将算法思想转化为简洁的代码。

```
1. function Karatsuba(X, Y):
2.     if X < 10 or Y < 10:
3.         return X * Y
4.
5.     m = min(位数(X), 位数(Y)) // 2
6.
7.     A = X的高位部分
8.     B = X的低位部分
9.     C = Y的高位部分
10.    D = Y的低位部分
11.
12.    AC = Karatsuba(A, C)
13.    BD = Karatsuba(B, D)
14.    AD_BC = Karatsuba(A + B, C + D) - AC - BD
15.
16.    结果 = AC * 10^(2*m) + AD_BC * 10^m + BD
17.
18.    return 结果
```

平衡拆分与并行加速

挖掘算法潜力，追求极致性能。

平衡拆分策略

确保子问题规模差 ≤ 1 ，防止复杂度退化：

对于非2的幂长度，采用高位补齐。

或采用动态m选取策略。

保证子问题规模尽可能均衡。

并行计算加速

三次递归调用无数据依赖，可并行执行：

可丢入线程池或GPU并行计算。

合并阶段仅为线性加法，通信开销小。

在NUMA架构下需绑定内存节点。

常数优化与工程细节

理论到实践：工业级实现的考量



内存优化

预分配结果数组，避免重复拷贝。



算法切换

利用FFT-friendly基例切换至更高阶算法。



进制优化

对进制 10^k 改用 2^{64} ，减少除法开销。



SIMD加速

采用SIMD指令批处理加减法。

复杂度分析：突破 $O(n^2)$ 屏障

从递归式到渐进复杂度

递归式： $T(n) = 3T(n/2) + O(n)$



根据主定理 (Master Theorem), 其解为:

$$T(n) = O(n^{\log_2 3}) \approx O(n^{1.585})$$

加法线性项 $O(n)$ 被乘法项覆盖, 复杂度严格小于 $O(n^2)$ 。

传统算法
 $O(n^2)$

VS

Karatsuba
 $O(n^{1.585})$

更高阶算法速览

乘法算法的演进之路

Toom-Cook

将分裂段数 k 推广到任意，乘法降至 $2k-1$ 次。

$O(n^{1.465})$

Schönhage-Strassen

借助快速傅里叶变换(FFT)，达到拟线性复杂度。

$\tilde{O}(n \log n \log \log n)$

Fürer / HH

进一步压缩对数项，达到理论上的新高度。

$O(n \log n)$

注：尽管存在更高阶算法，但Karatsuba因其简单高效，仍是教学与实际中32k位内的首选。

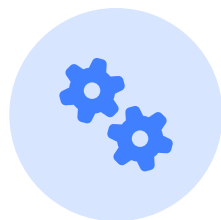
分治思维的价值回顾

超越算法本身：一种普适的问题解决范式



核心思想

“用代数技巧换计算量”的典范，启发众多高效算法。



普适价值

“拆一治一合”三步流程在排序、搜索等领域广泛应用。



实践启发

鼓励将“减少一次乘法”的思想迁移到日常性能优化。

体会算法设计的 **优雅** 与 **威力**，是算法学习中最宝贵的收获。

实践指引与下一步探索

从理论到代码，开启你的算法优化之旅

动手实践

在1024, 4096, 16384位规模下实测，绘制耗时曲线。
尝试将基例切换为GMP内联汇编，体验常数级提升。
阅读NTL, MIRACL等库的源码，理解混合算法调度。

GitHub: [your-karatsuba-repo](#)

深入研究

探索Toom-Cook算法，理解其通用性。
学习FFT原理，为理解Schönhage-Strassen算法打下基础。
研究并行算法设计，将Karatsuba应用于GPU。

推荐阅读:《现代计算机代数》



PART 05

$$\begin{bmatrix} c_{11} & c_{12} & \cdots & c_{1n} \\ c_{21} & c_{22} & \cdots & c_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ c_{n1} & c_{n2} & \cdots & c_{nn} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix} \cdot \begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1n} \\ b_{21} & b_{22} & \cdots & b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ b_{n1} & b_{n2} & \cdots & b_{nn} \end{bmatrix}$$

$$c_{ij} = \sum_{k=1}^n a_{ik} \cdot b_{kj}$$

Standard algorithm

```
for  $i \leftarrow 1$  to  $n$ 
  do for  $j \leftarrow 1$  to  $n$ 
    do  $c_{ij} \leftarrow 0$ 
      for  $k \leftarrow 1$  to  $n$ 
        do  $c_{ij} \leftarrow c_{ij} + a_{ik} \cdot b_{kj}$ 
```

Running time = $\Theta(n^3)$

Strassen算法： 分治矩阵乘法



矩阵乘法困局

朴素矩阵乘法： $O(n^3)$ 的性能瓶颈

传统三重循环算法在应对大规模数据时，面临三重性能局限。



缓存命中率低

按行/列遍历导致频繁的缓存失效，内存带宽成为瓶颈，处理器空转等待数据。



并行扩展性差

内层循环间存在数据依赖，难以被向量化，无法充分利用现代CPU的多核与SIMD特性。

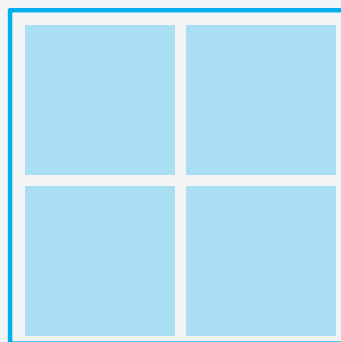


计算量立方级膨胀

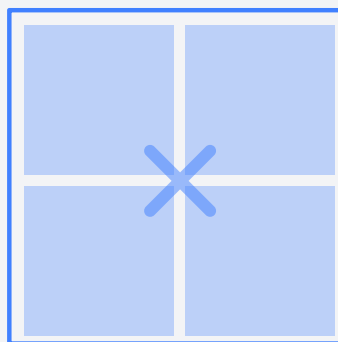
当矩阵维度 n 达到数千时， $O(n^3)$ 的计算量使得运算时间变得难以承受。

分治策略初探：从二分开始

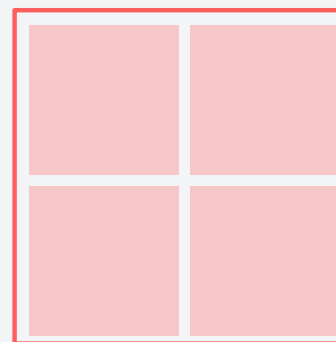
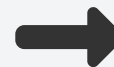
将矩阵一分为四，递归处理子块，是突破的第一步。但为何单纯的二分无法降低复杂度？



A x B



8次 $n/2 \times n/2$ 乘法



结果矩阵 C

$n \times n$ matrix = 2×2 matrix of $(n/2) \times (n/2)$ submatrices:

$$r = ae + bg$$

$$s = af + bh$$

$$t = ce + dh$$

$$u = cf + dg$$

$$\begin{bmatrix} r & s \\ t & u \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \cdot \begin{bmatrix} e & f \\ g & h \end{bmatrix}$$

8 mults of $(n/2) \times (n/2)$ submatrices

4 adds of $(n/2) \times (n/2)$ submatrices

结论：根据主定理，8次递归调用 和 $O(n^2)$ 的合并代价，使得总复杂度仍为 $O(n^3)$ 。

Strassen的突破：用7次乘法替代8次

通过精心设计的线性组合，Strassen算法将乘法次数降至7次，从而突破了 $O(n^3)$ 的壁垒。

计算7个中间积 (P1-P7)

$$P_1 = A_{11}(B_{12} - B_{22})$$

$$P_2 = (A_{11} + A_{12})B_{22}$$

$$P_3 = (A_{21} + A_{22})B_{11}$$

$$P_4 = A_{22}(B_{21} - B_{11})$$

$$P_5 = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$P_6 = (A_{12} - A_{22})(B_{21} + B_{22})$$

$$P_7 = (A_{11} - A_{21})(B_{11} + B_{12})$$

组合出结果矩阵的四个子块

$$\begin{aligned} C_{11} \\ = P_5 + P_4 - P_2 + P_6 \end{aligned}$$

$$\begin{aligned} C_{12} \\ = P_1 + P_2 \end{aligned}$$

$$\begin{aligned} C_{21} \\ = P_3 + P_4 \end{aligned}$$

$$\begin{aligned} C_{22} \\ = P_5 + P_1 - P_3 - P_7 \end{aligned}$$

加法次数增至18次，但其 $O(n^2)$ 成本远低于乘法，整体仍获优势。

递归流程与复杂度分析

Strassen算法的成功体现在其优雅的递推式中，它决定了算法的时间复杂度。



分解

将问题分解为7个规模为 $n/2$ 的子问题。

+



合并

通过 $\Theta(n^2)$ 的矩阵加减法合并子问题结果。

=

递推式

$$T(n) = 7T\left(\frac{n}{2}\right) + \Theta(n^2)$$

应用主定理，最终时间复杂度为： $O(n^{\log_2 7}) \approx O(n^{2.81})$

这是算法史上首次突破 $O(n^3)$ 的里程碑，确立了其在理论计算机科学史上的重要地位。

边界与实现

跨越截断阈值：混合策略

当子矩阵规模小于某个阈值时，继续递归反而变慢。这时需要切换到更高效的基例算法。



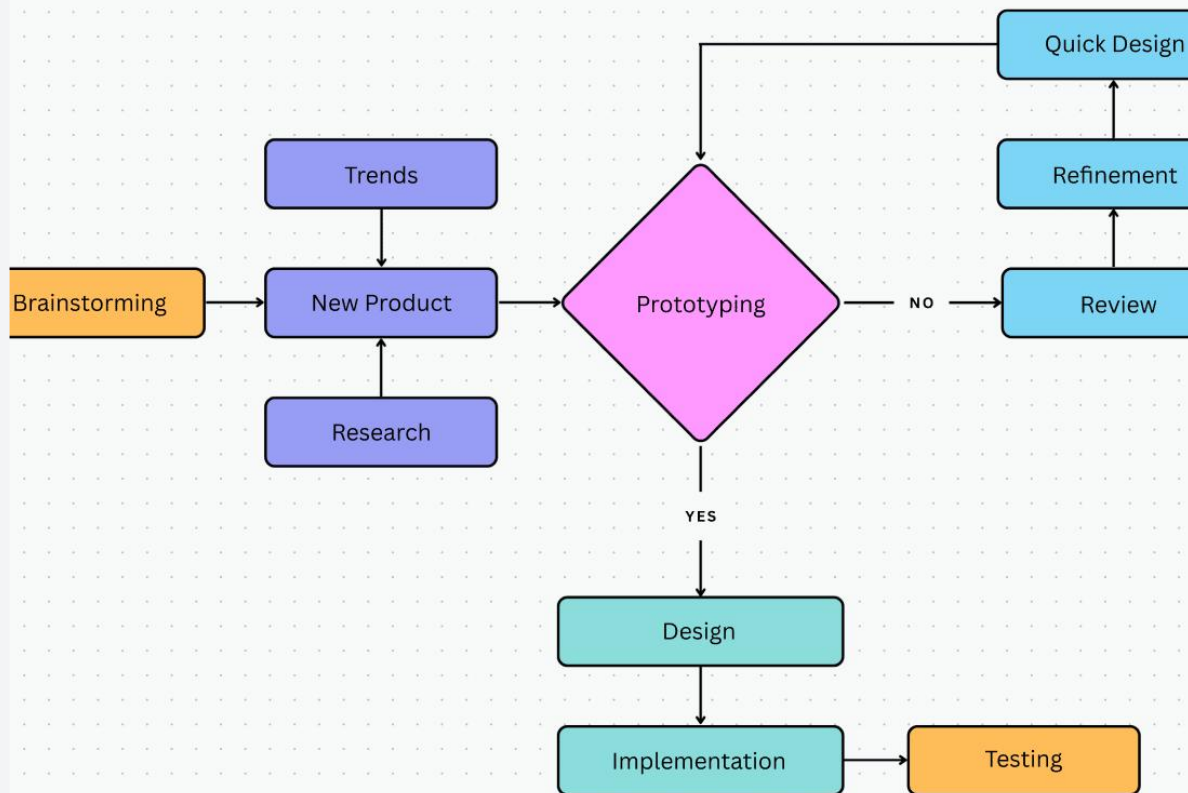
常数因子膨胀

递归调用本身有开销，当问题规模小于 64×64 或 128×128 时，朴素算法或BLAS库更优。



缓存不友好

小规模递归可能破坏数据局部性，导致缓存失效，抵消算法优势。



数值稳定性：误差的来源与传播

Strassen算法中大量的加减法操作会放大浮点运算的舍入误差，这是其在工程应用中需要重点关注的问题。



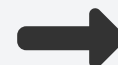
1. 初始误差

浮点数本身的精度限制引入初始舍入误差。



2. 误差放大

算法中的加减法组合(如P1-P7的计算)会放大这些误差。



3. 结果偏差

尤其对于病态矩阵，最终结果的精度可能显著低于传统算法。

实践中需权衡性能与精度，或采用动态精度扩展等技术保证可靠性。

并行与缓存

缓存友好的数据布局

递归算法容易导致缓存失效。通过优化数据存储和访问模式，可以显著提升性能。

行主序 (Row-major)

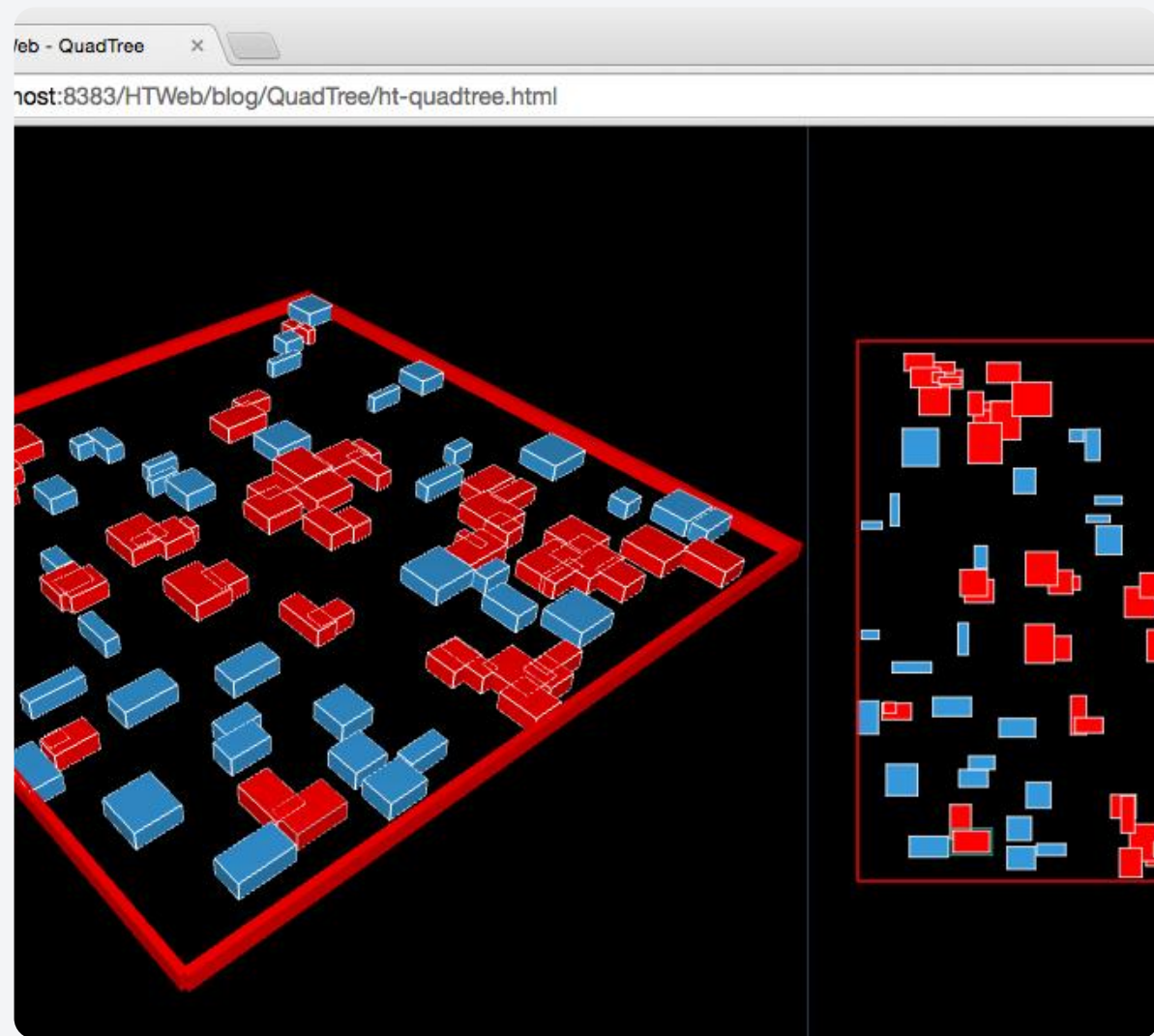
C/C++默认布局，对递归分块不友好。

块递归 (Block-recursive / Morton序)

将逻辑上相邻的子块在内存中也连续存储，极大提升空间局部性，是Strassen的理想选择。

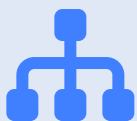
四叉树平铺 (Quad-tree Tiling)

一种实现Morton序的有效方式，通过指针或位运算实现分块访问。



共享内存并行：任务树拆分与工作窃取

Strassen算法的7个子问题相互独立，天然适合并行化。关键在于如何高效地调度和管理任务。



任务队列与Fork-Join

将7个乘法任务放入队列，由线程池中的工作线程并行执行。使用fork-join模型等待所有子任务完成。



工作窃取 (Work Stealing)

使用OpenMP或TBB等库，实现动态负载均衡。空闲线程可以从忙碌线程的任务队列中“窃取”任务，最大化CPU利用率。

挑战：合并阶段存在数据依赖，需小心设计以避免 **伪共享 (False Sharing)**。在NUMA架构下，绑定内存节点可进一步提升性能。

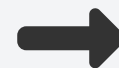
理论极限的探索：从Strassen到Coppersmith-Winograd

Strassen的突破激励了后续研究，算法的时间复杂度理论上可以被推进到更低，但这些算法目前仅具理论价值。

Strassen (1969)
 $O(n^{2.81})$



Coppersmith-Winograd (1990)
 $O(n^{2.376})$



后续改进
 $O(n^{2.372864})$

巨大的常数因子

这些算法隐藏了巨大的常数，导致其只有在难以想象的大矩阵上才能体现出优势。因此，它们目前主要存在于理论计算机科学的研究中。

深度学习中的结构化加速

深度学习中的核心操作——卷积，可以被重写为Toeplitz矩阵乘法，为Strassen算法提供了新的应用场景。



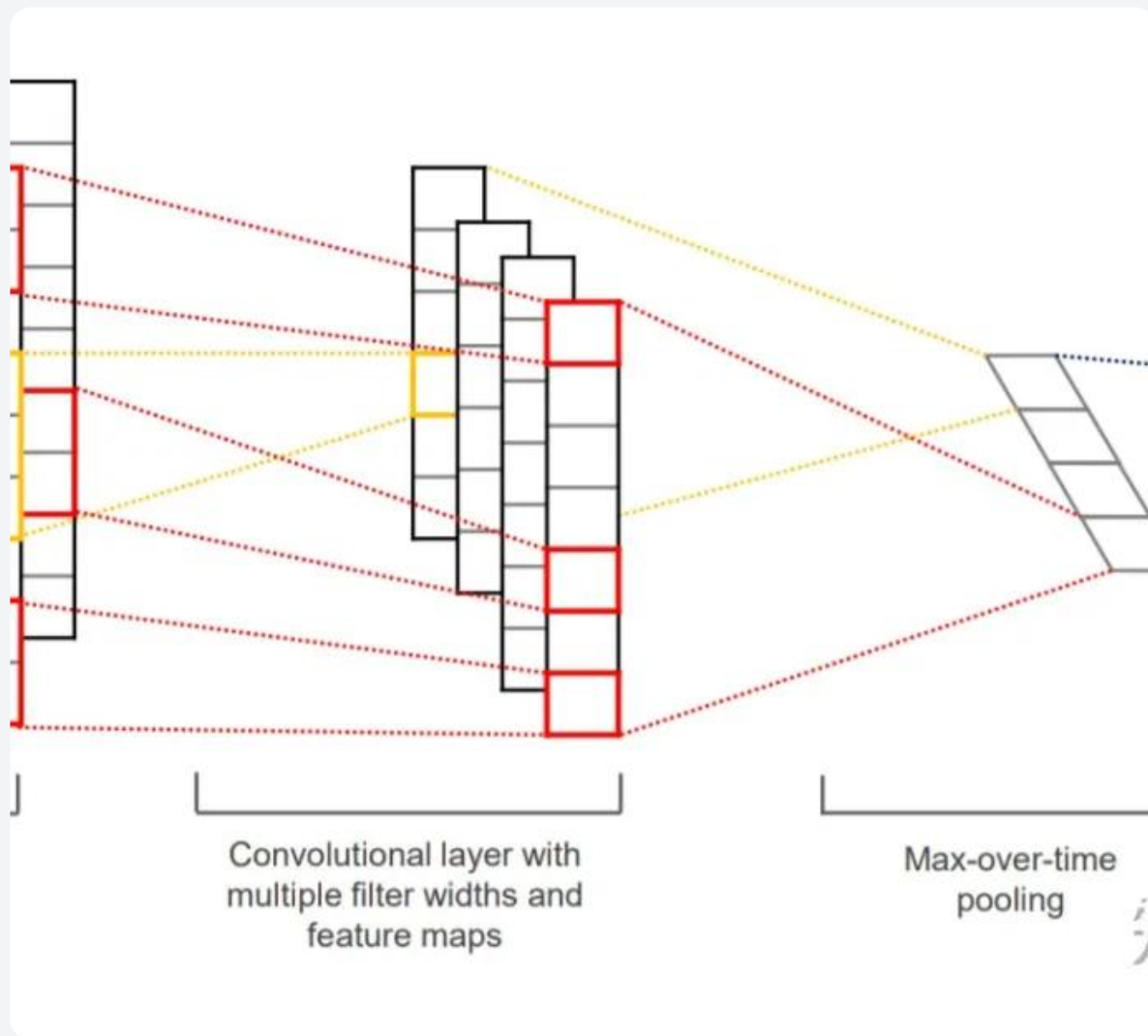
卷积即矩阵乘法

通过im2col等操作，卷积运算被转化为通用矩阵乘法 (GEMM)，这是Strassen可以发挥作用的地方。



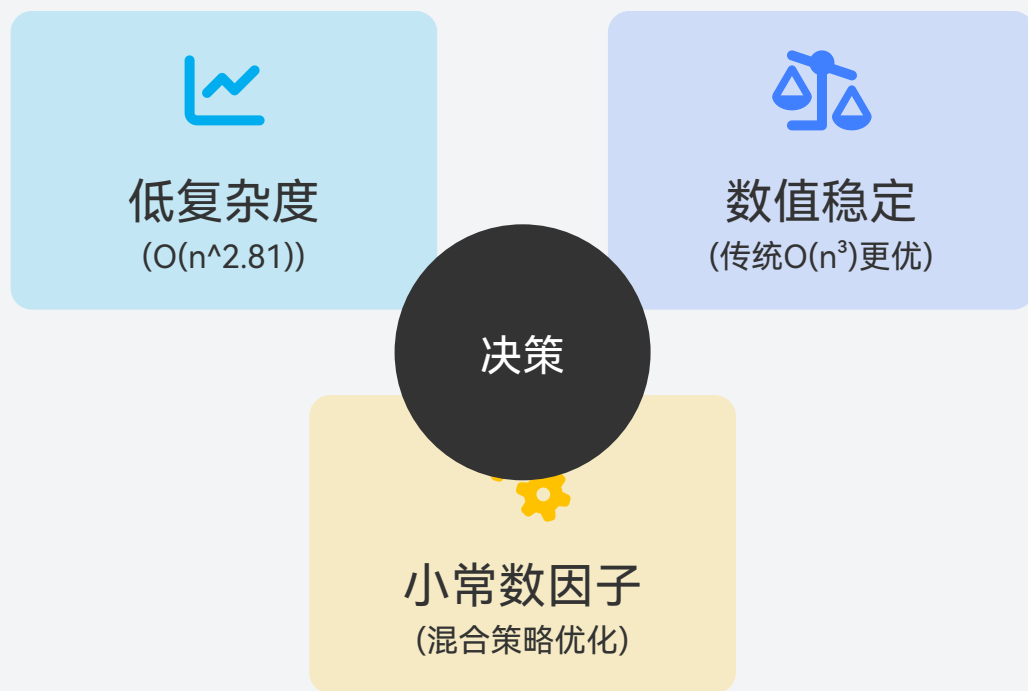
GPU上的机遇

当Batch Size足够大时，Strassen的访存模式可与GPU的张量核心 (Tensor Cores) 融合，获得额外加速。



算法选择的黄金三角：复杂度、常数与稳定性

Strassen算法教会我们，一个优秀的算法需要在理论复杂度、实际常数因子和数值稳定性之间取得精妙平衡。



决策应基于：问题规模 + 硬件特性 + 精度要求

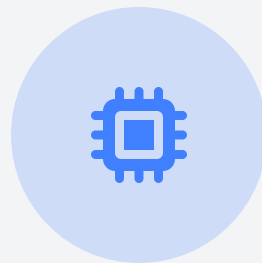
未来方向：自适应算法与硬件共生

算法与硬件的协同演进是持续加速的关键。未来的矩阵乘法算法将更加智能和自适应。



自适应算法选择

运行时根据矩阵规模、硬件负载和精度要求，动态选择最优算法(如在不同递归层切换策略)。



硬件感知优化

为特定架构(如TPU、NPU) 定制算法变体，利用其独特的内存层次和计算单元。



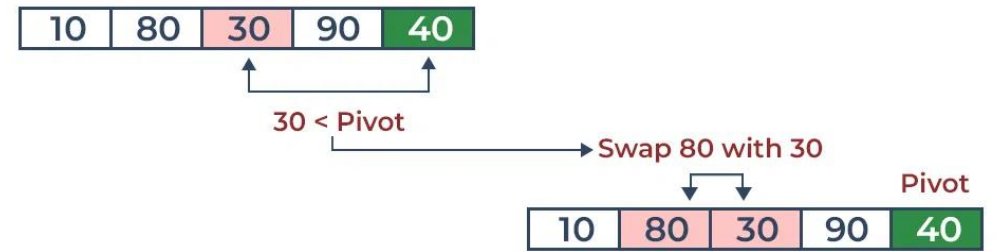
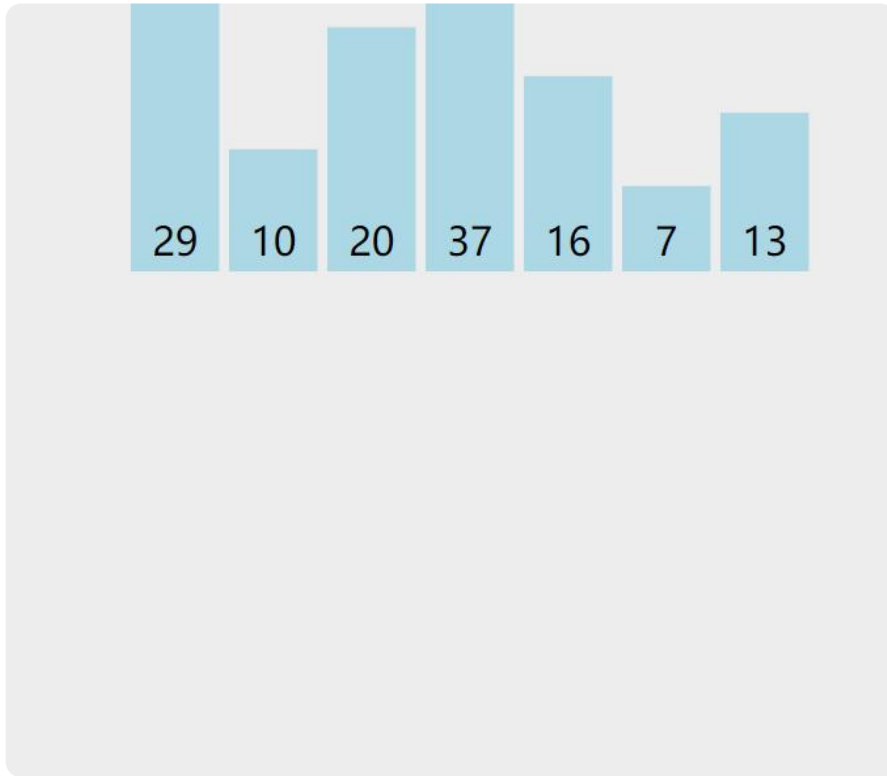
运行时编译与融合

利用JIT技术，在运行时生成针对当前任务特化的代码，并与前后操作融合，消除内存瓶颈。

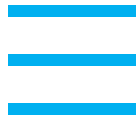
在Strassen的精神下，继续探索矩阵乘法的下一条**加速曲线**。



PART 06



分治思想与排序概览



分治双雄：合并与快排

分治范式：拆分-治理-合并三步走

将原问题拆为同质子问题，递归求解后合并结果，典型复杂度满足 $T(n)=aT(n/b)+f(n)$

合并排序 (Merge Sort)

- ✓ **稳定**: 相等元素相对次序不变。
- 🛡 **最坏最优**: 复杂度恒为 $\Theta(n \log n)$ ，无性能陷阱。

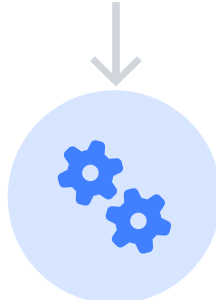
快速排序 (Quick Sort)

- 🚀 **平均最快**: 平均复杂度 $\Theta(n \log n)$ ，常数因子小。
- 🔥 **就地排序**: 仅需 $O(\log n)$ 栈空间，内存友好。

递归流程：先分后治再合并



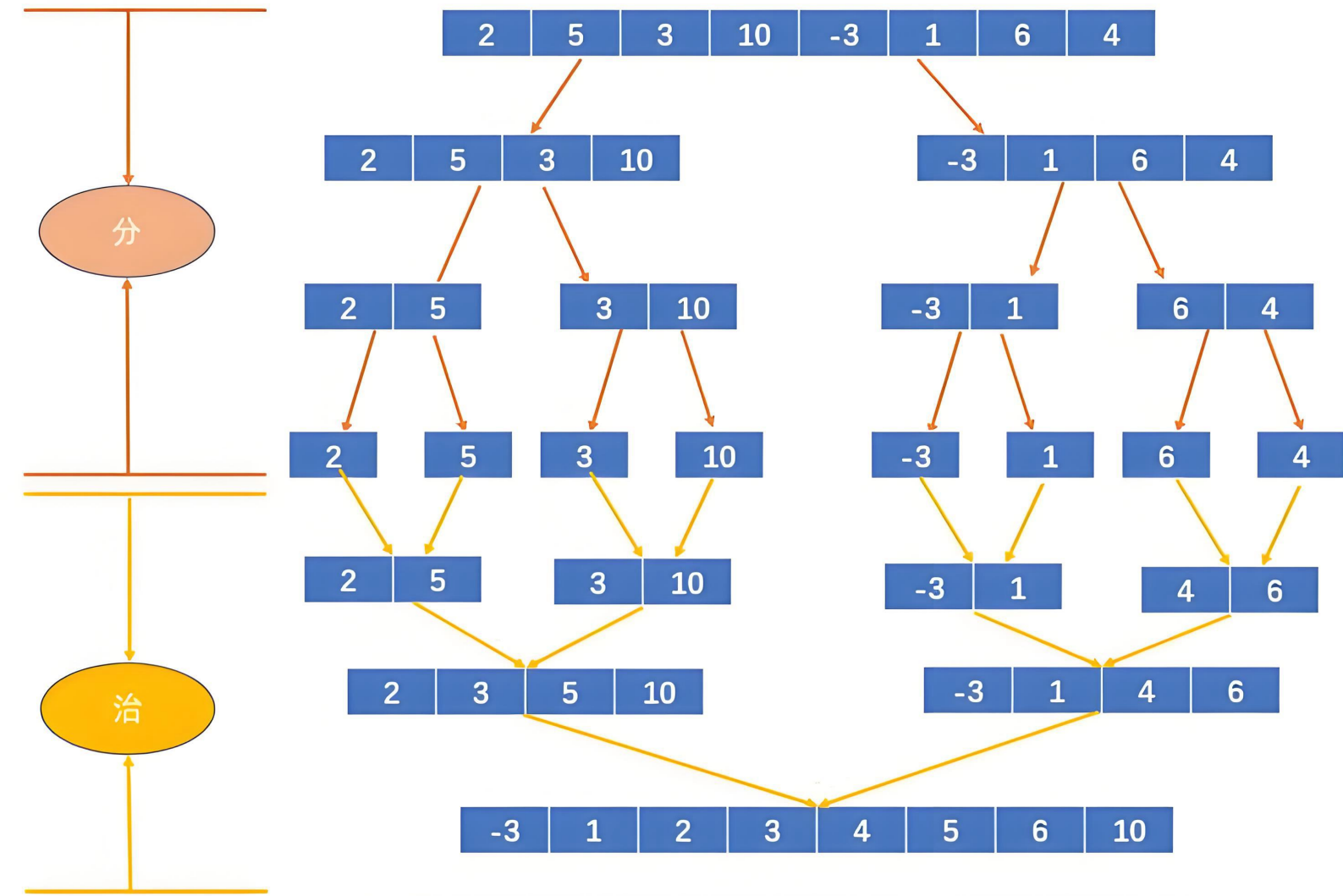
1. 分解 (Divide)
将数组平分为左右两半。



2. 解决 (Conquer)
递归地对两个子数组排序。



3. 合并 (Merge)
将两个有序子数组合并。



递归流程：先分后治再合并

```
1. def merge_sort(arr):
2.     # 如果数组长度小于等于1, 直接返回 (基本情况)
3.     if len(arr) <= 1:
4.         return arr
5.
6.     # 分解: 找到中间点, 将数组分为左右两部分
7.     mid = len(arr) // 2
8.     left_half = arr[:mid]
9.     right_half = arr[mid:]
10.
11.    # 递归解决: 对左右两部分分别进行合并排序
12.    left_sorted = merge_sort(left_half)
13.    right_sorted = merge_sort(right_half)
14.
15.    # 合并: 将两个有序数组合并为一个有序数组
16.    return merge(left_sorted, right_sorted)
17.
```

```
18. def merge(left, right):
19.     result = []
20.     i = j = 0
21.
22.     # 比较左右两个数组的元素, 依次将较小的元素加入结果数组
23.     while i < len(left) and j < len(right):
24.         if left[i] < right[j]:
25.             result.append(left[i])
26.             i += 1
27.         else:
28.             result.append(right[j])
29.             j += 1
30.
31.     # 将剩余的元素直接加入结果数组
32.     result.extend(left[i:])
33.     result.extend(right[j:])
34.
35.     return result
```

复杂度分析

$$T(n) = \begin{cases} O(1) & n \leq 1 \\ 2T(n/2) + O(n) & n > 1 \end{cases}$$

$T(n)=O(n\log n)$ 渐进意义下的最优算法

复杂度与稳定性深度剖析



性能稳定

最坏、平均、最好情况均为 $\Theta(n \log n)$ ，
无输入敏感风险。



稳定排序

相等元素在合并时相对次序得以保持，
适用于需要稳定性的场景。



空间代价

需要 $\Theta(n)$ 辅助空间，但可预测，无
栈溢出隐忧。

核心操作：合并 (Merge)

使用双指针扫描，将较小元素放入辅助数组，最后拷贝回原数组。时间复杂度 $\Theta(n)$ 。递归式 $T(n)=2T(n/2)+\Theta(n)$ ，由主定理得 $\Theta(n \log n)$ 。空间复杂度 $\Theta(n)$ 。

适用场景： 因其稳定性和可预测的复杂度，归并排序是 **大数据外部排序** 和 **并行计算** 的优选。

非递归合并排序

自底向上归并：从单元到全数组

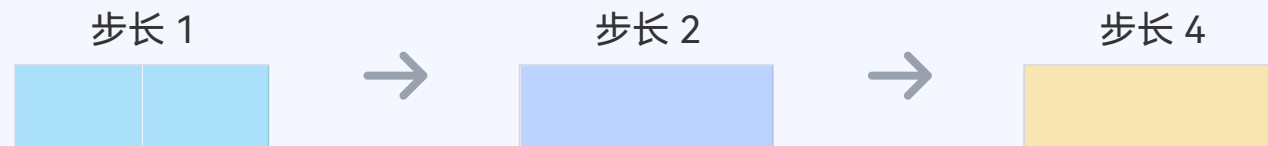
核心思想

取消系统栈，用 **显式循环** 控制子区间长度。
初始步长为1，每轮 **倍增**，模拟递归的每一层。

优势

- 消除递归调用，降低函数切换开销。
- 适合栈空间受限的嵌入式环境。
- 结构更清晰，易于并行化改造。

迭代过程示意



每轮将相邻的两个已排序子区间合并为一个更大的有序区间。

初始序列

[49] [38] [65] [97] [76] [13] [27]

第一步

[38 49] [65 97] [13 76] [27]

第二步

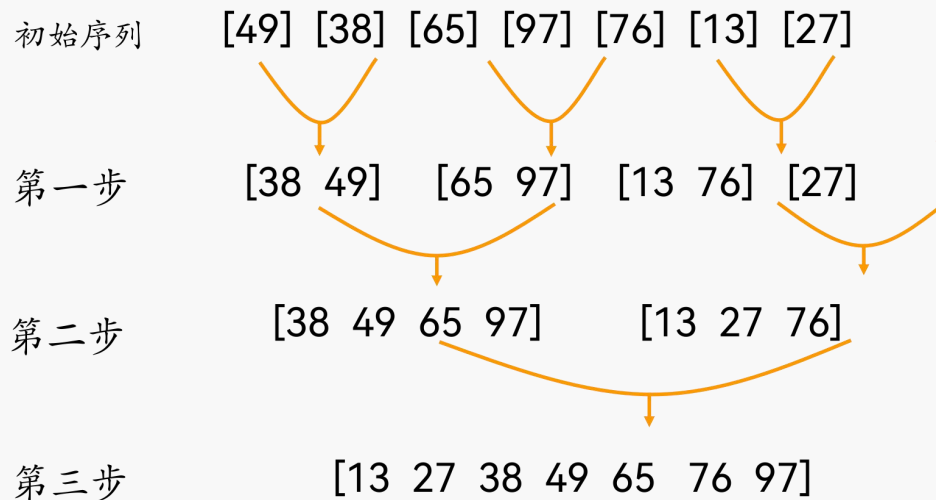
[38 49 65 97] [13 27 76]

第三步

[13 27 38 49 65 76 97]

显式栈与缓存优化技巧

```
1. def merge_sort_non_recursive(arr):
2.     n = len(arr)
3.     size = 1
4.     while size < n:
5.         for start in range(0, n, 2*size):
6.             mid = min(start + size, n)
7.             end = min(start + 2*size, n)
8.             # 合并 arr[start:mid] 和 arr[mid:end]
9.             left = arr[start:mid]
10.            right = arr[mid:end]
11.            i = j = 0
12.            k = start
13.            while i < len(left) and j < len(right):
14.                if left[i] < right[j]:
15.                    arr[k] = left[i]
16.                    i += 1
17.                else:
18.                    arr[k] = right[j]
19.                    j += 1
20.                k += 1
21.            while i < len(left):
22.                arr[k] = left[i]
23.                i += 1
24.            k += 1
25.            while j < len(right):
26.                arr[k] = right[j]
27.                j += 1
28.            k += 1
29.        size *= 2
30.    return arr
```



逆序对问题与暴力枚举代价

逆序对 (Inversion)

在数组中，如果存在 $i < j$ 但 $a[i] > a[j]$ 的有序对，则其为一个逆序对。其总量衡量了数据的混乱程度。

双重循环枚举所有对，时间复杂度高达 $\Theta(n^2)$ ，无法应对大规模数据。

暴力枚举的局限

示例: 数组 [3, 1, 4, 2]

3	1	4	2
下标 0	下标 1	下标 2	下标 3

逆序对有: (3,1), (3,2), (4,2)

总计: 3 对

归并过程中统计逆序对技巧

合并阶段统计

当右侧元素 $R[j]$ 被优先取出时，它小于左侧指针 $L[i]$ 及其后的所有元素。

L: [1, 5, 6, 8]

&

R: [3, 4, 7, 9]

此时，3 与 L 中的 6, 8 构成逆序对。逆序对数量直接累加 L 的剩余长度。

核心思想

利用归并排序 **已排序的子数组** 特性，在 $O(1)$ 时间内统计跨越左右子数组的逆序对。

复杂度

时间: $\Theta(n \log n)$, 空间: $\Theta(n)$ 。

应用

可用于计算 Kendall tau 距离、Spearman 秩相关等。



快速排序核心机制

快速排序核心：枢纽选取与分区

枢纽元 (Pivot)

选取一个枢纽元，将数组划分为小于、等于、大于三部分。

分区 (Partition)

采用双指针法（如Hoare或Lomuto）进行原地分区，是快排的核心操作。

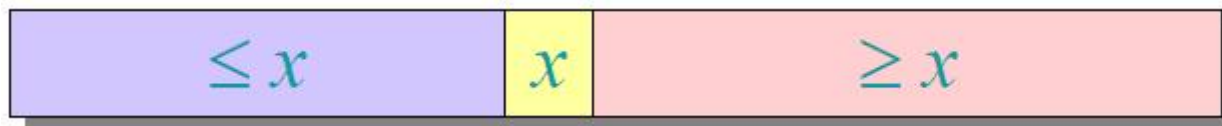
分区过程示意 (Hoare)

[pivot, 1, 4, 2, 5, 8, 7]

初始化 i, j 指针。

[2, 1, 4, pivot, 5, 8, 7]

指针相遇，交换 pivot 到最终位置。



平均复杂度 $T(n)=2T(n/2)+\Theta(n) = \Theta(n \log n)$ ，最坏 $\Theta(n^2)$ （如已排序数组）。

Pseudocode for quicksort

QUICKSORT(A, p, r)

if $p < r$

then $q \leftarrow \text{PARTITION}(A, p, r)$

 QUICKSORT($A, p, q-1$)

 QUICKSORT($A, q+1, r$)

Initial call: QUICKSORT($A, 1, n$)

Example of partitioning

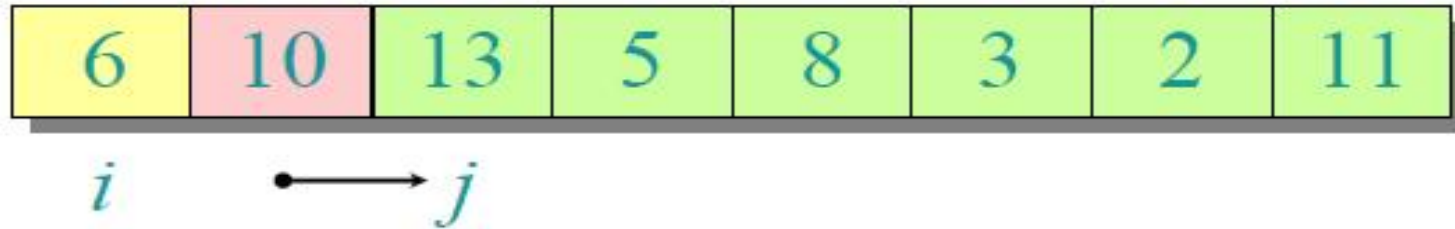
6	10	13	5	8	3	2	11
---	----	----	---	---	---	---	----

$x \leftarrow A[p]$ i j
 $i \leftarrow p$ $\triangleright$ pivot = $A[p]$

do if $A[j] \leq x$
then $i \leftarrow i + 1$
exchange $A[i] \leftrightarrow A[j]$

x	$\leq x$	$\geq x$	?
p	i	j	q

Example of partitioning



do if $A[j] \leq x$

then $i \leftarrow i + 1$

exchange $A[i] \leftrightarrow A[j]$

Example of partitioning

6	10	13	5	8	3	2	11
---	----	----	---	---	---	---	----

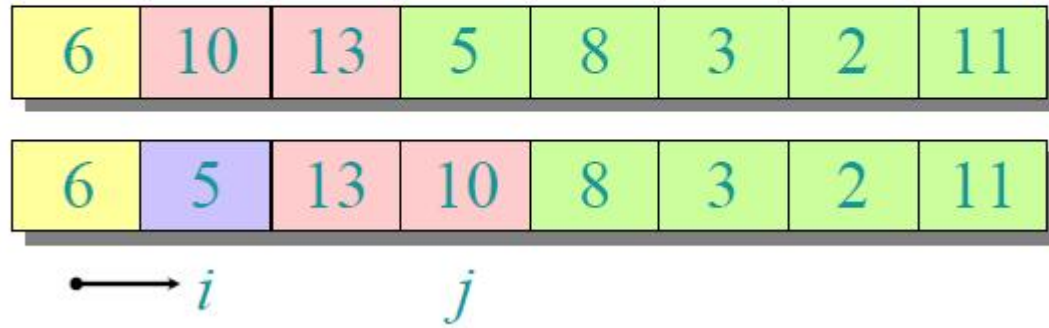
i $\bullet \longrightarrow j$

do if $A[j] \leq x$

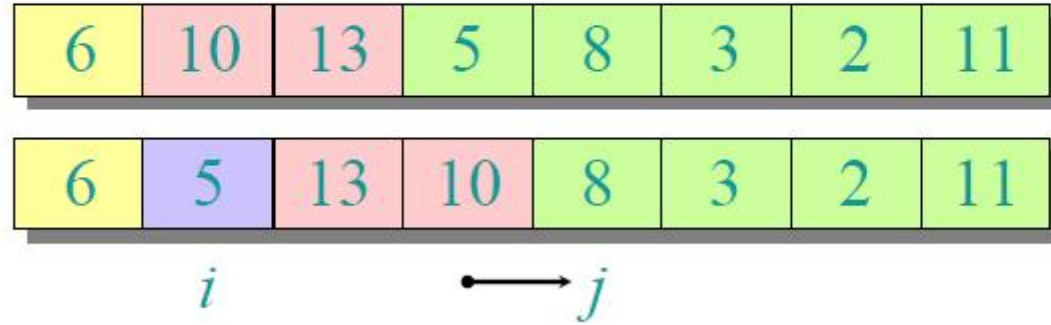
then $i \leftarrow i + 1$

exchange $A[i] \leftrightarrow A[j]$

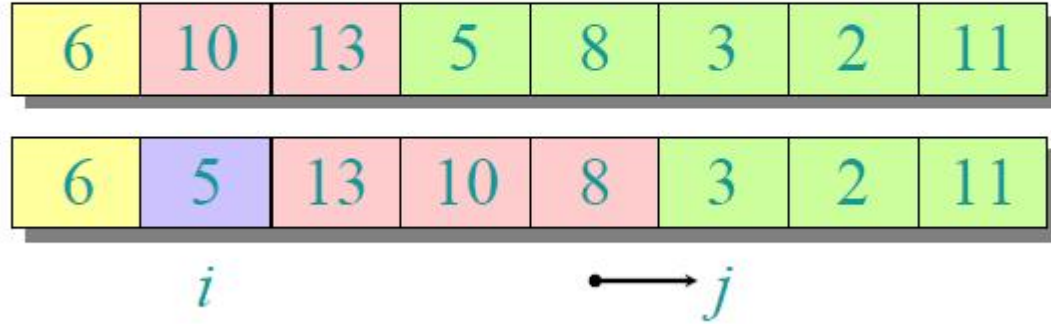
Example of partitioning



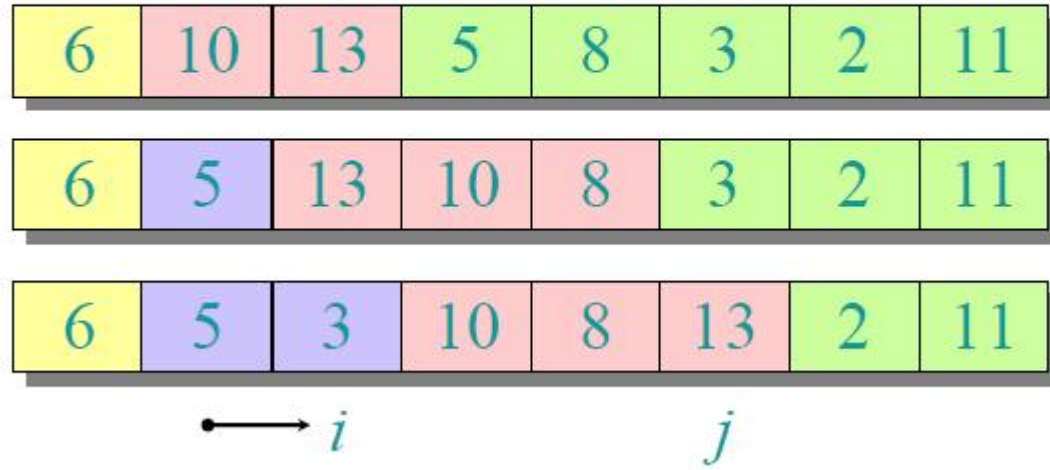
Example of partitioning



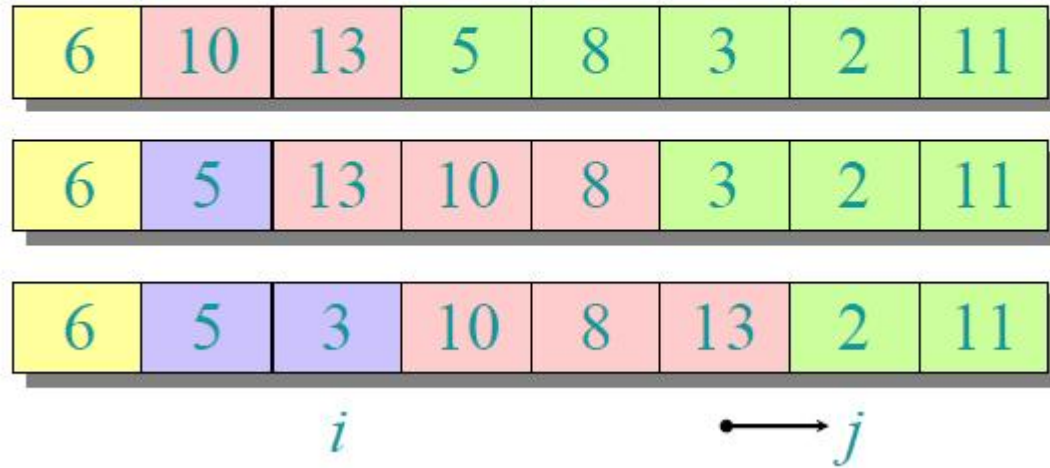
Example of partitioning



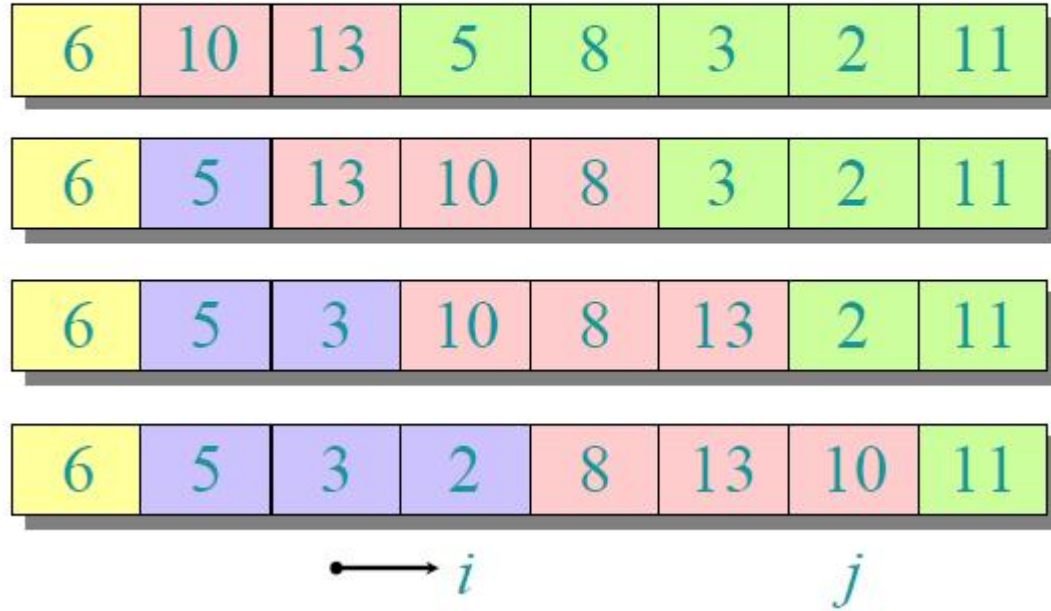
Example of partitioning



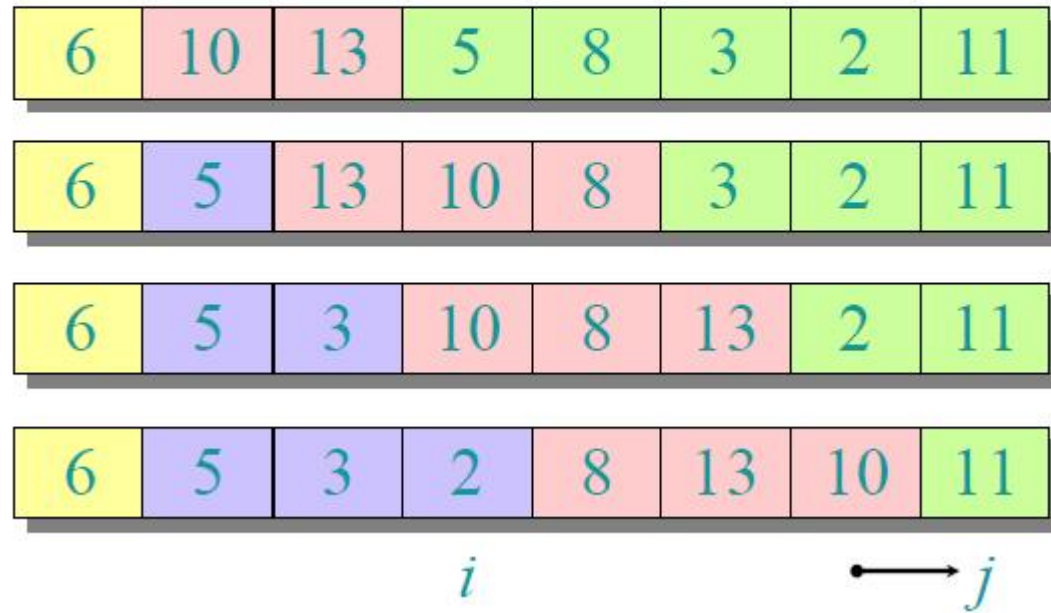
Example of partitioning



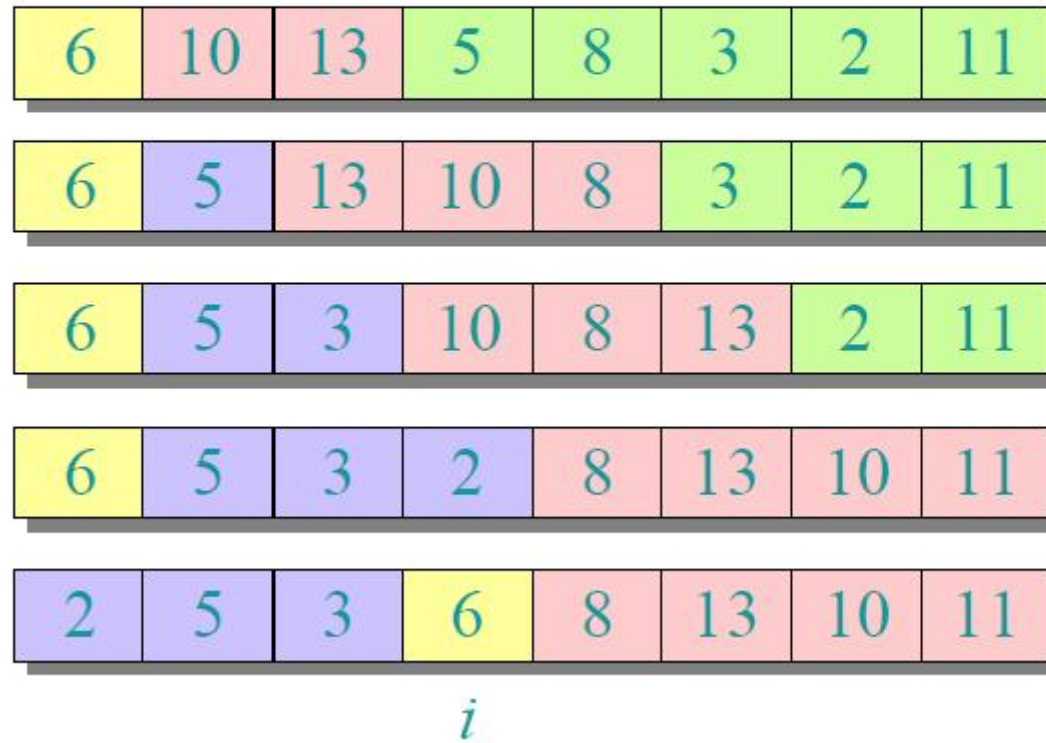
Example of partitioning



Example of partitioning



Example of partitioning

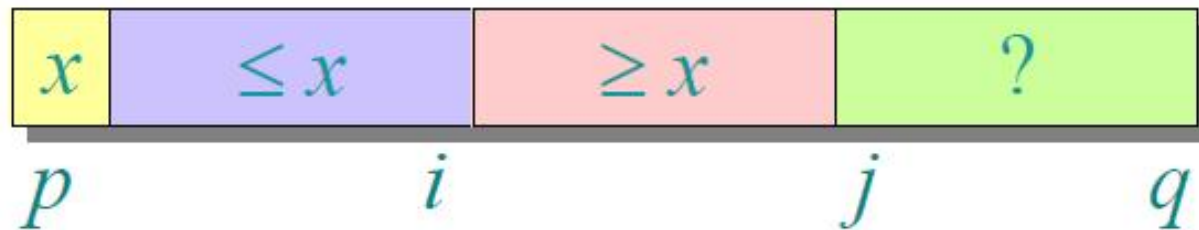


Partitioning subroutine

```
PARTITION( $A, p, q$ )  $\triangleright A[p \dots q]$   
   $x \leftarrow A[p]$   $\triangleright$  pivot =  $A[p]$   
   $i \leftarrow p$   
  for  $j \leftarrow p + 1$  to  $q$   
    do if  $A[j] \leq x$   
      then  $i \leftarrow i + 1$   
           exchange  $A[i] \leftrightarrow A[j]$   
  exchange  $A[p] \leftrightarrow A[i]$   
  return  $i$ 
```

Running time
= $O(n)$ for n
elements.

Invariant:



Pseudocode for quicksort

QUICKSORT(A, p, r)

if $p < r$

then $q \leftarrow \text{PARTITION}(A, p, r)$

 QUICKSORT($A, p, q-1$)

 QUICKSORT($A, q+1, r$)

Initial call: QUICKSORT($A, 1, n$)

Analysis of quicksort

- Assume all input elements are distinct.
- In practice, there are better partitioning algorithms for when duplicate input elements may exist.
- Let $T(n)$ = worst-case running time on an array of n elements.

Worst-case of quicksort

- Input sorted or reverse sorted.
- Partition around min or max element.
- One side of partition always has no elements.

$$T(n) = T(0) + T(n-1) + \Theta(n)$$

$$= \Theta(1) + T(n-1) + \Theta(n)$$

$$= T(n-1) + \Theta(n)$$

$$= \Theta(n^2) \quad (\textit{arithmetic series})$$

Worst-case recursion tree

$$T(n) = T(0) + T(n-1) + cn$$

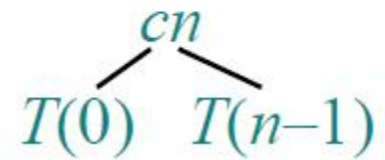
Worst-case recursion tree

$$T(n) = T(0) + T(n-1) + cn$$

$$T(n)$$

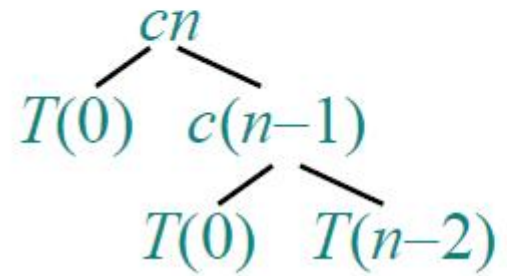
Worst-case recursion tree

$$T(n) = T(0) + T(n-1) + cn$$



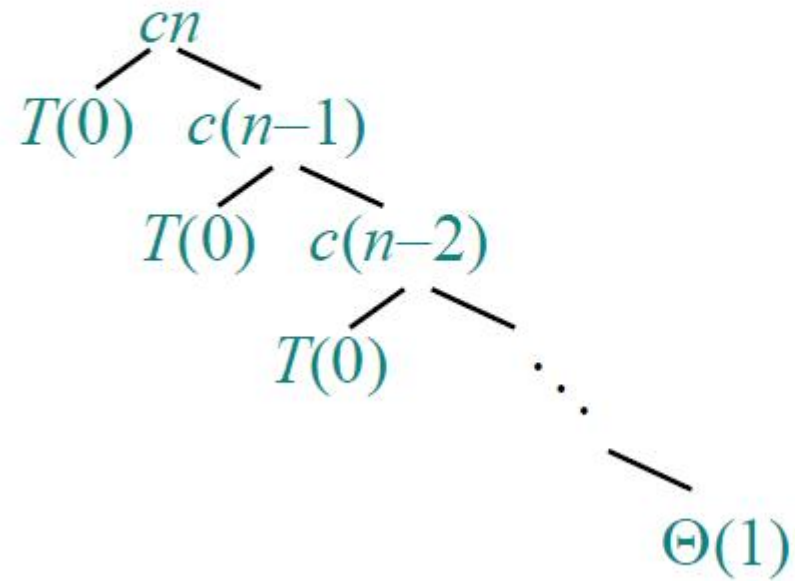
Worst-case recursion tree

$$T(n) = T(0) + T(n-1) + cn$$



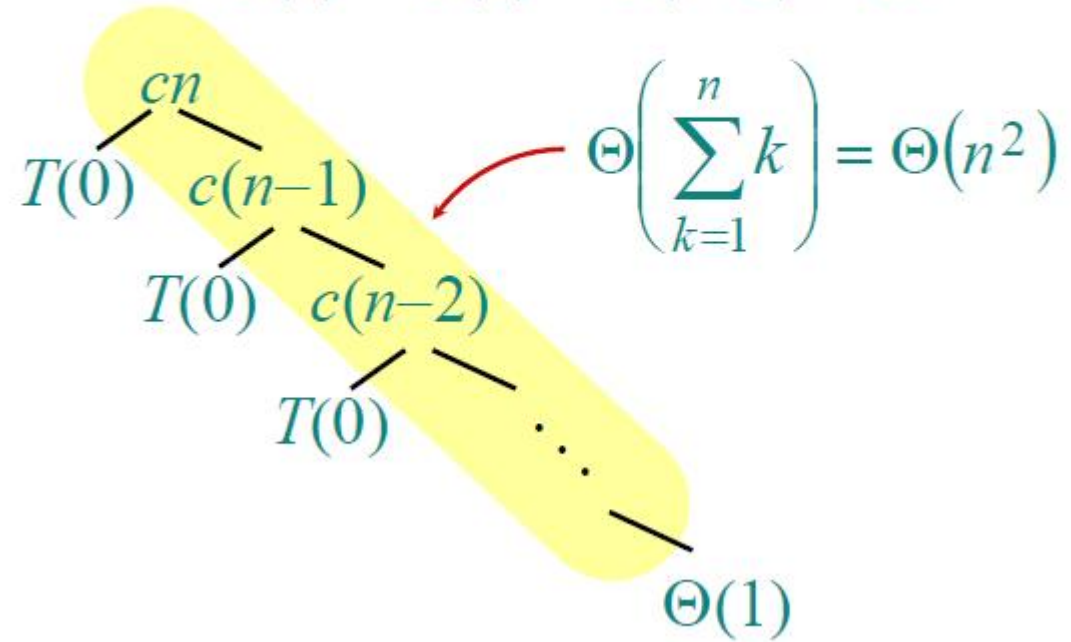
Worst-case recursion tree

$$T(n) = T(0) + T(n-1) + cn$$



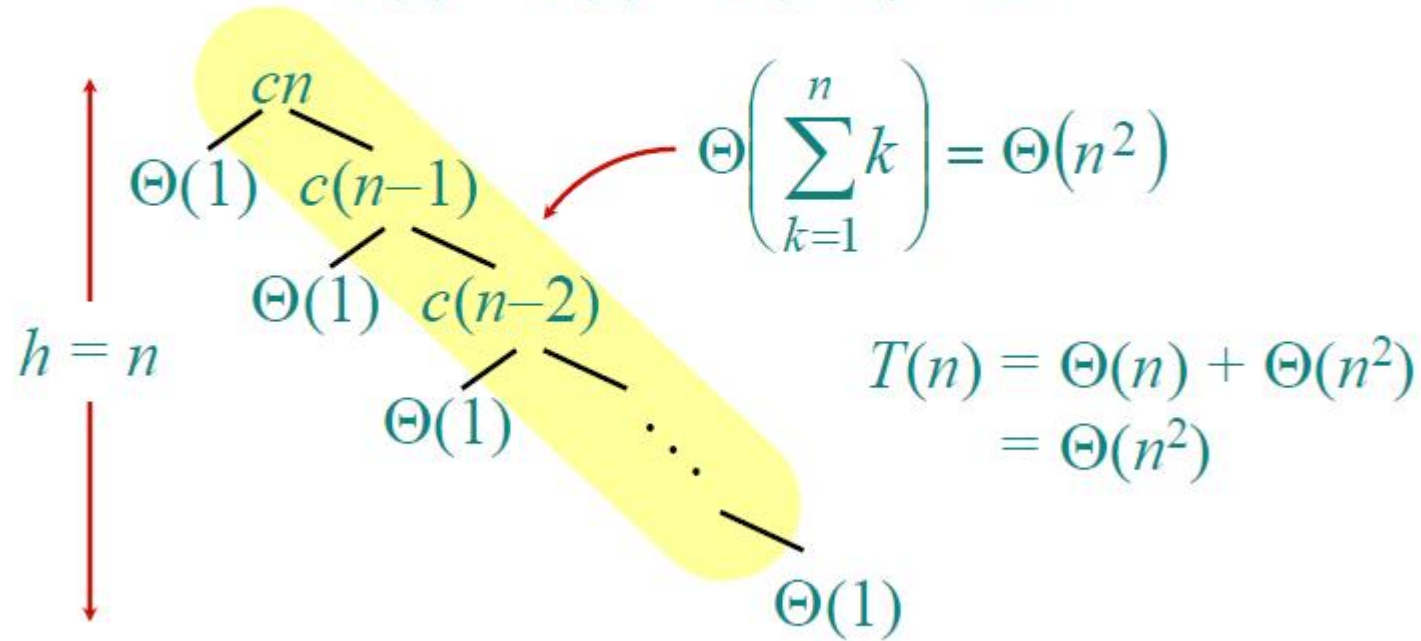
Worst-case recursion tree

$$T(n) = T(0) + T(n-1) + cn$$



Worst-case recursion tree

$$T(n) = T(0) + T(n-1) + cn$$



Best-case analysis

(For intuition only!)

If we're lucky, PARTITION splits the array evenly:

$$\begin{aligned}T(n) &= 2T(n/2) + \Theta(n) \\ &= \Theta(n \lg n) \quad (\text{same as merge sort})\end{aligned}$$

What if the split is always $\frac{1}{10} : \frac{9}{10}$?

$$T(n) = T\left(\frac{1}{10}n\right) + T\left(\frac{9}{10}n\right) + \Theta(n)$$

What is the solution to this recurrence?

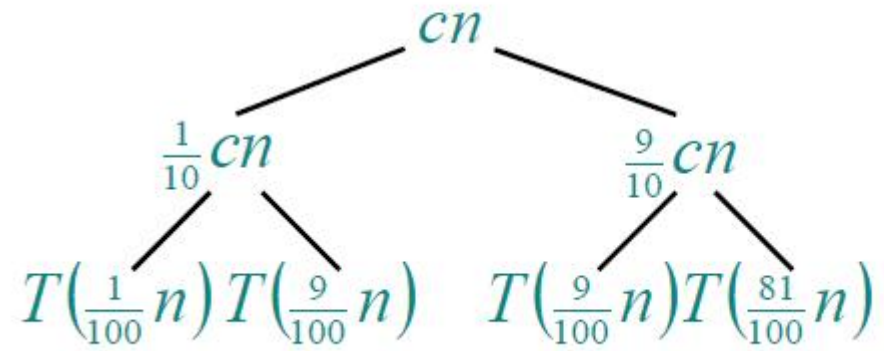
Analysis of “almost-best” case

$$T(n)$$

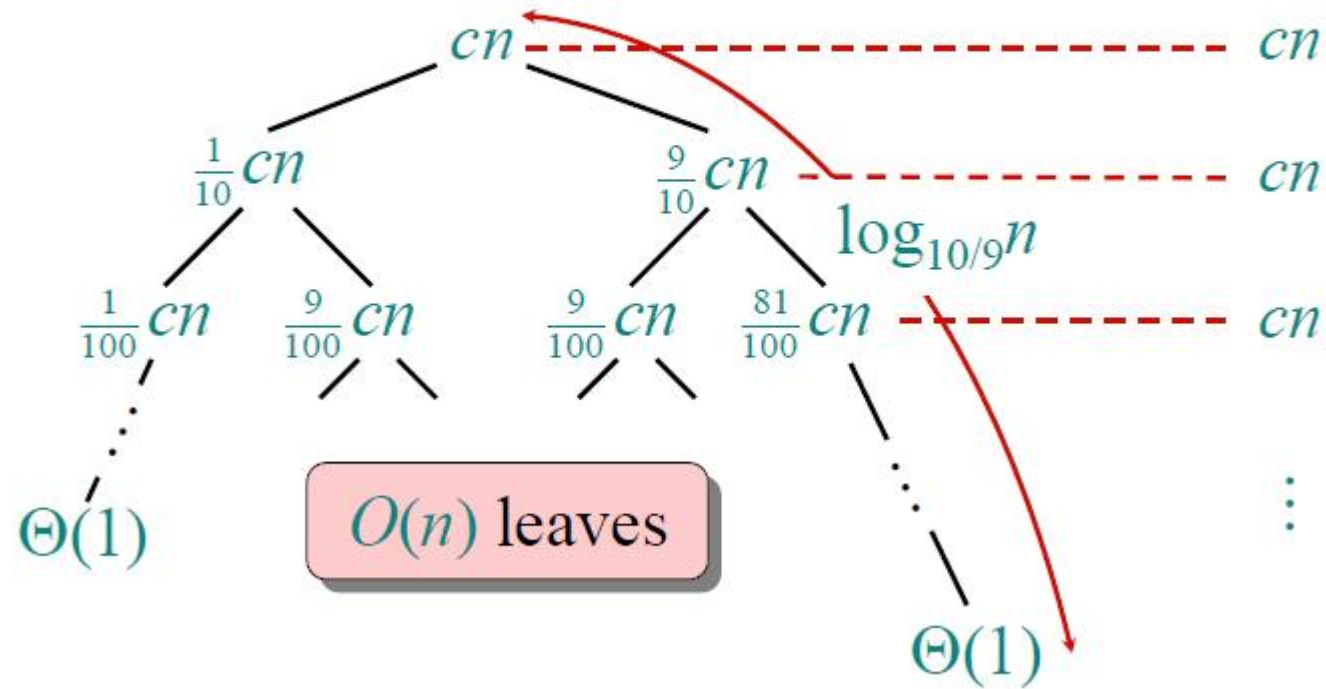
Analysis of “almost-best” case

$$\begin{array}{ccc} & cn & \\ / & & \backslash \\ T\left(\frac{1}{10}n\right) & & T\left(\frac{9}{10}n\right) \end{array}$$

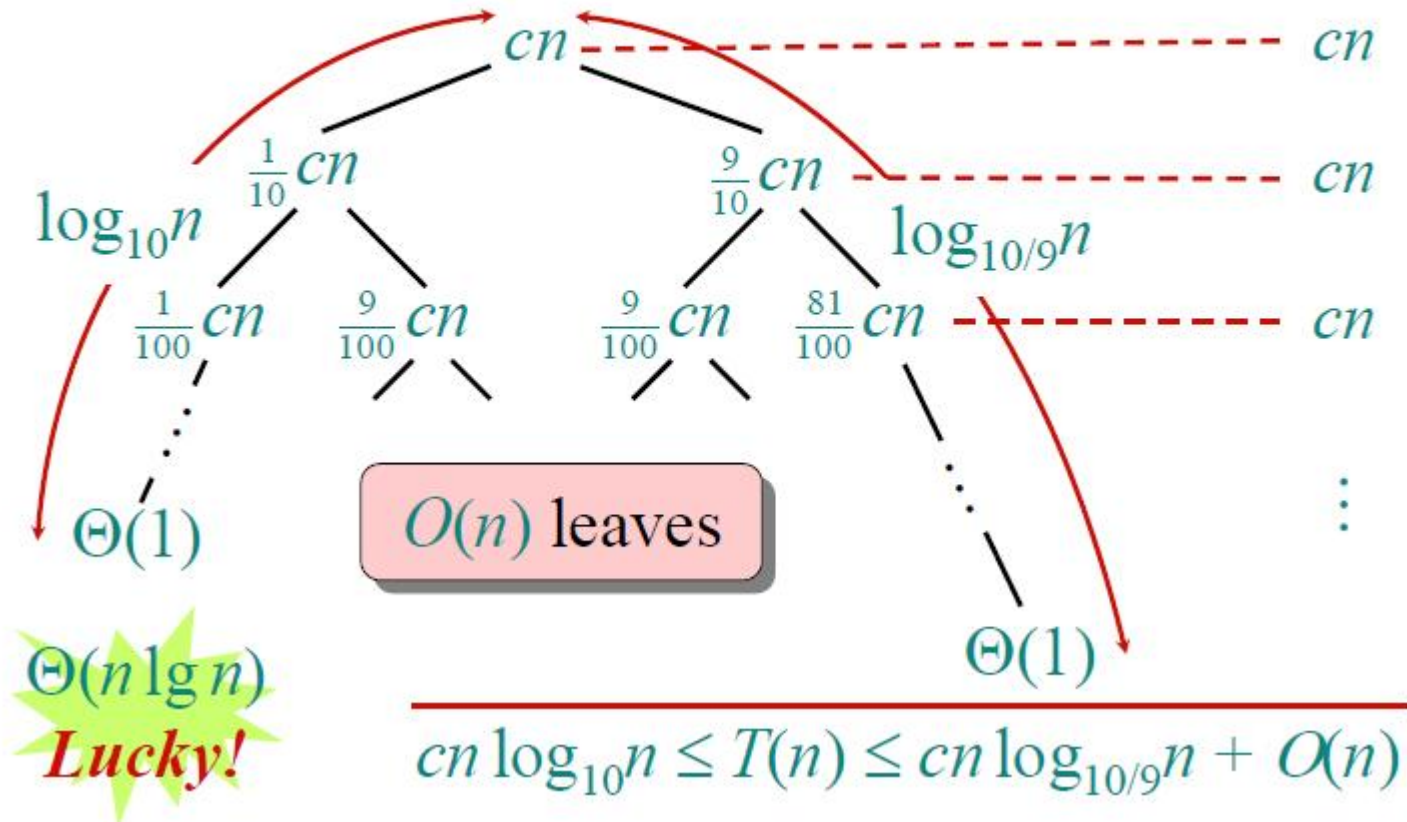
Analysis of “almost-best” case



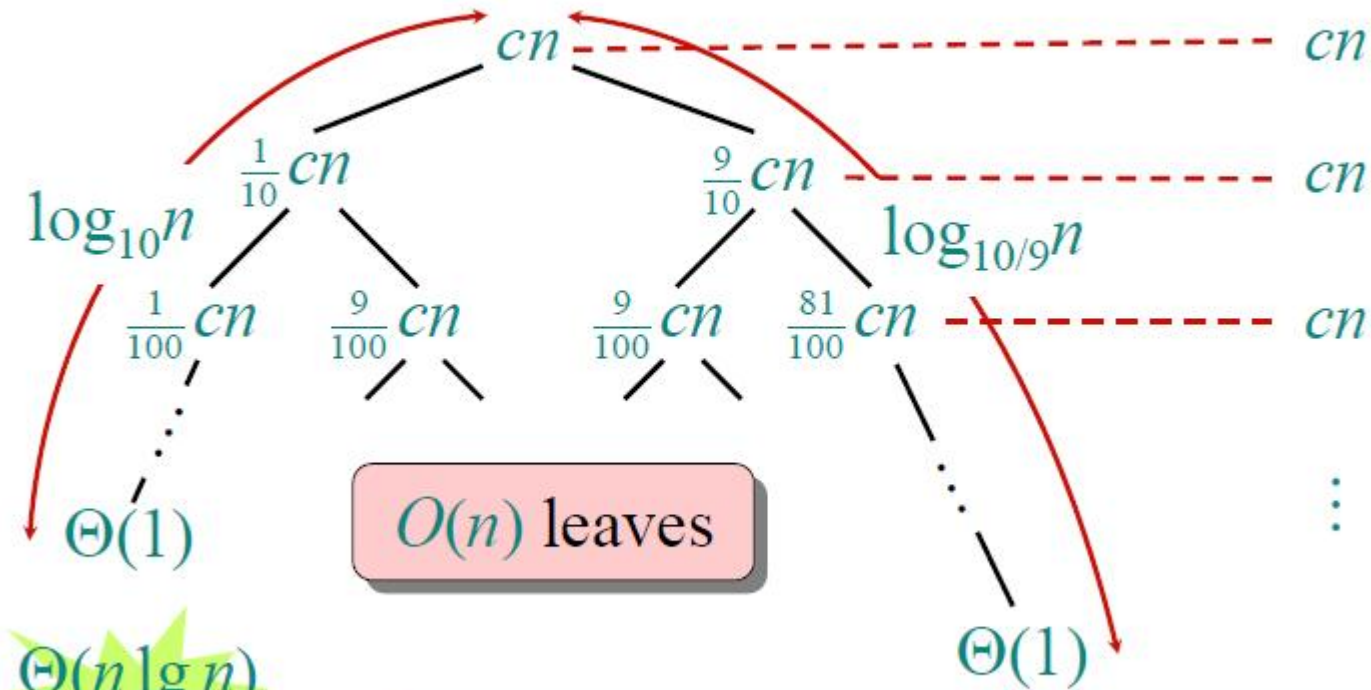
Analysis of “almost-best” case



Analysis of “almost-best” case



Analysis of “almost-best” case



$\Theta(n \lg n)$
Lucky!

$$cn \log_{10} n \leq T(n) \leq cn \log_{10/9} n + O(n)$$

More intuition

Suppose we alternate lucky, unlucky, lucky, unlucky, lucky,

$$L(n) = 2U(n/2) + \Theta(n) \quad \textit{lucky}$$

$$U(n) = L(n-1) + \Theta(n) \quad \textit{unlucky}$$

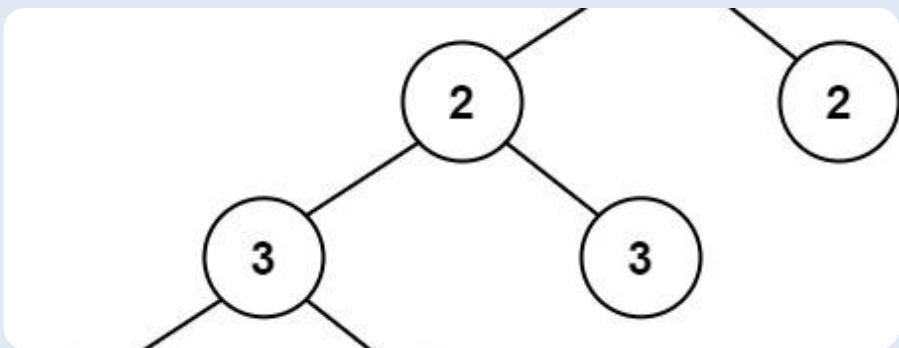
Solving:

$$\begin{aligned} L(n) &= 2(L(n/2 - 1) + \Theta(n/2)) + \Theta(n) \\ &= 2L(n/2 - 1) + \Theta(n) \\ &= \Theta(n \lg n) \quad \textit{Lucky!} \end{aligned}$$

How can we make sure we are usually lucky?

复杂度分析：成也分区，败也分区

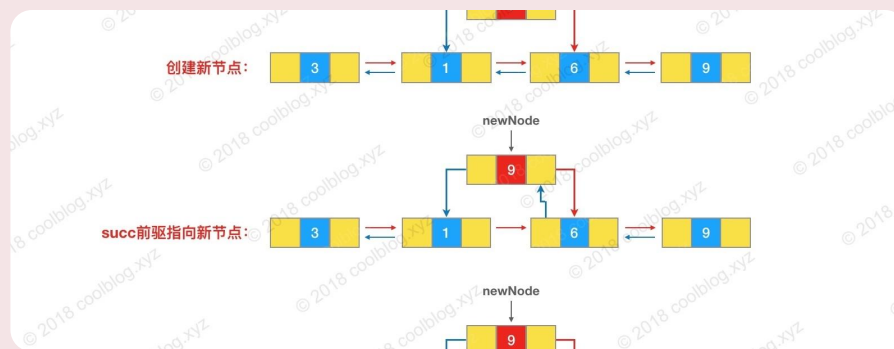
理想情况：平衡分区



每次分区都完美平衡，递归树高度为 $\log n$ 。

$$T(n) = O(n \log n)$$

最坏情况：极端不平衡



每次分区都极不平衡(如一个子数组为空)，递归树退化为链表，高度为 n 。

$$T(n) = O(n^2)$$

快速排序

可以看出：**快速排序算法的性能取决于划分的对称性。**
通过修改算法**partition**，可以设计出采用随机选择策略的快速排序算法。在快速排序算法的每一步中，当数组还没有被划分时，可以在 $a[p:r]$ 中随机选出一个元素作为划分基准，这样可以使划分基准的选择是随机的，从而可以期望划分是较对称的。

📖 最坏时间复杂度： $O(n^2)$

📖 平均时间复杂度： $O(n \log n)$

📖 稳定性：不稳定

优化策略：随机化与三数取中法



随机化 (Randomization)

将枢纽元 **随机选取**，将最坏情况由 **特定输入** 转为 **概率事件**，期望复杂度保持 $\Theta(n \log n)$ 。

```
private static int randomizedPartition (int p, int r)
{
    int i = random(p,r);
    MyMath.swap(a, i, p);
    return partition (p, r);
}
```



三数取中 (Median-of-Three)

采样 **首、尾、中** 三个元素，用其 **中位数** 作为枢纽，有效抵御基本有序输入，常数因子小。

工程实践： 实际工程中常同时启用 **随机化+三数取中**，提供双重保障。

优化策略：规避最坏情况

虽然最坏情况理论上存在，但通过优化基准选择策略，可以使其在实际应用中几乎不可能发生。



随机化

从数组中随机选择一个元素作为基准。这可以消除输入数据顺序对算法性能的影响，使得算法运行时间成为一个随机变量，其期望值为 $O(n \log n)$ 。



三数取中法

选择首、中、尾三个元素的中位数作为基准。这种策略在处理近乎有序的数组时非常有效，能大概率避免最坏情况，且开销很小。

使用三数取中法选择分区点的快速排序实现示例（以Python为例）

```
1. import random
2.
3. def median_of_three(arr, low, high):
4.     mid = (low + high) // 2
5.     if arr[low] > arr[mid]:
6.         arr[low], arr[mid] = arr[mid], arr[low]
7.     if arr[mid] > arr[high]:
8.         arr[mid], arr[high] = arr[high], arr[mid]
9.     if arr[low] > arr[mid]:
10.        arr[low], arr[mid] = arr[mid], arr[low]
11.    return mid # 返回中位数的索引
12.
```

```
13. def partition(arr, low, high):
14.    # 使用三数取中法选择分区点
15.    pivot_index = median_of_three(arr, low, high)
16.    pivot = arr[pivot_index]
17.    # 将分区点交换到数组末尾（或其他合适位置）
18.    arr[pivot_index], arr[high] = arr[high], arr[pivot_index]
19.    i = low - 1
20.    for j in range(low, high):
21.        if arr[j] < pivot:
22.            i += 1
23.            arr[i], arr[j] = arr[j], arr[i]
24.    arr[i + 1], arr[high] = arr[high], arr[i + 1]
25.    return i + 1
26.
```

```
27. def quicksort(arr, low, high):
28.     if low < high:
29.         pi = partition(arr, low, high)
30.         quicksort(arr, low, pi - 1)
31.         quicksort(arr, pi + 1, high)
32.
33. # 示例数组
34. arr = [10, 7, 8, 9, 1, 5]
35. quicksort(arr, 0, len(arr) - 1)
36. print("排序后的数组:", arr)
```

两种实现方法

```
1. def quick_sort(arr):
2.     # 递归终止条件: 数组长度为0或1
3.     if len(arr) <= 1:
4.         return arr
5.
6.     # 选择基准值, 这里选择最后一个元素
7.     pivot = arr[-1]
8.     # 初始化左子数组和右子数组
9.     left = []
10.    right = []
11.
12.    # 遍历数组, 进行分区操作
13.    for i in range(len(arr) - 1): # 排除基准值本身
14.        if arr[i] < pivot:
15.            left.append(arr[i])
16.        else:
17.            right.append(arr[i])
18.
19.    # 递归调用快速排序函数对左右子数组进行排序
20.    sorted_left = quick_sort(left)
21.    sorted_right = quick_sort(right)
22.
23.    # 合并排序后的左右子数组和基准值
24.    return sorted_left + [pivot] + sorted_right
```

```
1. def quick_sort_inplace(arr, low, high):
2.     if low < high:
3.         # 获取分区点
4.         pi = partition(arr, low, high)
5.
6.         # 递归排序左子数组
7.         quick_sort_inplace(arr, low, pi - 1)
8.         # 递归排序右子数组
9.         quick_sort_inplace(arr, pi + 1, high)
10.
11. def partition(arr, low, high):
12.     # 选择基准值, 这里选择最后一个元素
13.     pivot = arr[high]
14.     # 初始化指针i
15.     i = low - 1
16.
17.     # 遍历数组, 进行分区操作
18.     for j in range(low, high):
19.         if arr[j] < pivot:
20.             i += 1
21.             arr[i], arr[j] = arr[j], arr[i]
22.
23.     # 将基准值放到正确的位置上
24.     arr[i + 1], arr[high] = arr[high], arr[i + 1]
25.     return i + 1
```

尾递归消除与混合优化

尾递归消除

先对 **短区间** 递归，对 **长区间** 用循环处理，可将栈深度从 $O(n)$ 降至 $O(\log n)$ 。

// 优化前

```
quickSort(arr, l, p-1)
```

```
quickSort(arr, p+1, r)
```

// 优化后 (尾递归)

```
while (l < r) {
```

```
    quickSort(arr, l, p-1)
```

```
    l = p+1
```

```
}
```

混合优化

当子区间长度小于 **阈值** (如16) 时，切换到 **插入排序**，利用其低常数和缓存局部性。



实测可提升 **10%** 以上性能

理论复杂度与实测常数对照

理论对比

合并排序: 最坏 $\Theta(n \log n)$, 稳定, 但需要 $\Theta(n)$ 额外空间。

快速排序: 平均 $\Theta(n \log n)$, 就地, 但最坏 $\Theta(n^2)$ 。

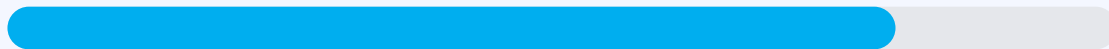
实测数据 (1e7 随机整数)

随机快排: ~1.2s (常数小, 领先约20%)

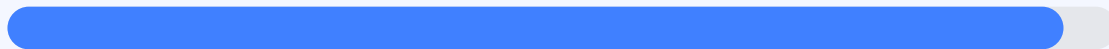
合并排序: ~1.5s (性能稳定)

不同输入下的表现

随机数据



升序数据 (随机化快排)



降序数据 (随机化快排)



注: 条形图长度代表相对耗时, 越长越慢。

多核并行：归并与快排的并行策略

并行归并排序

可并行处理左右子区间。合并阶段可使用并行双缓冲或串行 fallback。



负载均衡易控制

任务划分清晰，并行扩展性好

并行快速排序

可并行递归，但分区阶段需单线程完成，是并行瓶颈。



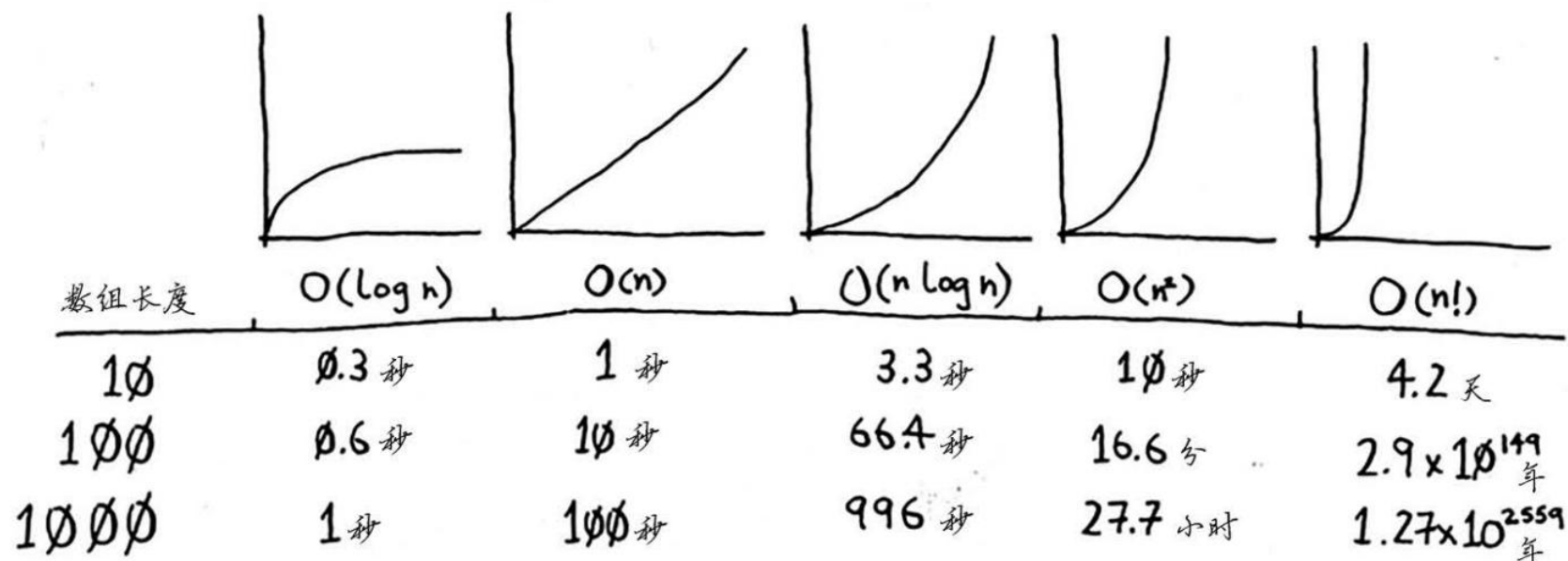
需任务窃取与粒度控制

防止线程饥饿，实现更复杂

再谈大 O 表示法

快速排序的独特之处在于，其速度取决于选择的基准值。
在讨论快速排序的运行时间前，再来看看最常见的大O运行时间。

算法 二分查找 简单查找 快速排序 选择排序 旅行商问题算法



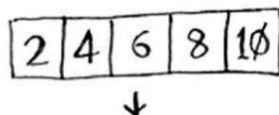
这里的估算是基于
每秒执行10次操作的
慢速计算机

比较合并排序和快速排序

假设有下面这样打印列表中每个元素的简单函数。

```
def print_items(list):  
    for item in list:  
        print item
```

这个函数遍历列表中的每个元素并将其打印出来。它迭代整个列表一次，因此运行时间为 $O(n)$ 。



print_items: 2 4 6 8 10

print_items2: <休眠> 2 <休眠> 4 <休眠> 6 <休眠> 8 <休眠> 10

固定的
时间量

$c * n$

A hand-drawn arrow points from the text '固定的时间量' (Fixed time quantity) to the variable 'c' in the formula 'c * n'.

对这个函数进行修改，使其在打印每个元素前都休眠1秒钟。

```
from time import sleep  
def print_items2(list):  
    for item in list:  
        sleep(1)  
        print item
```

这两个函数都迭代整个列表一次，因此它们的运行时间都为 $O(n)$ 。
认为哪个函数的速度更快呢？

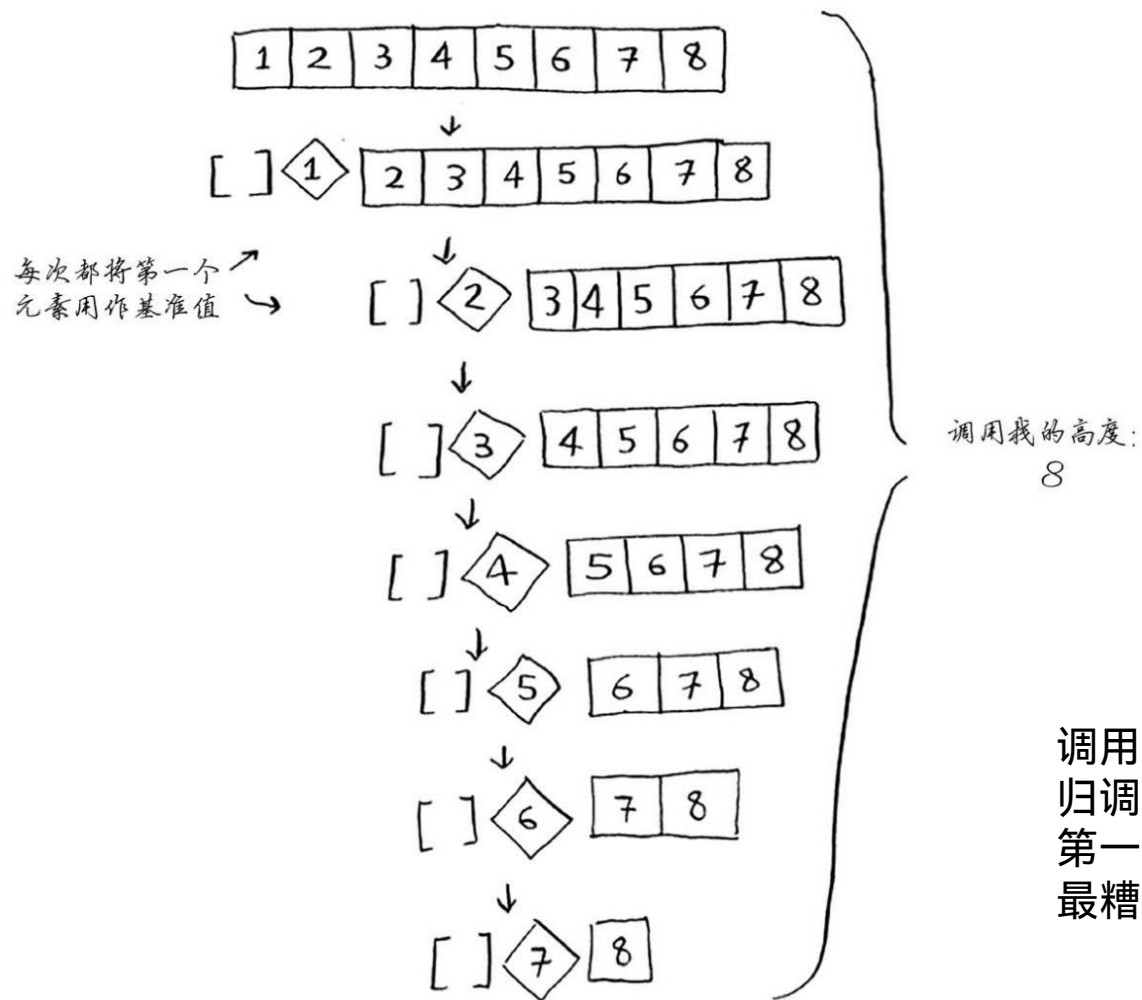
print_items 要快得多，因为它没有每次打印元素前都暂停1秒钟。

因此，虽然使用大O表示法表示时，这两个函数的速度相同，但实际上print_items的速度更快。
在大O表示法 $O(n)$ 中， n 实际上指的是这样的。

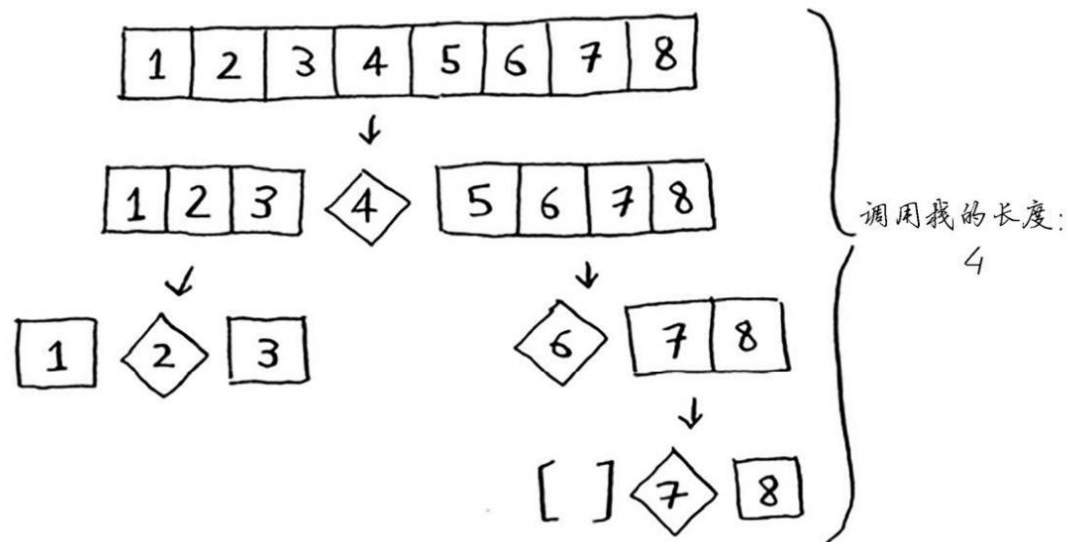
c 是算法所需的固定时间量，被称为常量。例如，print_items 所需的时间可能是10毫秒 * n ，而print_items2 所需的时间为1秒 * n 。

平均情况和最糟情况

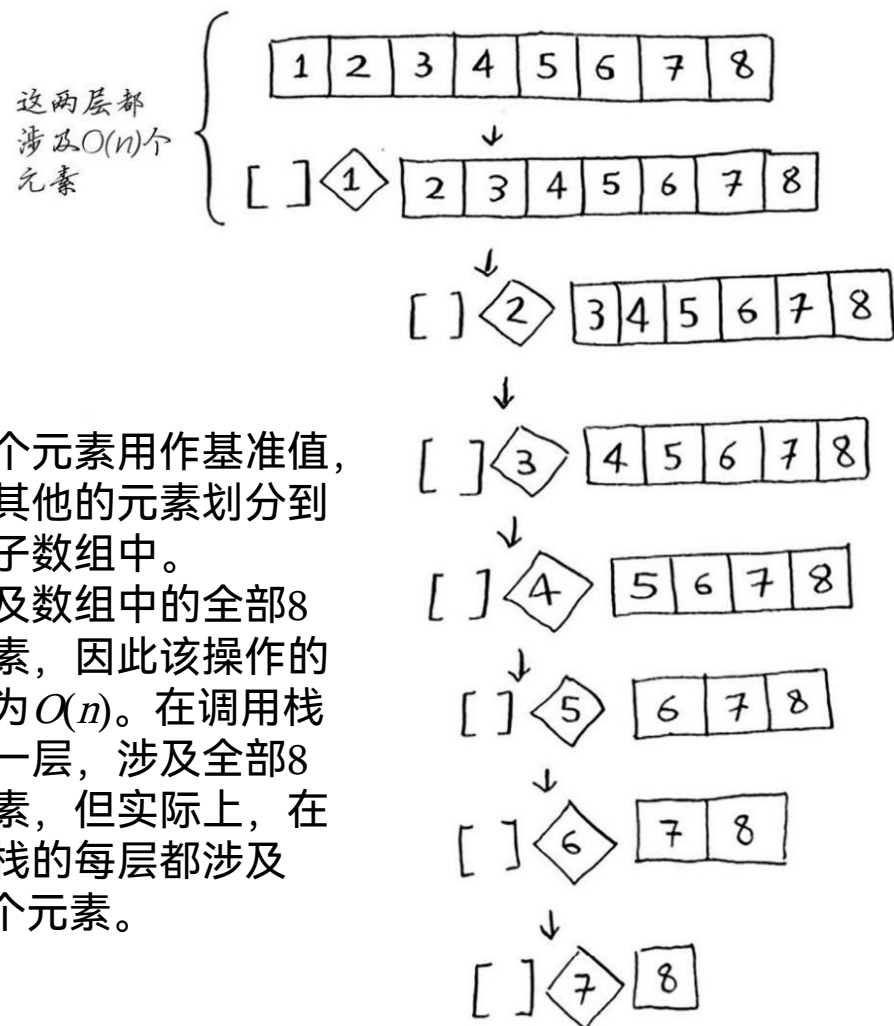
快速排序的性能高度依赖于你选择的基准值。假设你总是将第一个元素用作基准值，且要处理的数组是有序的。由于快速排序算法不检查输入数组是否有序，因此它依然尝试对其进行排序。



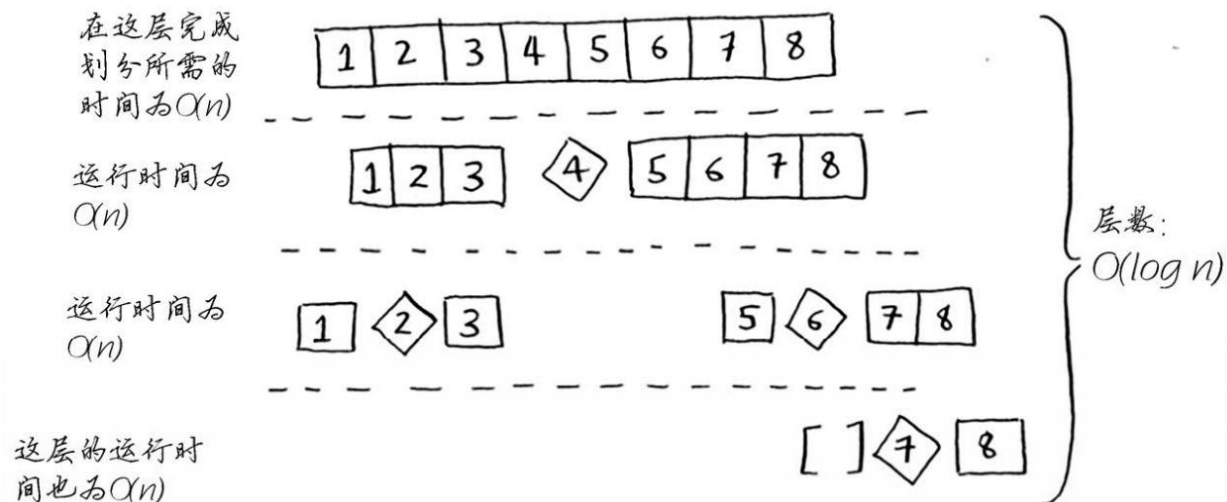
现在假设你总是将中间的元素用作基准值，在这种情况下，调用栈如下。



调用栈短得多！因为你每次都将数组分成两半，所以不需要那么多递归调用。你很快就到达了基线条件，因此调用栈短得多。第一个示例展示的是最糟情况，而第二个示例展示的是最佳情况。在最糟情况下，栈长为 $O(n)$ ，而在最佳情况下，栈长为 $O(\log n)$ 。



完成每层所需的时间都为 $O(n)$ 。

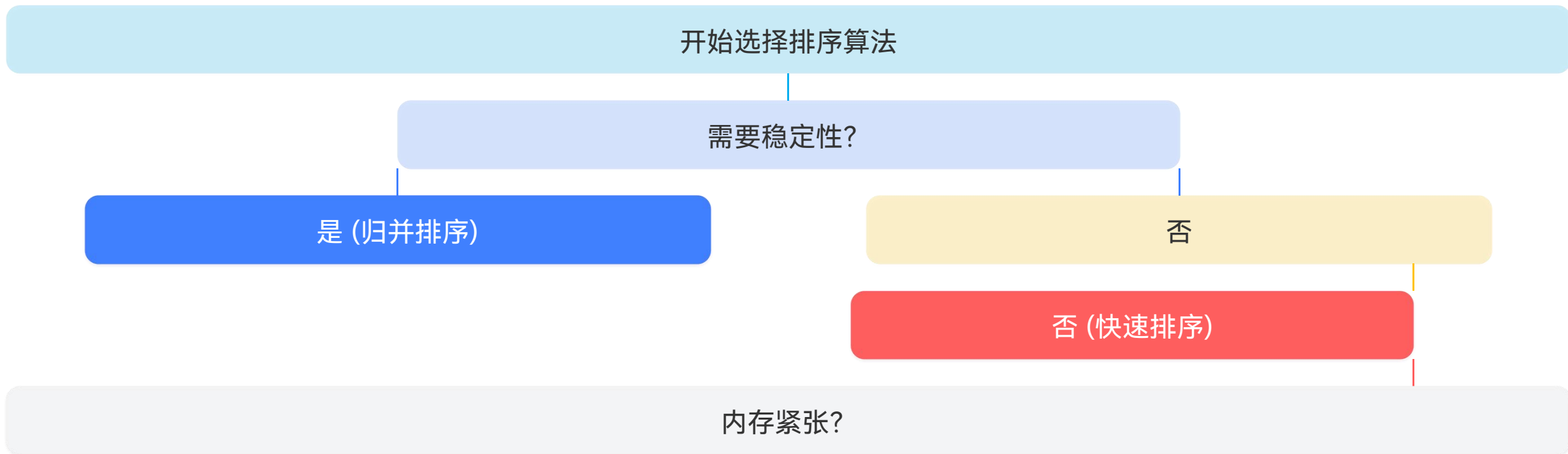


示例中，层数为 $O(\log n)$ (用技术术语说，调用栈的高度为 $O(\log n)$)，而每层需要的时间为 $O(n)$ 。因此，整个算法需要的时间为 $O(n) * O(\log n) = O(n \log n)$ 。这就是最佳情况。在最糟情况下，有 $O(n)$ 层，因此该算法的运行时间为 $O(n) * O(n) = O(n^2)$ 。

合并排序 vs. 快速排序

特性	合并排序 (Merge Sort)	快速排序 (Quick Sort)
时间复杂度	稳定 $O(n \log n)$	平均 $O(n \log n)$, 最坏 $O(n^2)$
空间复杂度	$O(n)$ (需辅助数组)	平均 $O(\log n)$ (递归栈), 最优可达 $O(1)$
稳定性	稳定	不稳定
实现难度	相对复杂 (归并逻辑)	相对简单 (分区逻辑)
缓存友好性	较好 (顺序访问)	一般 (跳跃式访问)

算法选型决策树与调优要点



关键阈值(如切换插入排序的数组大小) 需通过 **基准测试** 进行校准。

回顾与展望：分治思想的延伸

本次回顾

- ✓ 见证了 **稳定 vs 就地** 两条技术路线。
- ✓ 合并阶段可承载 **附加计算**（如逆序对）。
- ✓ 揭示了 **理论最坏** 与 **工程平均** 的落差。

核心思想

分治三步曲——**拆分-治理-合并**——是算法设计的普适性思维。

思想延伸



选择算法



快速傅里叶变换



矩阵乘法

将分治思维迁移至更多计算挑战！

应用场景与选型建议



选择合并排序

- 稳定性是刚需：如数据库、需要保持原有顺序的场景。
- 处理链表：归并操作在链表上效率很高且无需额外空间。
- 外部排序：数据量巨大，无法全部装入内存时。



选择快速排序

- 追求平均速度：在内存充足时，常数因子小，平均性能优异。
- 内存空间有限：原地排序(或仅需很小栈空间)是巨大优势。
- 通用排序库：如 C++ 的 `std::sort`，通常是内省排序(快排+堆排+插排)。



PART 07



排序的其他算法



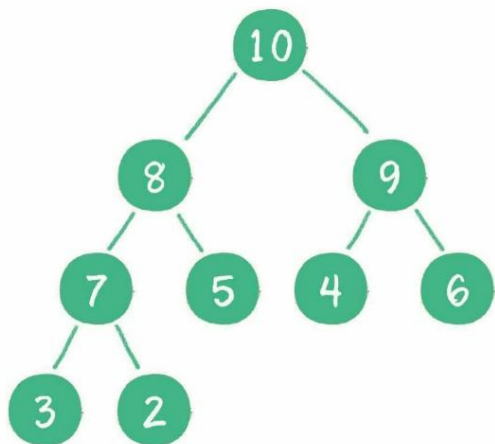
堆排序： 二叉堆

二叉堆？

二叉堆本质上是一种完全二叉树，它分为两个类型。

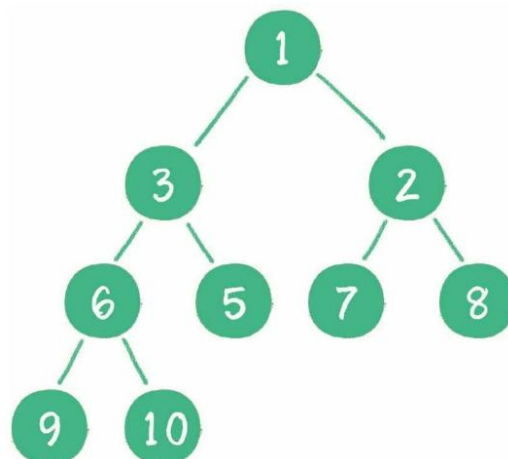
- 1.最大堆。
- 2.最小堆。

二叉堆的根结点叫做堆顶。



最大堆？

最大堆的任何一个父节点的值，都大于或等于它左、右孩子节点的值。



最小堆？

最小堆的任何一个父节点的值，都小于或等于它左、右孩子节点的值。

二叉堆的自我调整

对于二叉堆，有如下几种操作：

- 1.插入节点
- 2.删除节点
- 3.构建二叉堆

这几种操作都基于堆的自我调整。

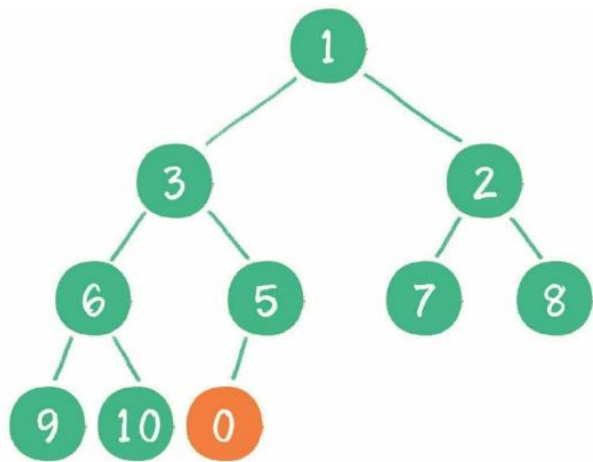
所谓堆的自我调整，就是把一个不符合堆性质的完全二叉树，调整成一个堆。

二叉堆的自我调整

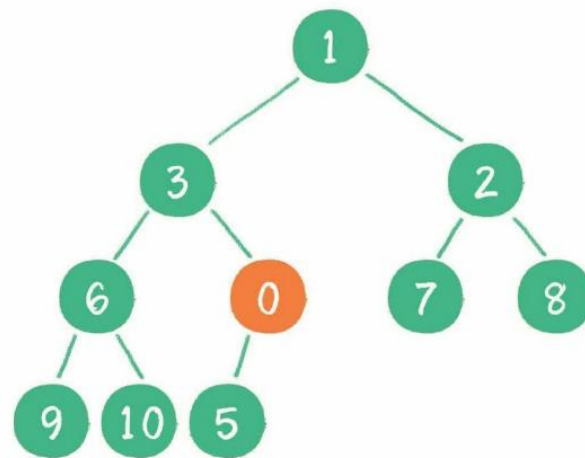
1. 插入节点

当二叉堆插入节点时，**插入位置**是完全二叉树的**最后一个位置**。

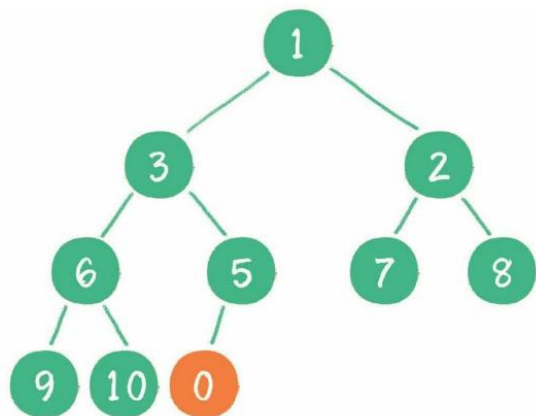
例如插入一个新节点，值是0。



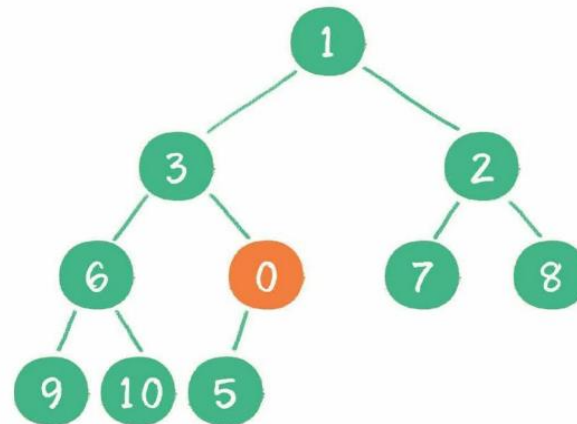
这时，新节点的父节点5比0大，显然不符合最小堆的性质。于是让新节点“上浮”，和父节点交换位置。



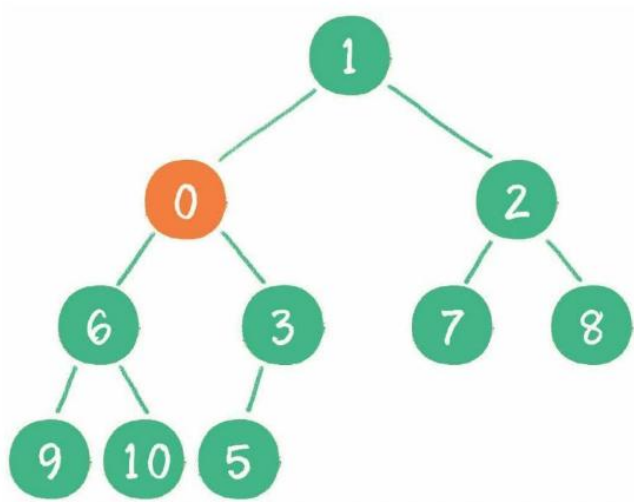
继续用节点0和父节点3做比较，因为0小于3，则让新节点继续“上浮”。



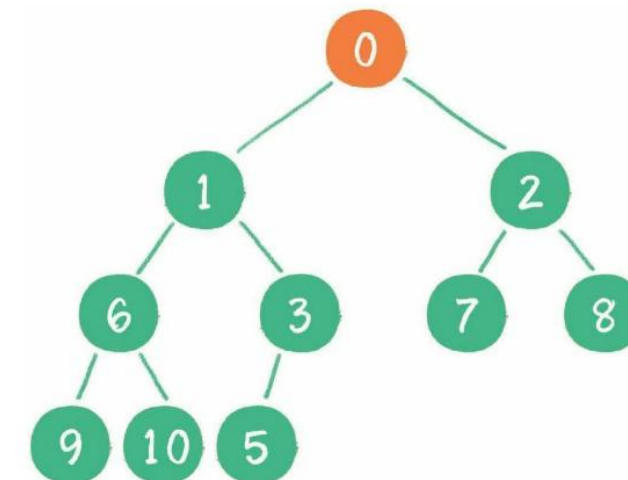
这时，新节点的父节点5比0大，显然不符合最小堆的性质。于是让新节点“上浮”，和父节点交换位置。



继续用节点0和父节点3做比较，因为0小于3，则让新节点继续“上浮”。



继续比较，最终新节点0“上浮”到了堆顶位置。

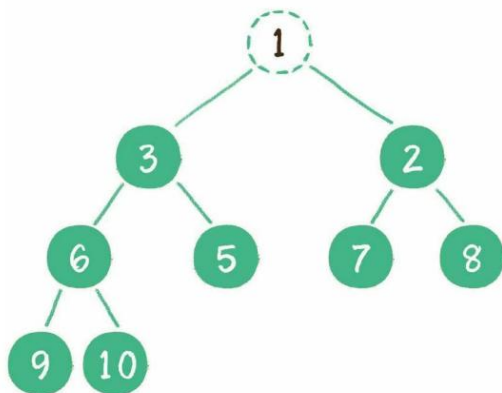


二叉堆的自我调整

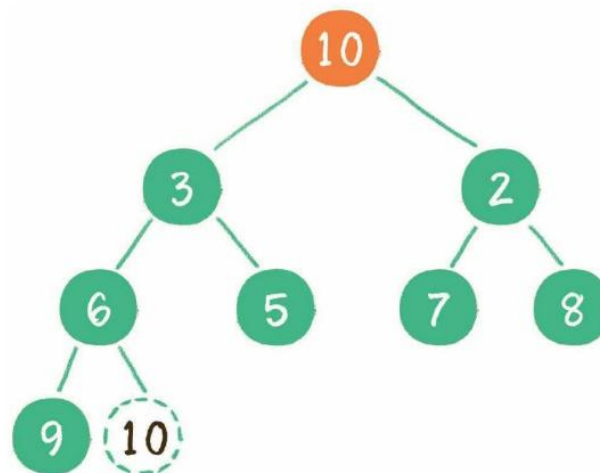
2. 删除节点

二叉堆删除节点的过程和插入节点的过程正好相反，所删除的是处于堆顶的节点。

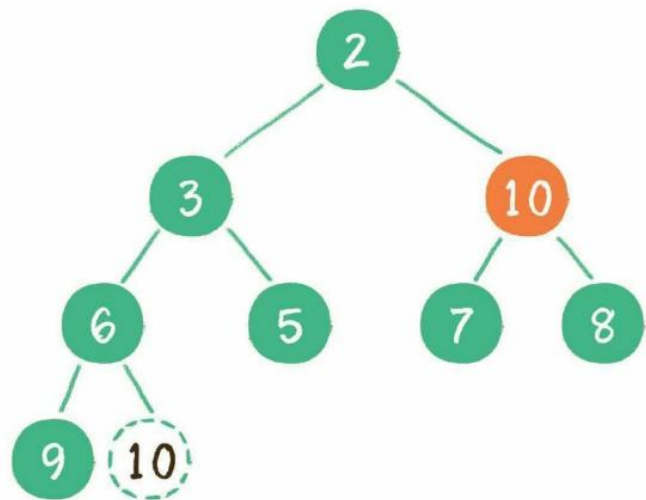
例如：删除最小堆的堆顶节点1。



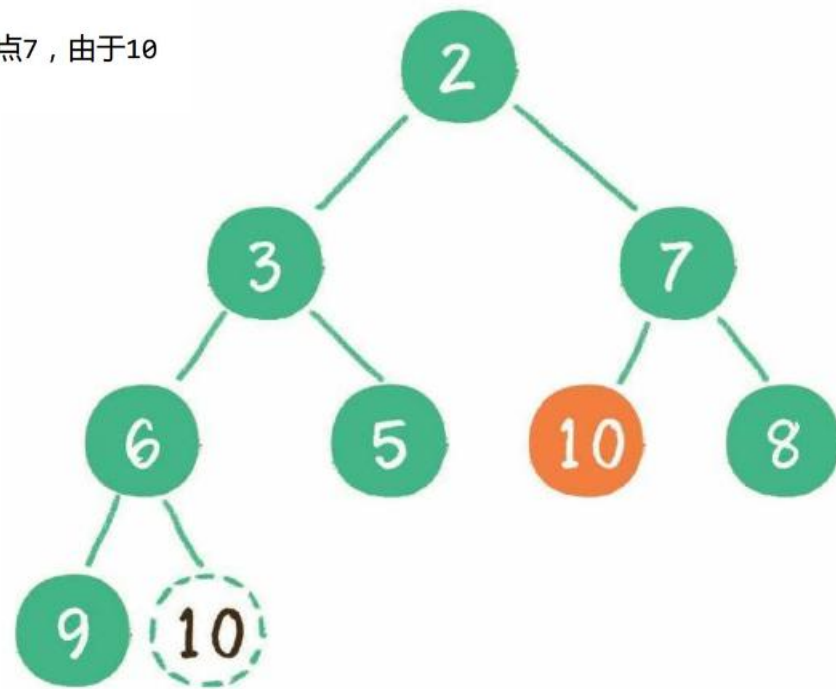
这时，为了继续维持完全二叉树的结构，我们把堆的最后一个节点10临时补到原本堆顶的位置。



接下来，让暂处堆顶位置的节点10和它的左、右孩子进行比较，如果左、右孩子节点中最小的一个（显然是节点2）比节点10小，那么让节点10“下沉”。



继续让节点10和它的左、右孩子做比较，左、右孩子中最小的是节点7，由于10大于7，让节点10继续“下沉”。

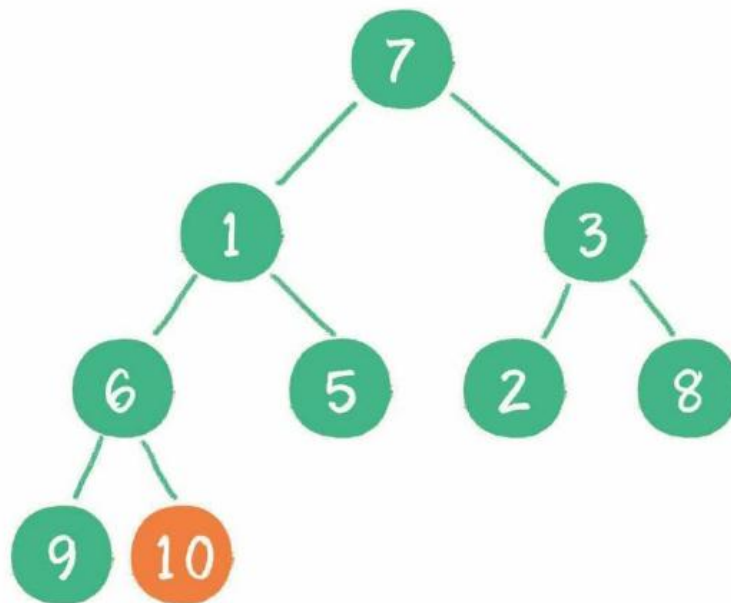
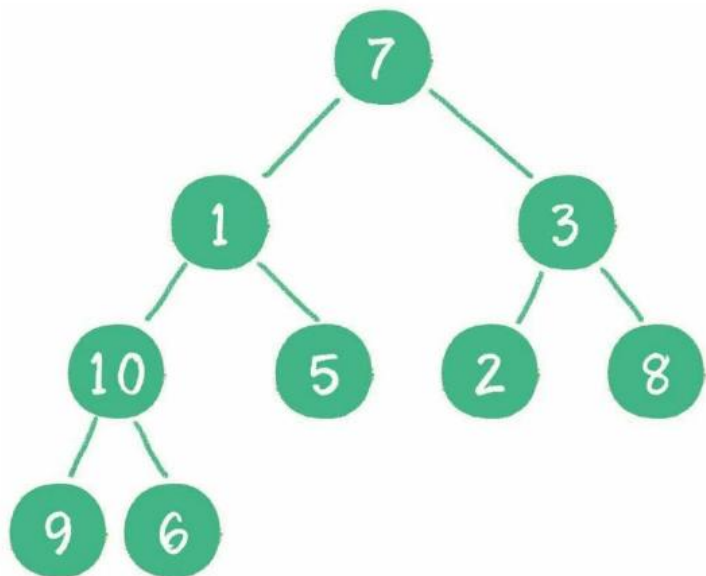


这样一来，二叉堆重新得到了调整。

二叉堆的自我调整

构建二叉堆，也就是把一个无序的完全二叉树调整为二叉堆，本质就是让所有非叶子节点依次“下沉”。

例如：下图无序完全二叉树→最小堆。



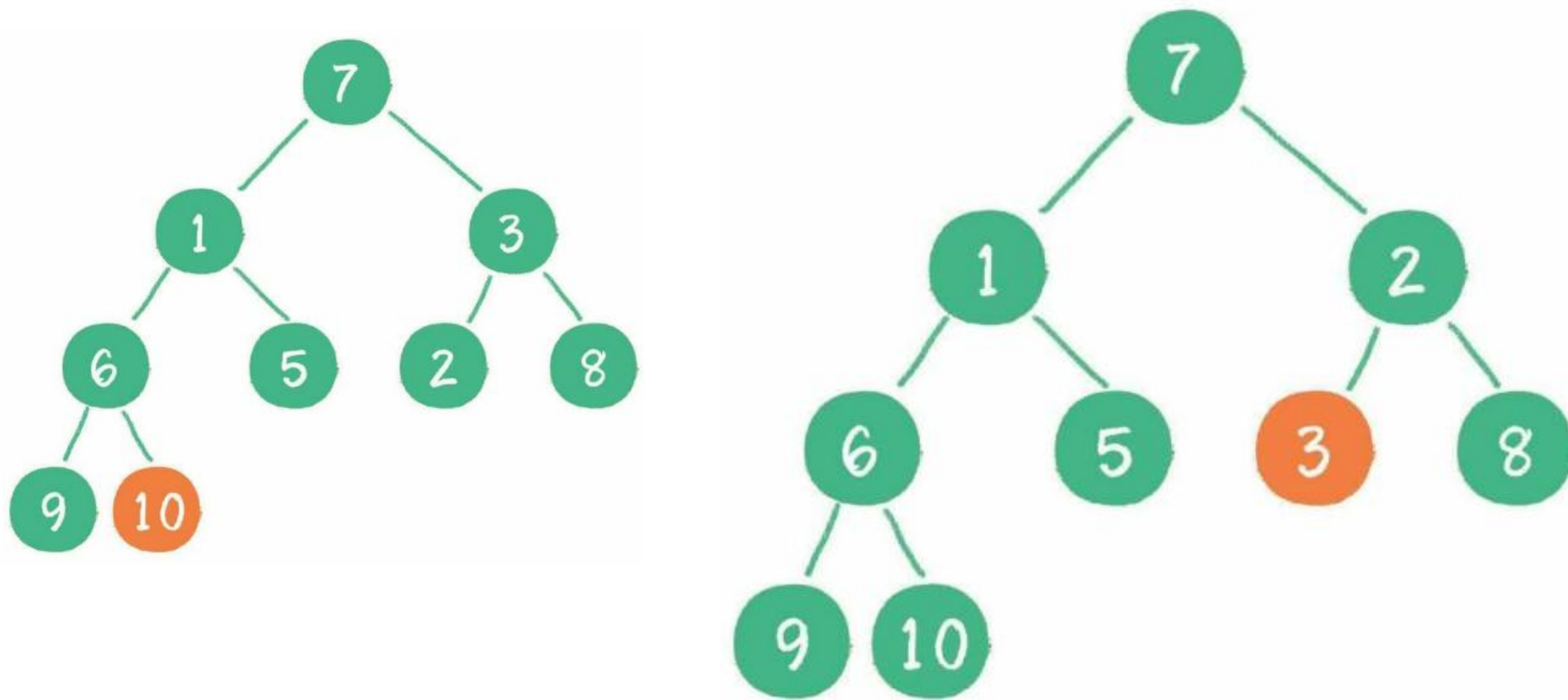
首先，从最后一个非叶子节点开始，也就是从节点10开始。

如果节点10大于它左、右孩子节点中最小的一个，则节点10“下沉”。

二叉堆的自我调整

接下来轮到节点3。

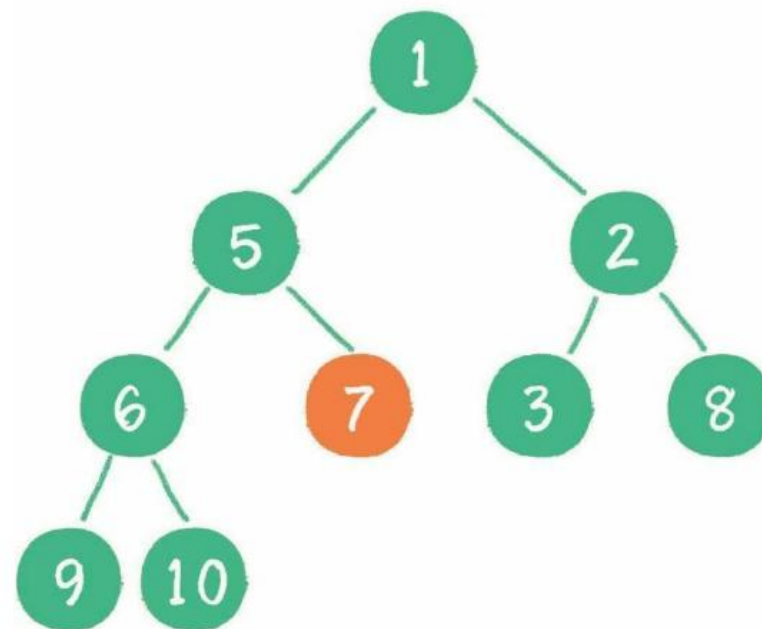
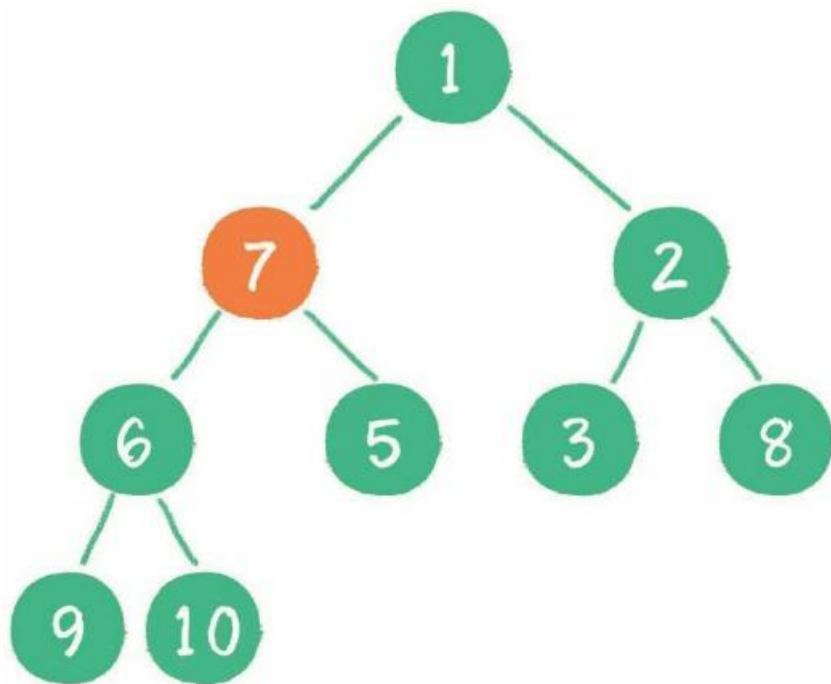
如果节点3大于它左、右孩子节点中最小的一个，则节点3“下沉”。



二叉堆的自我调整

- 轮到**节点1**：如果节点1大于它左、右孩子节点中最小的一个，则节点1“下沉”。事实上节点1小于它的左、右孩子，所以不用改变。
- 轮到**节点7**：如果节点7大于它左、右孩子节点中最小的一个，则节点7“下沉”。

节点7继续比较，继续“下沉”。



经过上述几轮比较和“下沉”操作，最终每一节点都小于它的左、右孩子节点，一个无序的**完全二叉树**就被构建成了一个**最小堆**。

堆的插入和删除操作复杂度是 $O(\log N)$

二叉堆的复杂性分析

```
1
2  def up_adjust(array=[]):
3      """
4      二叉堆的尾节点上浮操作
5      :param array: 原数组
6      """
7      child_index = len(array) - 1
8      parent_index = (child_index - 1) // 2
9      # temp保存插入的叶子节点值, 用于最后的赋值
10     temp = array[child_index]
11     while child_index > 0 and temp < array[parent_index]:
12         # 无需真正交换, 单向赋值即可
13         array[child_index] = array[parent_index]
14         child_index = parent_index
15         parent_index = (parent_index - 1) // 2
16     array[child_index] = temp
17
18
19  def down_adjust(parent_index, length, array=[]):
20      """
21      二叉堆的节点下沉操作
22      :param parent_index: 待下沉的节点下标
23      :param length: 堆的长度范围
24      :param array: 原数组
25      """
26      # temp保存父节点值, 用于最后的赋值
27      temp = array[parent_index]
28      child_index = 2 * parent_index + 1
29      while child_index < length:
30         # 如果有右孩子, 且右孩子小于左孩子的值, 则定位到右孩子
31         if child_index + 1 < length and array[child_index + 1] < array[child_index]:
32             child_index += 1
33         # 如果父节点小于任何一个孩子的值, 直接跳出
34         if temp <= array[child_index]:
35             break
36         # 无需真正交换, 单向赋值即可
37         array[parent_index] = array[child_index]
38         parent_index = child_index
39         child_index = 2 * child_index + 1
40     array[parent_index] = temp
41
```

```
42
43  def build_heap(array=[]):
44      """
45      二叉堆的构建操作
46      :param array: 原数组
47      """
48      # 从最后一个非叶子节点开始, 依次下沉调整
49      for i in range((len(array)-2) // 2, -1, -1):
50         down_adjust(i, len(array), array)
51
52
53     my_array = list([1, 3, 2, 6, 5, 7, 8, 9, 10, 0])
54     up_adjust(my_array)
55     print(my_array)
56     my_array = list([7, 1, 3, 10, 5, 2, 8, 9, 6])
57     build_heap(my_array)
58     print(my_array)
```

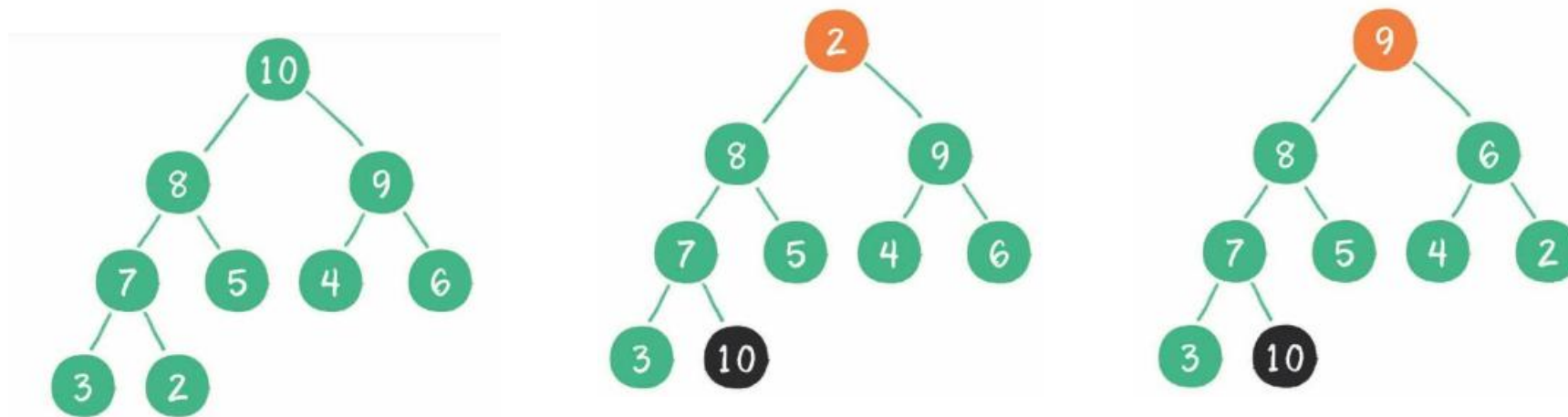
- 构建堆的时间复杂度是 $O(N)$

堆排序

二叉堆的特性：

- 1.最大堆的堆顶是整个堆中的最大元素。
- 2.最小堆的堆顶是整个堆中的最小元素。

以最大堆为例，如果删除一个最大堆的堆顶（并不是完全删除，而是跟末尾的节点交换位置），经过自我调整，第2大的元素就会被交换上来，成为最大堆的新堆顶。

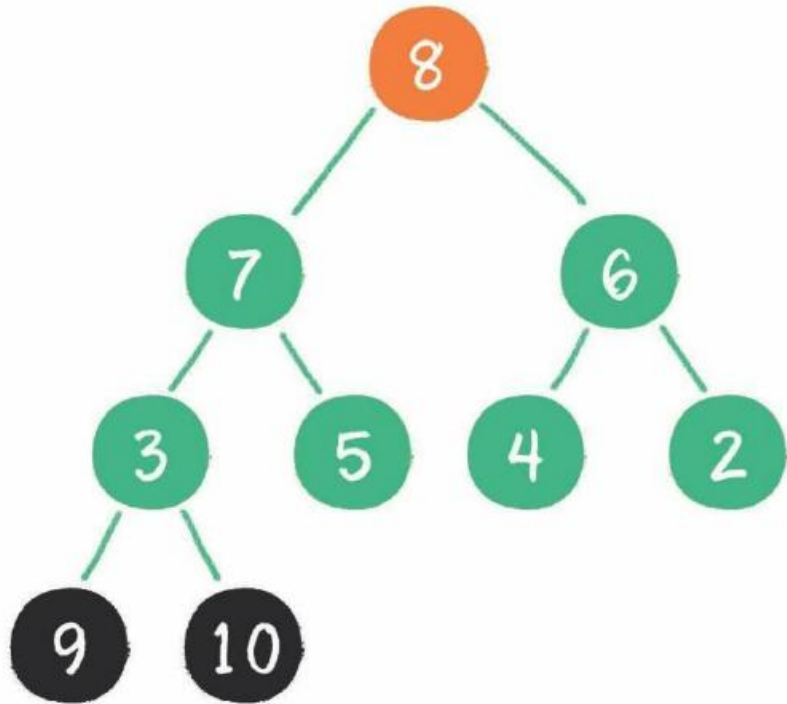


如上图所示，在删除值为10的堆顶节点后，经过调整，值为9的新节点就会顶替上来；在删除值为9的堆顶节点后，经过调整，值为8的新节点就会顶替上来.....

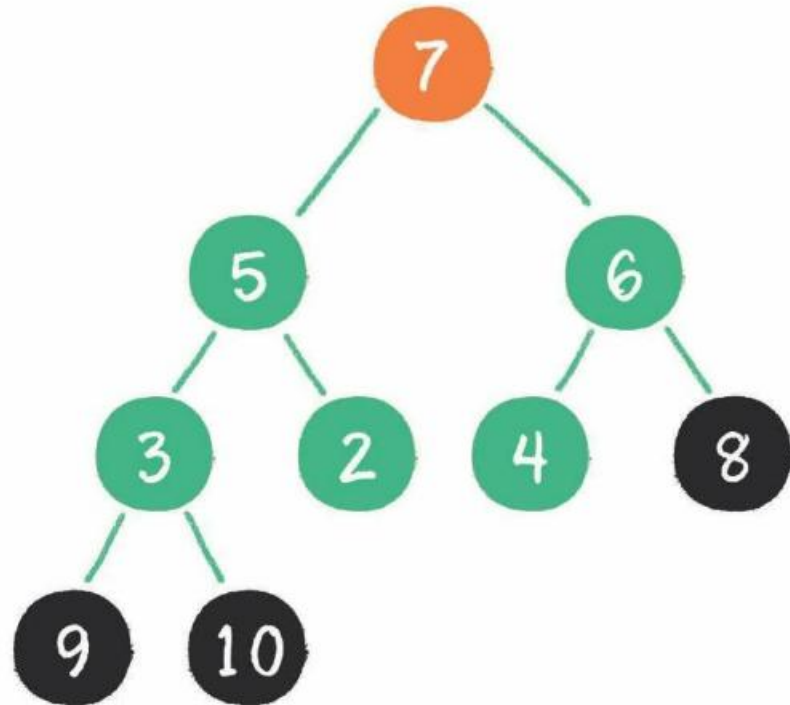
堆排序

- 由于二叉堆的特性：每一次删除旧堆顶，调整后的新堆顶都是大小仅次于旧堆顶的节点。那么只要反复删除堆顶，反复调整二叉堆，所得到的集合就会成为一个有序集合，过程如下。

删除节点9,节点8成为新堆顶。

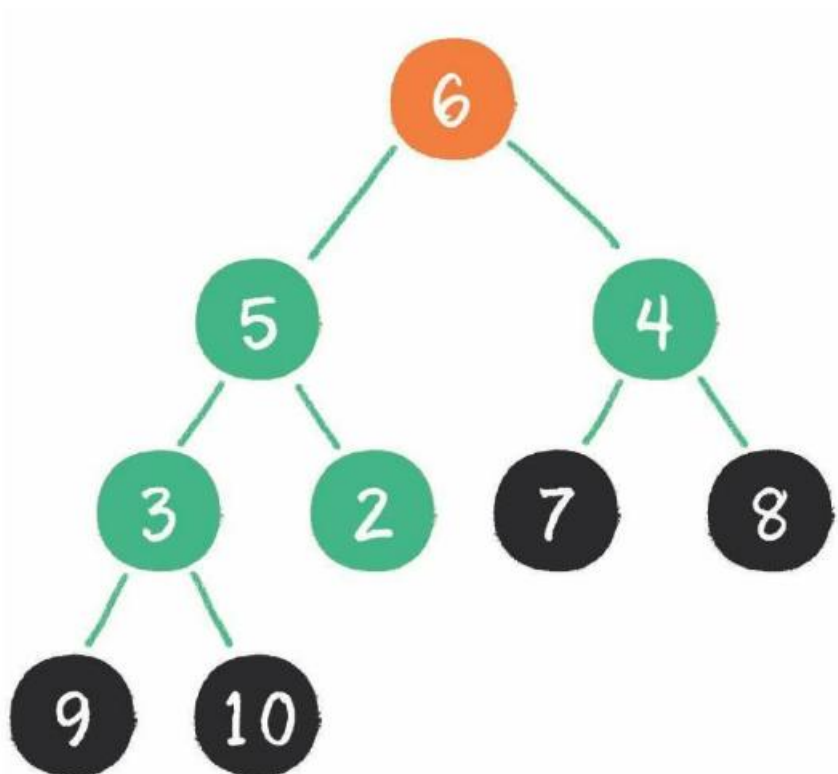


删除节点8，节点7成为新堆顶。

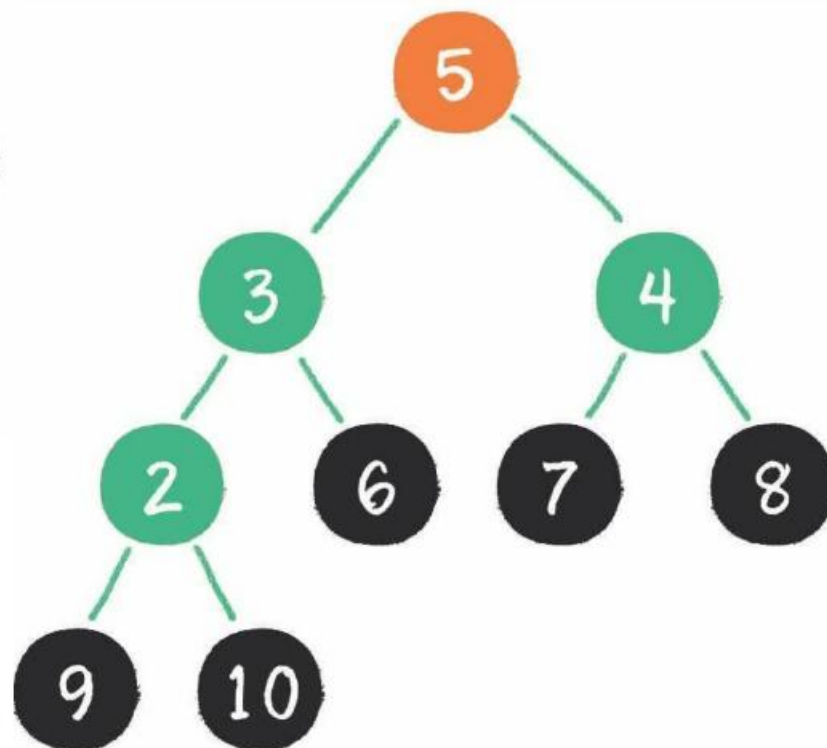


堆排序

- 删除节点7，节点6成为新堆顶

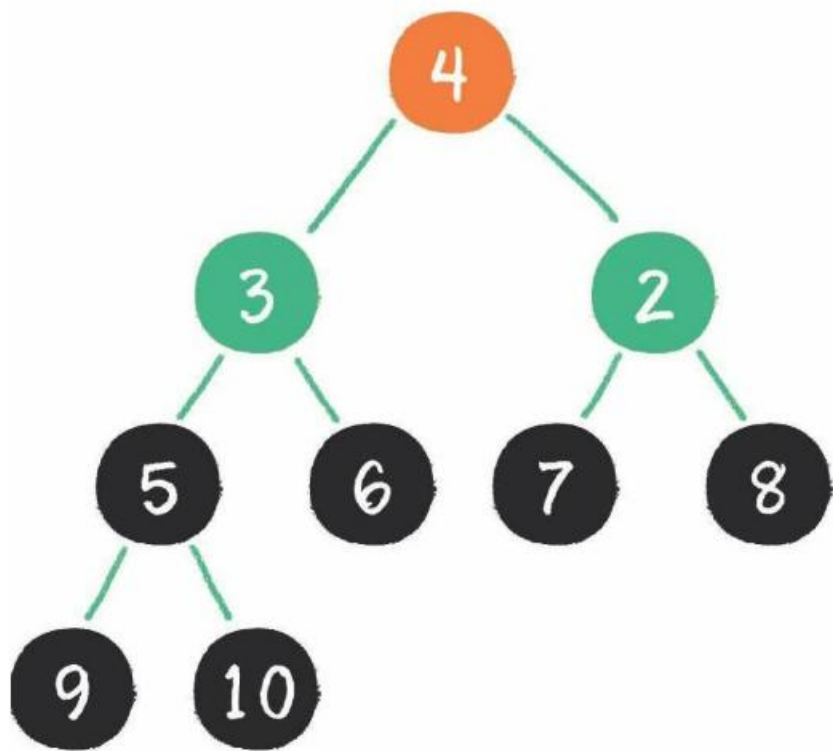


删除节点6，节点5成为新堆顶。

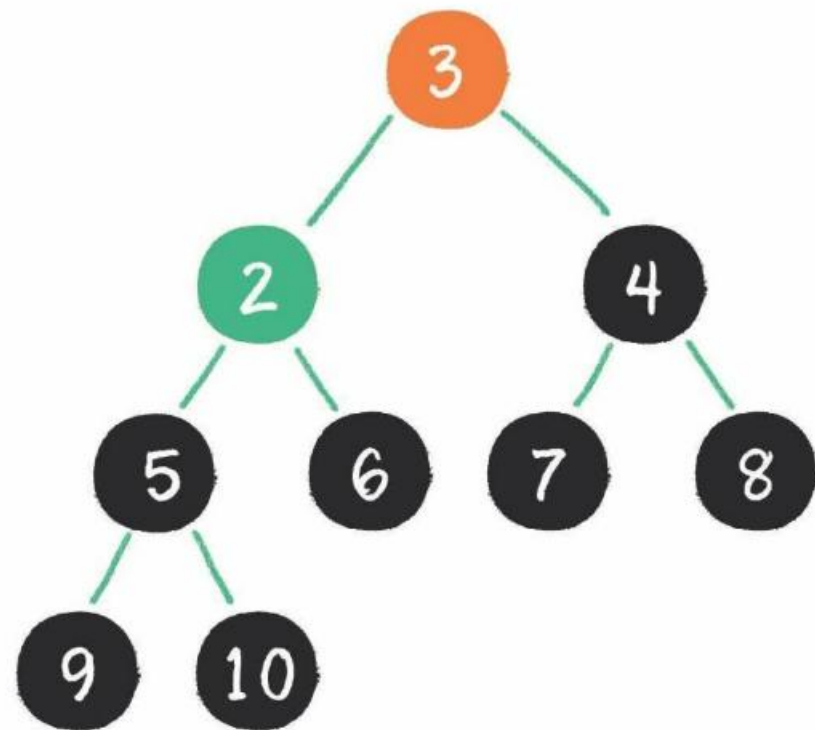


堆排序

删除节点5，节点4成为新堆顶。

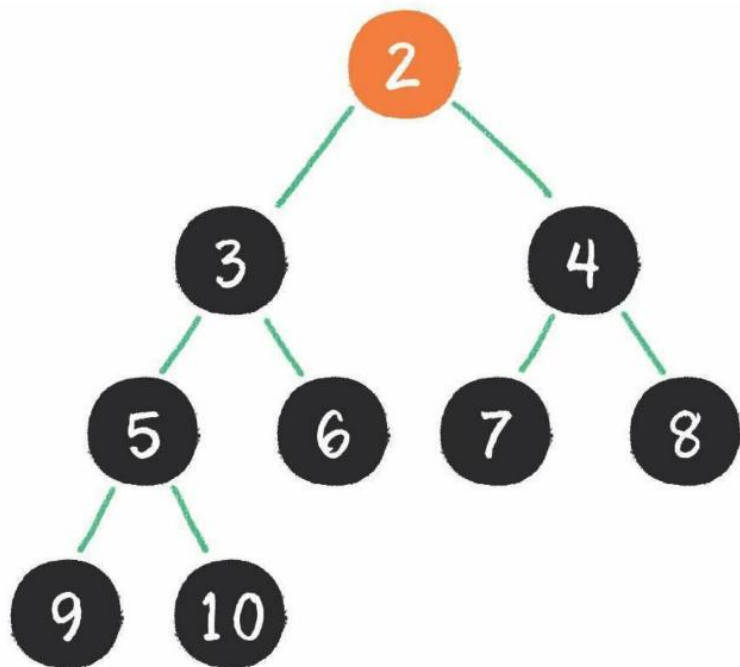


删除节点4，节点3成为新堆顶。



堆排序

删除节点3，节点2成为新堆顶。



到此为止，原本的最大二叉堆已经变成了一个从小到大的有序集合。

二叉堆实际存储在数组中，数组中的元素排列如下。



堆排序算法的步骤。

1.把无序数组构建成二叉堆。

需要从小到大排序，则构建成最大堆；需要从大到小排序，则构建成最小堆。

2.循环删除堆顶元素，替换到二叉堆的末尾，调整堆产生新的堆顶。

堆排序复杂性分析

```
1
2 ✓ def heap_sort(array=[]):
3     # 1.把无序数组构建成最大堆
4     for i in range((len(array)-2)//2, -1, -1):
5         down_adjust(i, len(array), array)
6     # 2.循环交换集合尾部元素到堆顶,并调节堆产生新的堆顶
7     for i in range(len(array)-1, 0, -1):
8         # 最后一个元素和第一元素进行交换
9         temp = array[i]
10        array[i] = array[0]
11        array[0] = temp
12        # 下沉调整最大堆
13        down_adjust(0, i, array)
14
15
16 ✓ def down_adjust(parent_index, length, array=[]):
17     # temp保存父节点值,用于最后的赋值
18     temp = array[parent_index]
19     child_index = 2 * parent_index + 1
20     while child_index < length:
21         # 如果有右孩子,且右孩子大于左孩子的值,则定位到右孩子
22         if child_index+1 < length and array[child_index+1] > array[child_index]:
23             child_index += 1
24         # 如果父节点大于等于任何一个孩子的值,直接跳出
25         if temp >= array[child_index]:
26             break
27         # 无需真正交换,单向赋值即可
28         array[parent_index] = array[child_index]
29         parent_index = child_index
30         child_index = 2 * child_index + 1
31     array[parent_index] = temp
32
33
34 my_array = list([3, 4, 14, 1, 5, 6, 7, 8, 1, -1, 0, 9, 11])
35 heap_sort(my_array)
36 print(my_array)
```

二叉堆的节点“下沉”调整（downAdjust方法）是堆排序算法的基础，这个调节操作本身的时间复杂度是 $O(\log n)$ 。

第1步，把无序数组构建成二叉堆，这一步的时间复杂度 $O(n)$ 。

第2步，需要进行 $n-1$ 次循环。

每次循环调用一次downAdjust方法，
所以第2步的计算规模是 $(n-1) \times \log n$ ，
时间复杂度为 $O(n \log n)$ 。

两个步骤是并列关系，所以整体的时间复杂度是 $O(n \log n)$ 。

线性时间排序：计数排序和桶排序

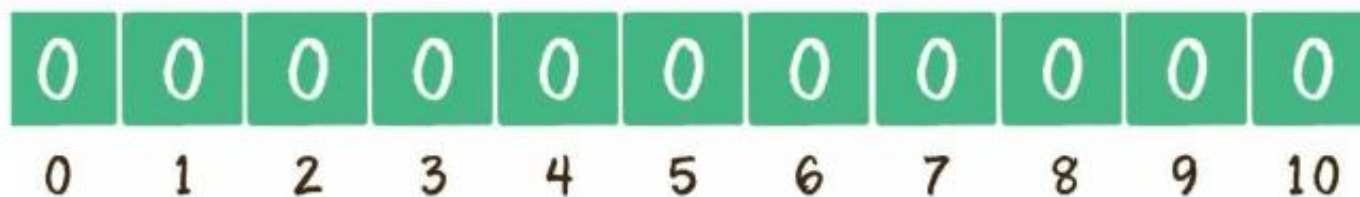
- 计数排序

- 假设数组中有20个随机整数，取值范围为0 ~ 10，要求用最快的速度把这20个整数从小到大进行排序。
如何给这些无序的随机整数进行排序？

假设20个随机整数的值如下所示。

9, 3, 5, 4, 9, 1, 2, 7, 8, 1, 3, 6, 5, 3, 4, 0, 10, 9, 7, 9

- 考虑到这些整数只能够在0、1、2、3、4、5、6、7、8、9、10这11个数中取值，取值范围有限。
- 所以，可以根据这有限的范围，建立一个长度为11的数组。数组下标从0到10，元素初始值全为0。

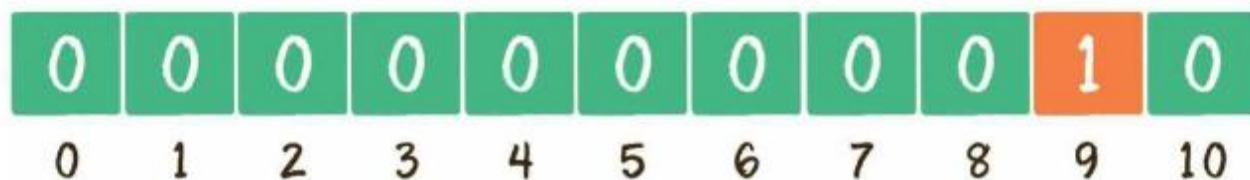


线性排序：计数排序

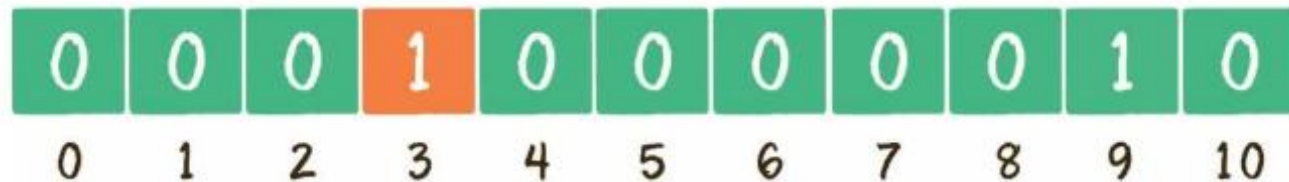
9, 3, 5, 4, 9, 1, 2, 7, 8, 1, 3, 6, 5, 3, 4, 0, 10, 9, 7, 9

遍历这个无序的随机数列，每一个整数按照其值对号入座，同时，对应数组下标的元素进行加1操作。

例如：第1个整数是9，那么数组下标为9的元素加1。



第2个整数是3，那么数组下标为3的元素加1。



继续遍历数列并修改数组.....最终，当数列遍历完毕时，数组的状态如下。



该数组中每一个下标位置的值代表数列中对应整数出现的次数。
有了这个统计结果，排序就很简单了。直接遍历数组，输出数组元素的下标值，元素的值是几，就输出几次。

0 , 1 , 1 , 2 , 3 , 3 , 3 , 4 , 4 , 5 , 5 , 6 , 7 , 7 , 8 , 9 , 9 , 9 , 9 , 10

在取值范围不是很大的情况下，它的性能甚至快过那些时间复杂度为 $O(n\log n)$ 的排序。

计数排序的优化(1)

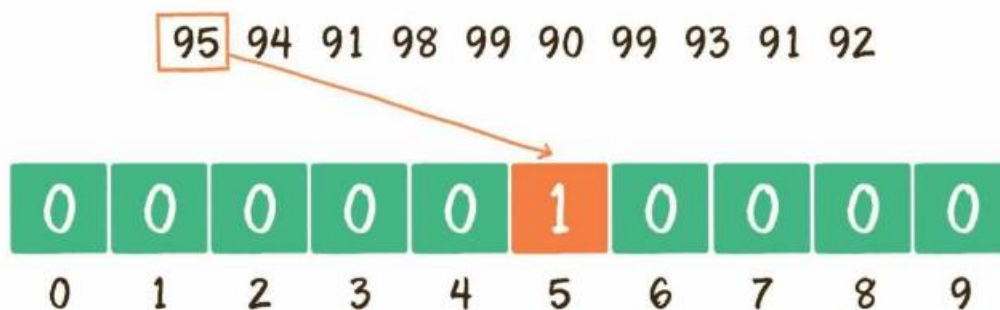
如下面的数列：95 94 91 98 99 90 99 93 91 92

这个数列的最大值是99，但最小的整数是90。

如果创建长度为100的数组，那么前面从0到89的空间位置就都浪费了！

怎么解决这个问题呢？

- 不再以输入数列的最大值+1作为统计数组的长度，而是以数列最大值-最小值+1作为统计数组的长度即可。
- 同时，数列的最小值作为一个偏移量，用于计算整数在统计数组中的下标。
- 以刚才的数列为例，统计出数组的长度为 $99-90+1=10$ ，偏移量等于数列的最小值90。对于第1个整数95，对应的统计数组下标是 $95-90=5$ ，如图所示。



计数排序的优化(2)

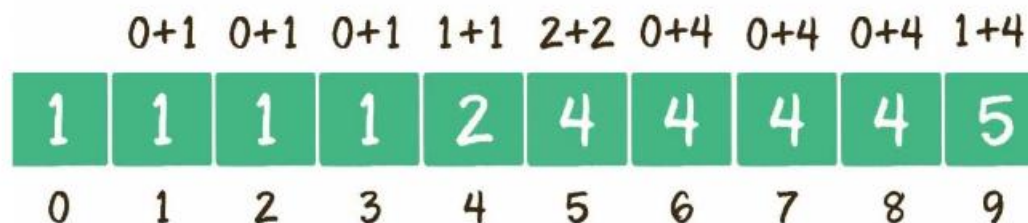
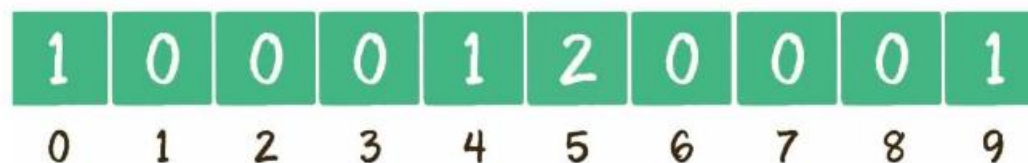
给出一个学生成绩表，要求按成绩从低到高进行排序，如果成绩相同，则遵循原表固有顺序。

姓名	成绩
小灰	90
大黄	99
小红	95
小白	94
小绿	95

有两个成绩为95分的学生，究竟小红在前还是小绿在前呢？



将之前的统计数组变形成为下面的样子。

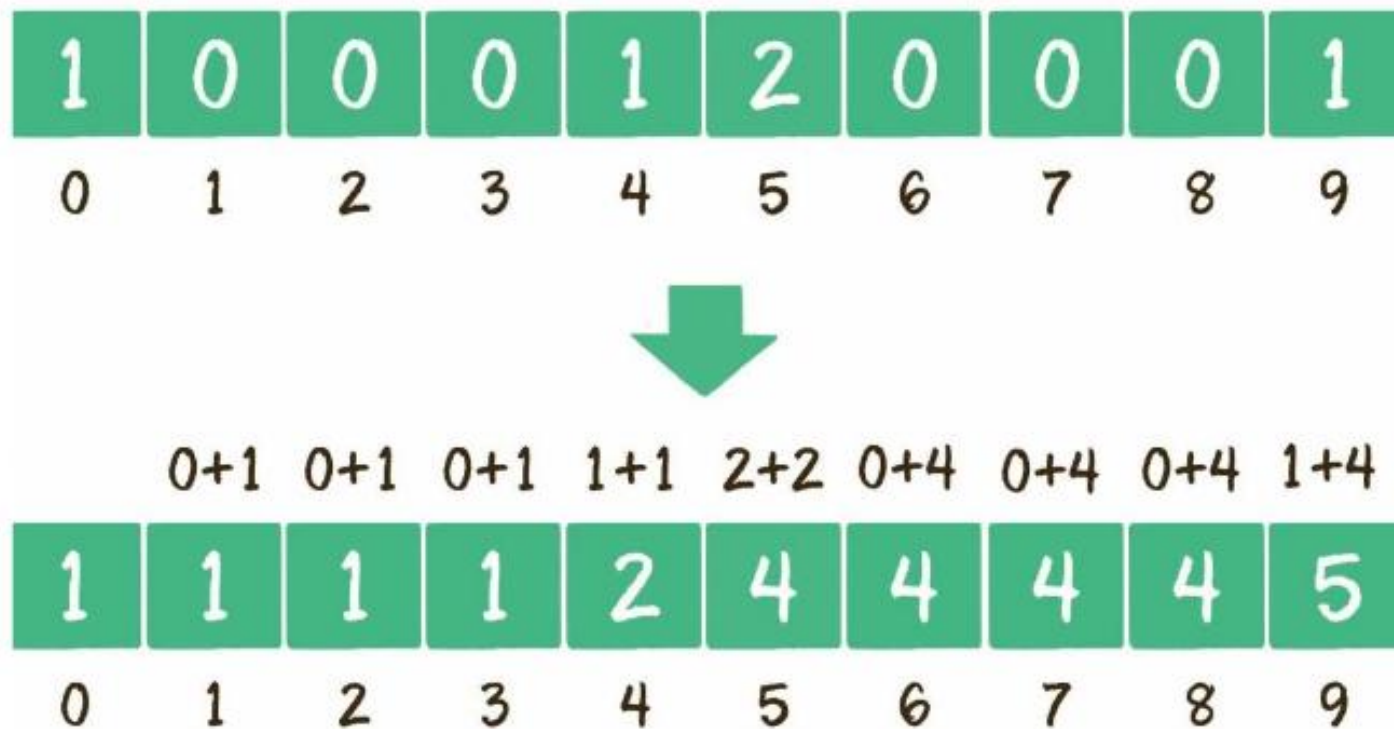


就是从统计数组的第2个元素开始，
每一个元素都加上前面所有元素之和。

相加的目的：是让统计数组存储的元素值，等于相应整数的最终排序位置的序号。

例如：下标是9的元素值为5，代表原始数列的整数9，最终的排序在**第5位**。

接下来，创建输出数组sortedArray，长度和输入数列一致。然后从后向前遍历输入数列。

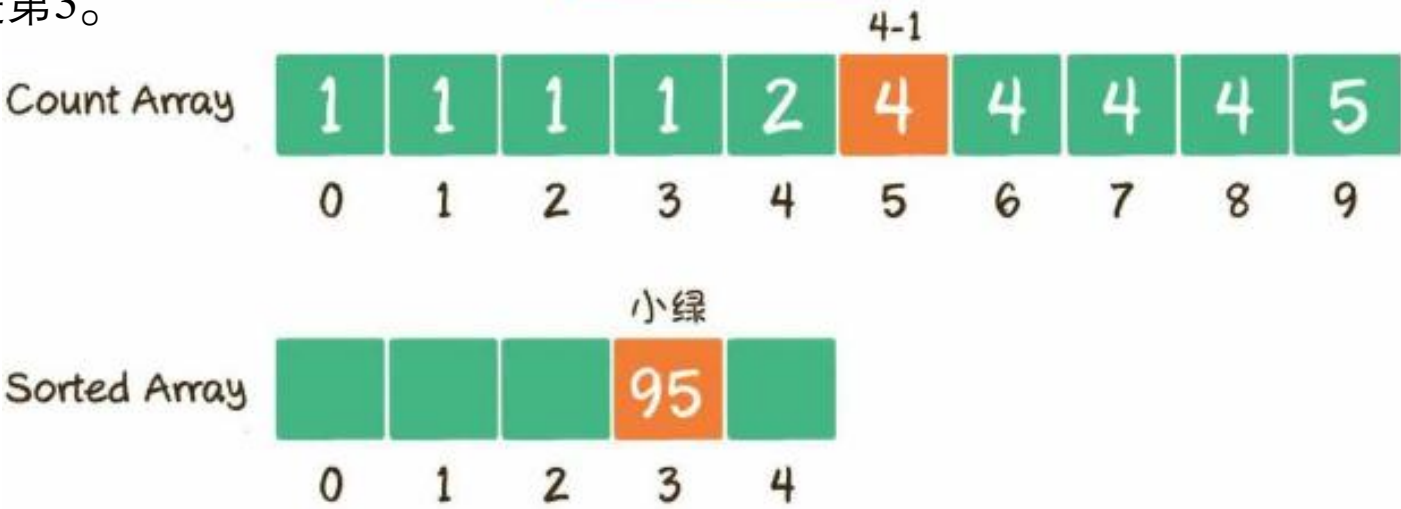


第1步：遍历成绩表最后一行的小绿同学的成绩。

小绿的成绩是95分，找到countArray下标是5的元素，值是4，代表小绿的成绩排名位置在第4位。

同时，给countArray下标是5的元素值减1,从4变成3，代表下次再遇到95分的成绩时，最终排名是第3。

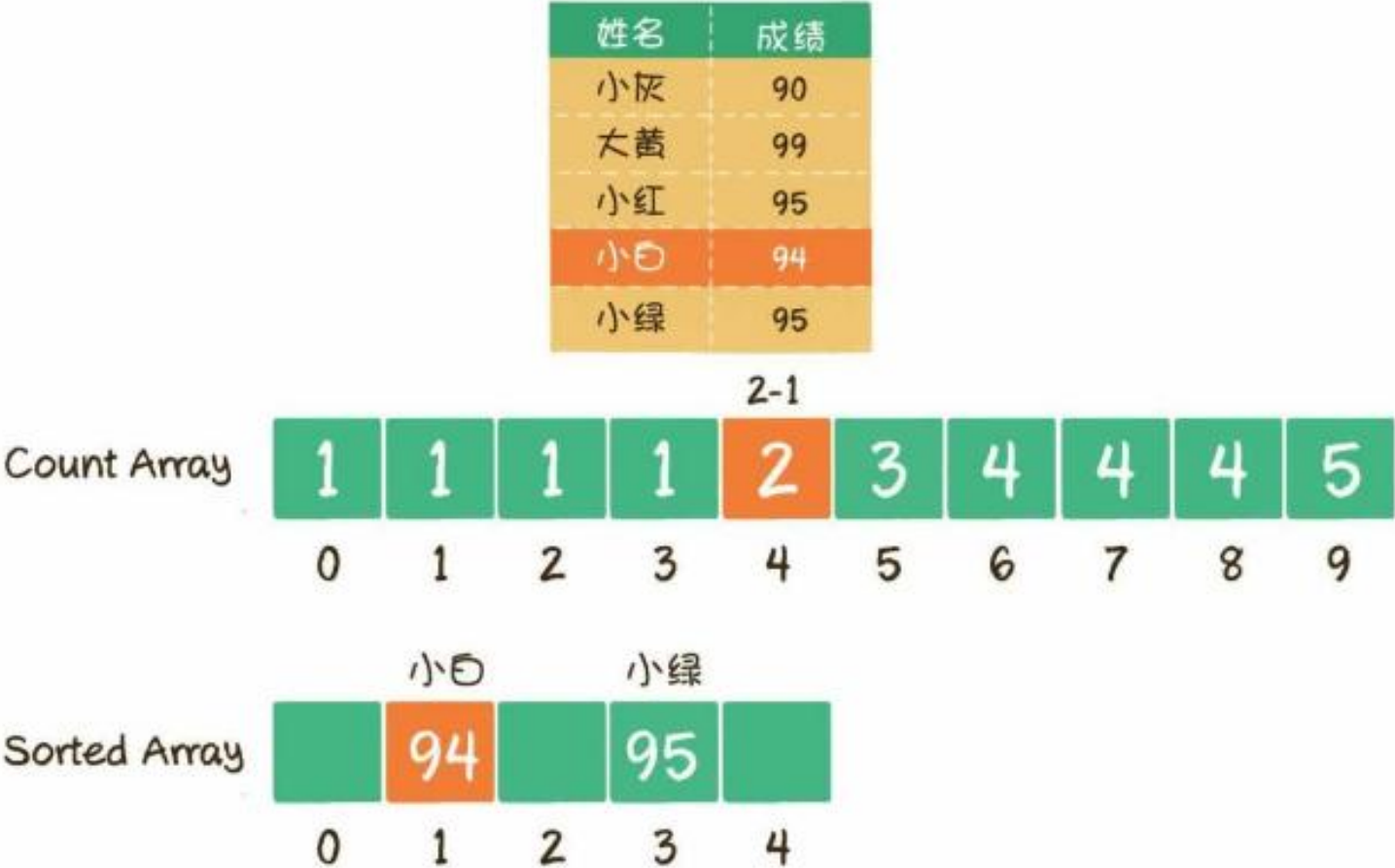
姓名	成绩
小灰	90
大黄	99
小红	95
小白	94
小绿	95



第2步，遍历成绩表倒数第2行的小白同学的成绩。

小白的成绩是94分，找到countArray下标是4的元素， 值是2，
代表小白的成绩排名位置在第2位。

同时， 给countArray下标是4的元素值减1， 从2变成1，
代表下次再遇到94分的成绩时（实际上已经遇不到了）， 最终排名是第1。



第3步，遍历成绩表倒数第3行的小红同学的成绩。

小红的成绩是95分，找到countArray下标是5的元素，值是3(最初是4,减1变成了3)，代表小红的成绩排名位置在第3位。

同时，给countArray下标是5的元素值减1，从3变成2，代表下次再遇到95分的成绩时(实际上已经遇不到了)，最终排名是第2。

姓名	成绩
小灰	90
大黄	99
小红	95
小白	94
小绿	95



同样是95分的小红和小绿就能够清楚地排出顺序了，也正因此，优化版本的计数排序属于**稳定排序**。

计数排序有它的局限性，主要表现为如下两点：

1.当数列最大和最小值差距过大时，并不适合用计数排序。

例如:给出20个随机整数，范围在0到1亿之间，这时如果使用计数排序，需要创建长度为1亿的数组。不但严重浪费空间，而且时间复杂度也会随之升高。

2.当数列元素不是整数时，也不适合用计数排序。

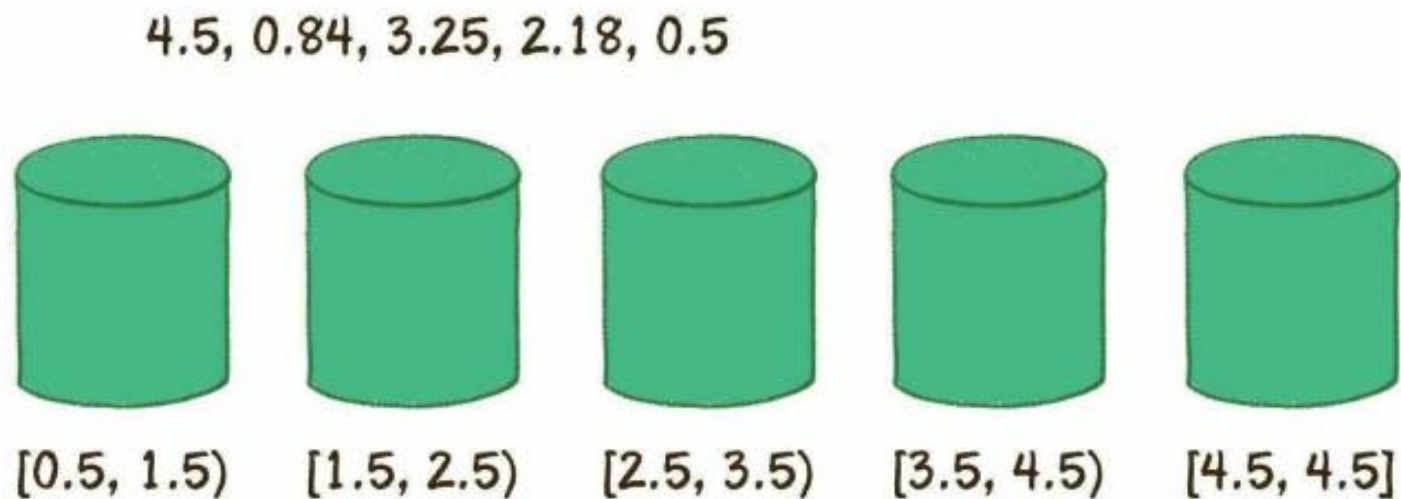
如果数列中的元素都是小数，如25.213，或0.00000001这样的数字，则无法创建对应的统计数组。这样显然无法进行计数排序。

桶排序

- 桶排序同样是一种线性时间的排序算法。
- 类似于计数排序所创建的统计数组，桶排序需要创建若干个桶来协助排序。
- 桶排序中所谓的“桶”，又是什么呢？
- 每一个桶（bucket）代表一个区间范围，里面可以承载一个或多个元素。

假设有一个非整数数列如下：4.5，0.84，3.25，2.18，0.5

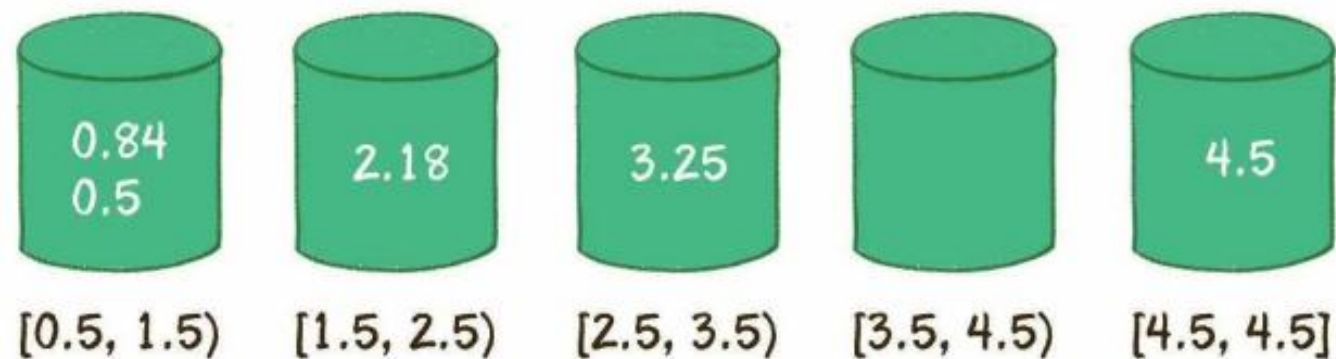
桶排序的第1步，就是创建这些桶，并确定每一个桶的区间范围。



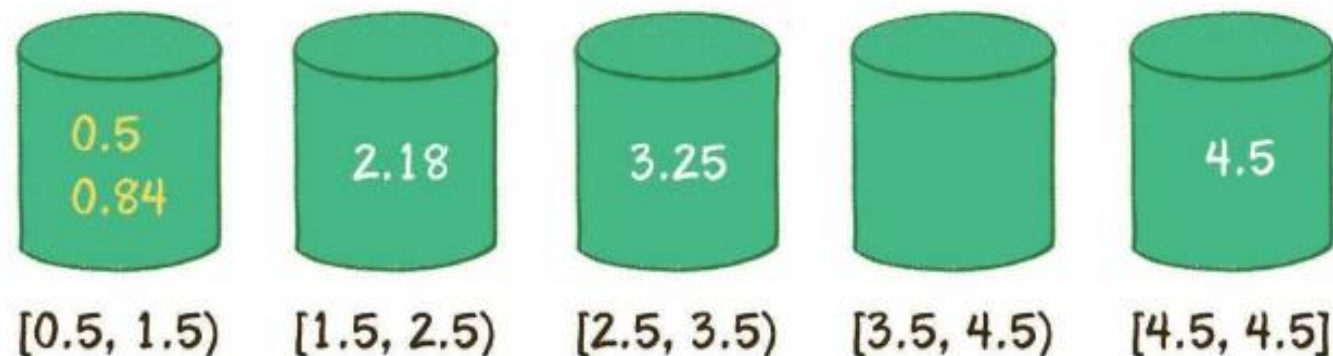
具体需要建立多少个桶，如何确定桶的区间范围，有很多种不同的方式。

- 例如，创建的桶数量等于原始数列的元素数量，除最后一个桶只包含数列最大值外，前面各个桶的区间按照比例来确定。
- **区间跨度=(最大值-最小值)/(桶的数量-1)**

第2步，遍历原始数列，把元素对号入座放入各个桶中。



第3步，对每个桶内部的元素分别进行排序（显然，只有第1个桶需要排序）。



第4步，遍历所有的桶，输出所有元素。

0.5, 0.84, 2.18, 3.25, 4.5

桶排序的复杂度分析

假设原始数列有 n 个元素，分成 n 个桶。下面逐步来分析一下算法复杂度。

第1步，求数列最大、最小值，运算量为 n 。

第2步，创建空桶，运算量为 n 。

第3步，把原始数列的元素分配到各个桶中，运算量为 n 。

第4步，在每个桶内部做排序，在元素分布相对均匀的情况下，所有桶的运算量之和为 n 。

第5步，输出排序数列，运算量为 n 。

- 桶排序的总体时间复杂度为 $O(n)$ 。
- 空间复杂度就很容易得到了，同样是 $O(n)$ 。

Homework

- 腾挪次数?
 - 合并排序, 快速排序, 堆排序
- 比较次数?
 - 都有哪些排序方法? 比较排序有哪些? 效率如何分析?

排序算法	平均时间复杂度	最坏时间复杂度	空间复杂度	是否稳定排序
冒泡排序	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
鸡尾酒排序	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
快速排序	$O(n \log n)$	$O(n^2)$	$O(\log n)$	不稳定
堆排序	$O(n \log n)$	$O(n \log n)$	$O(1)$	不稳定
计数排序	$O(n+m)$	$O(n+m)$	$O(m)$	稳定
桶排序	$O(n)$	$O(n \log n)$	$O(n)$	稳定



PART 08



线性选择问题



问题定义与复杂度下界

选择问题

定义与核心挑战

在无序数组中定位第K小元素，关键在于 **无需完全排序**，只追求 **局部有序**。



朴素思路

先排序，再取第K个。时间复杂度 $O(n \log n)$ 。



核心挑战

能否突破排序瓶颈，以 $O(n)$ 线性时间直接求解？

选择第K小问题

有没有更高效的方法呢？当然有！这就是分治法的应用场景。

比较模型与线性目标

任何基于比较的选择算法，都必须检查每个元素，因此存在理论下界。

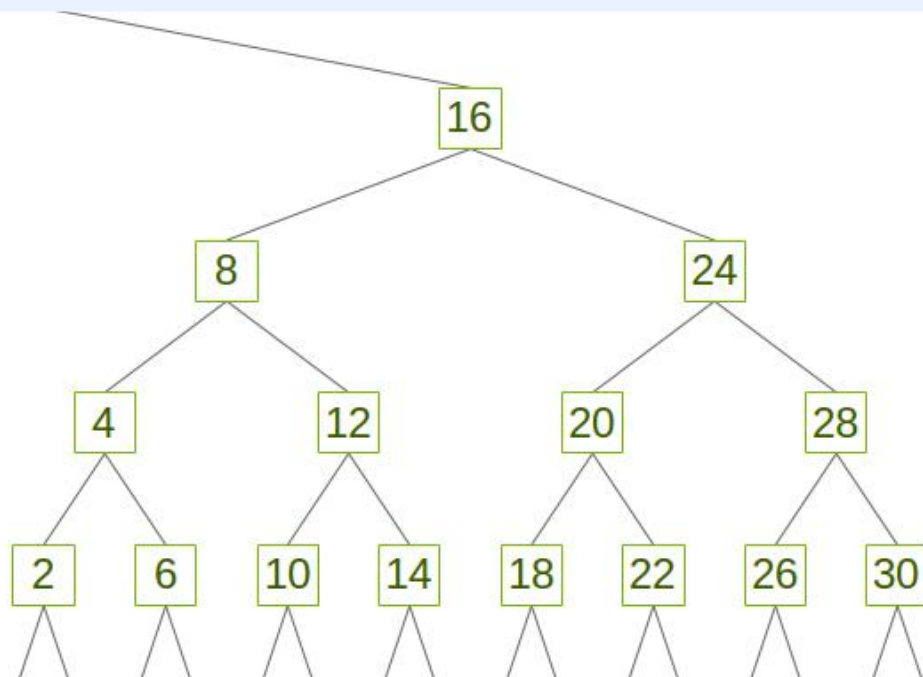


我们的目标不仅是性能提升，更是要设计出一种 **渐进最优** 的算法。

复杂度剖析：平均 vs 最坏

平均情况

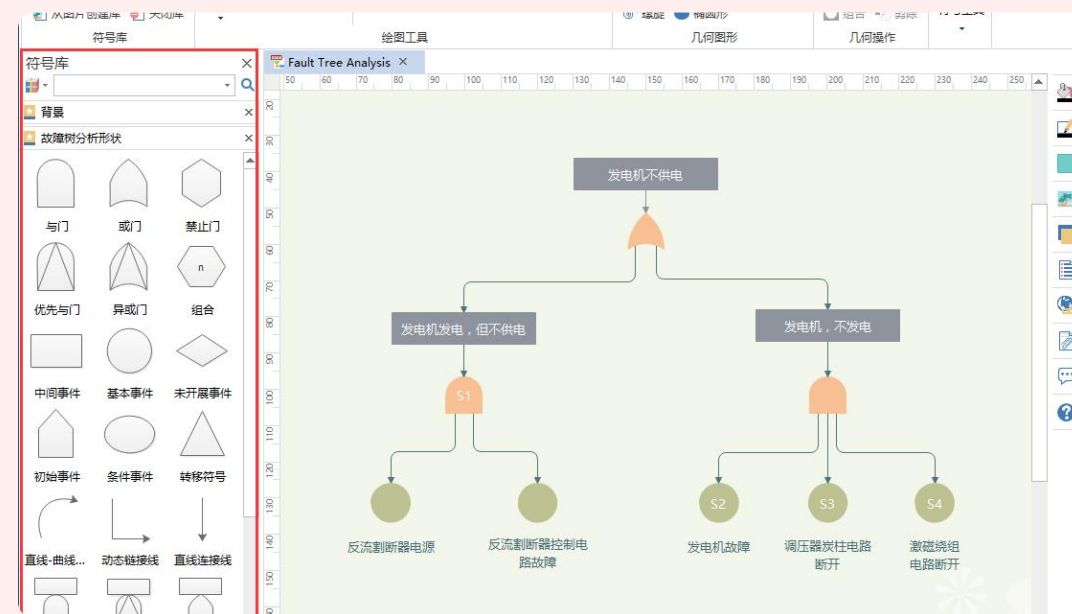
在“平均”的分裂下，递归深度为 $O(\log n)$ 。



$$T(n) = O(n)$$

最坏情况

对已排序数组且固定选首元为枢轴时，递归深度退化为 $O(n)$ 。



$$T(n) = O(n^2)$$

假设划分后基准的位置是 `p`：

- 如果 `p == K`，那么基准就是第K小的元素，直接返回。
- 如果 `p > K`，那么第K小的元素在左部分，递归处理左部分。
- 如果 `p < K`，那么第K小的元素在右部分，递归处理右部分，同时调整K的值 ($K = K - p - 1$)

示例演示

假设数组为 `[3, 2, 1, 5, 4]`，K=3:

1. 选择基准（例如第一个元素3），划分后数组可能变为 `[2, 1, 3, 5, 4]`（基准3的位置为2，索引从0开始）。
2. 因为 $K=3 > 2$ ，所以递归处理右部分 `[5, 4]`，同时 $K = K - 2 - 1 = 0$ （实际实现中可能需要调整索引方式）。
 - 更合理的做法是直接比较K与基准的索引位置。
3. 在右部分 `[5, 4]` 中选择基准（例如5），划分后为 `[4, 5]`，基准5的位置为1。
4. 因为原问题中K=3，基准3的位置为2（第3小），所以直接返回3？（这里需要更精确的索引处理，通常划分后基准的位置是其最终排序位置）

更准确的示例：

- 初始数组 `[3, 2, 1, 5, 4]`，K=3（希望找到第3小的元素，即3）。
- 选择基准3，划分后为 `[2, 1, 3, 5, 4]`，基准3的位置是2（索引从0开始）。
- 因为K=3（即第3小，索引为K-1=2），所以直接返回基准3。

```
1. import random
2.
3. def quick_select(arr, k):
4.     """
5.     使用分治法（快速选择）找到数组中第K小的元素。
6.     :param arr: 输入数组
7.     :param k: 第K小 (1-based index)
8.     :return: 第K小的元素
9.     """
10.    if len(arr) == 1:
11.        return arr[0]
12.
13.    # 随机选择一个基准值
14.    pivot = random.choice(arr)
15.
16.    # 分区操作
17.    left = [x for x in arr if x < pivot]
18.    right = [x for x in arr if x > pivot]
19.    pivots = [x for x in arr if x == pivot]
20.
21.    if k <= len(left):
22.        return quick_select(left, k)
23.    elif k <= len(left) + len(pivots):
24.        return pivots[0]
25.    else:
26.        return quick_select(right, k - len(left) - len(pivots))
27.
```

线性选择：中位数的中位数

BFPRT算法，又称中位数的中位数算法（Median of Medians Algorithm），是由Blum、Floyd、Pratt、Rivest和Tarjan提出的一种确定性选择算法。

它的核心目的是在未排序的列表中，找到第k小（或第k大）的元素，同时保证线性时间复杂度。

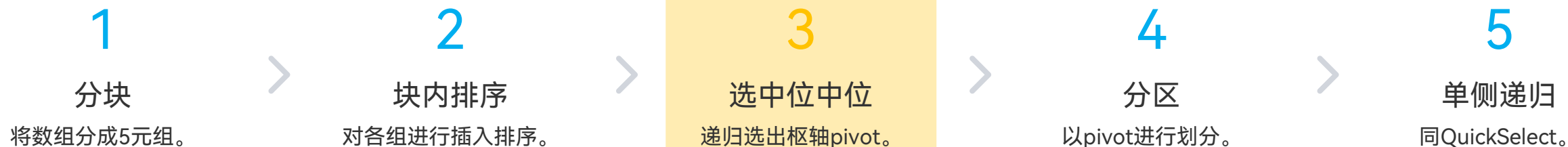
解决的核心问题

在处理大数据集时，我们经常需要快速找到某个位置上的元素，比如中位数或者某个百分位数。传统的排序方法虽然可以解决问题，但在面对海量数据时，效率就成了大问题。BFPRT算法应运而生，它通过一种巧妙的方式，确保了即使在最坏的情况下，也能以线性时间复杂度完成任务，这无疑是对传统方法的一次重大突破！

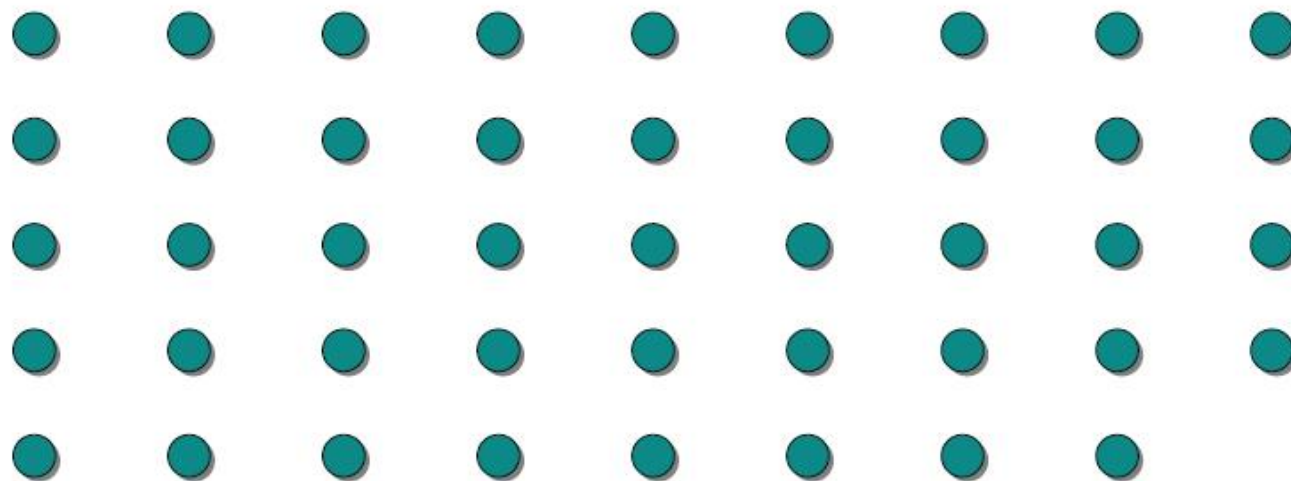
```
1. 开始
2. |
3. ▼
4. 数组分组 → 找每组中位数 → 递归找中位数的中位数 (med_of_meds)
5. |
6. ▼
7. 以med_of_meds为pivot划分数组
8. |
9. |— pivot是第k小? → 返回pivot
10. |
11. |— k < pivot位置 → 左半部分递归找第k小
12. |
13. |— k > pivot位置 → 右半部分递归找第(k - rank(pivot))小
```

BFPRT 算法：确定性线性选择

通过精心选取枢轴，保证每次递归规模不超过 $7n/10$ ，从而锁定 $O(n)$ 最坏时间。



Choosing the pivot

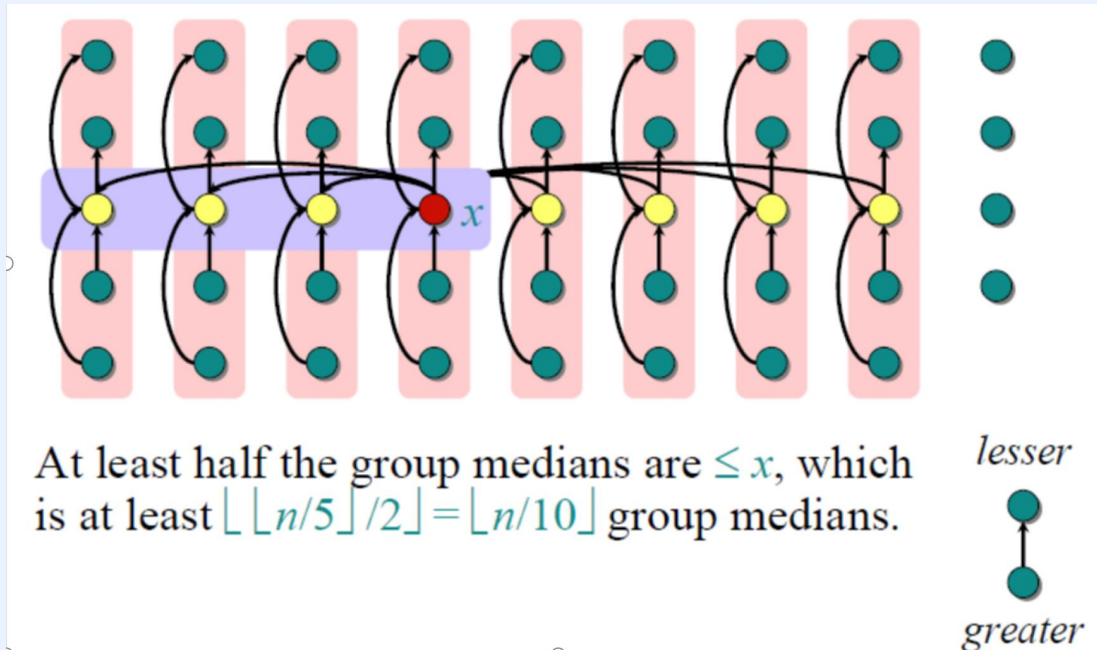


核心洞察：中位数的中位数 (MoM)

通过MoM选取的枢轴，能保证其“足够好”，从而确保问题规模被有效缩减。

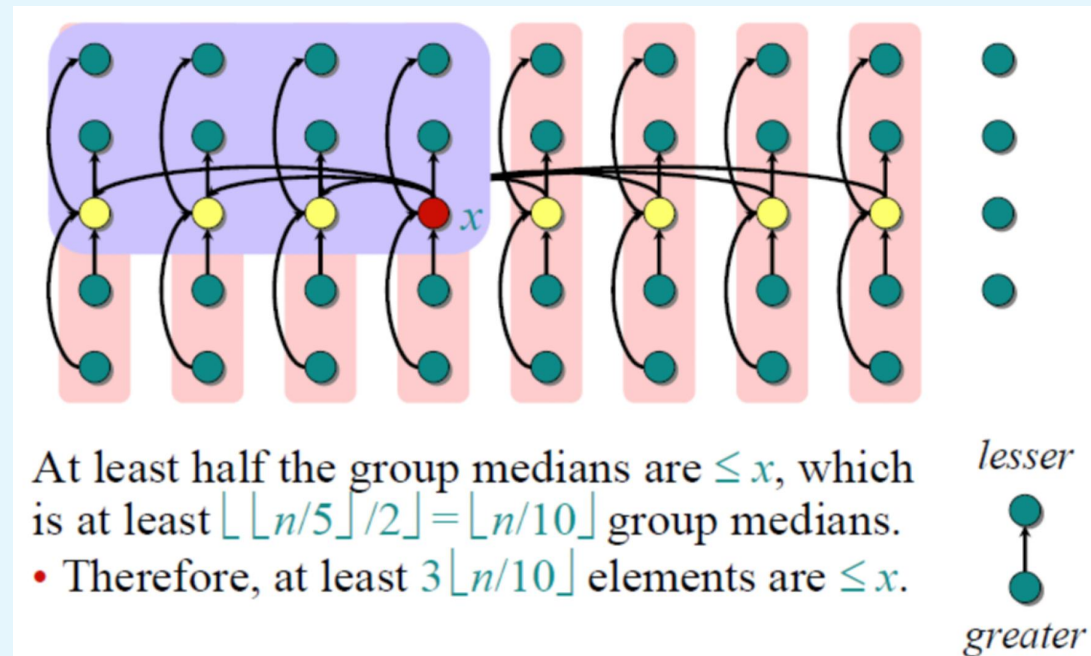
步骤1: 分组与排序

将 n 个元素分成 $\lfloor n/5 \rfloor$ 组，每组5个元素，并对每组进行排序。



步骤2: 选取MoM

取出所有组的中位数，构成一个新数组，然后递归调用BFPRT



关键性质：此枢轴至少比 $3n/10$ 个元素大，同时也至少比 $3n/10$ 个元素小，保证了递归规模。

复杂度递推与严谨证明

递归成本分解

总成本 = 选中位中位成本 + 处理剩余部分成本 + 线性其他工作

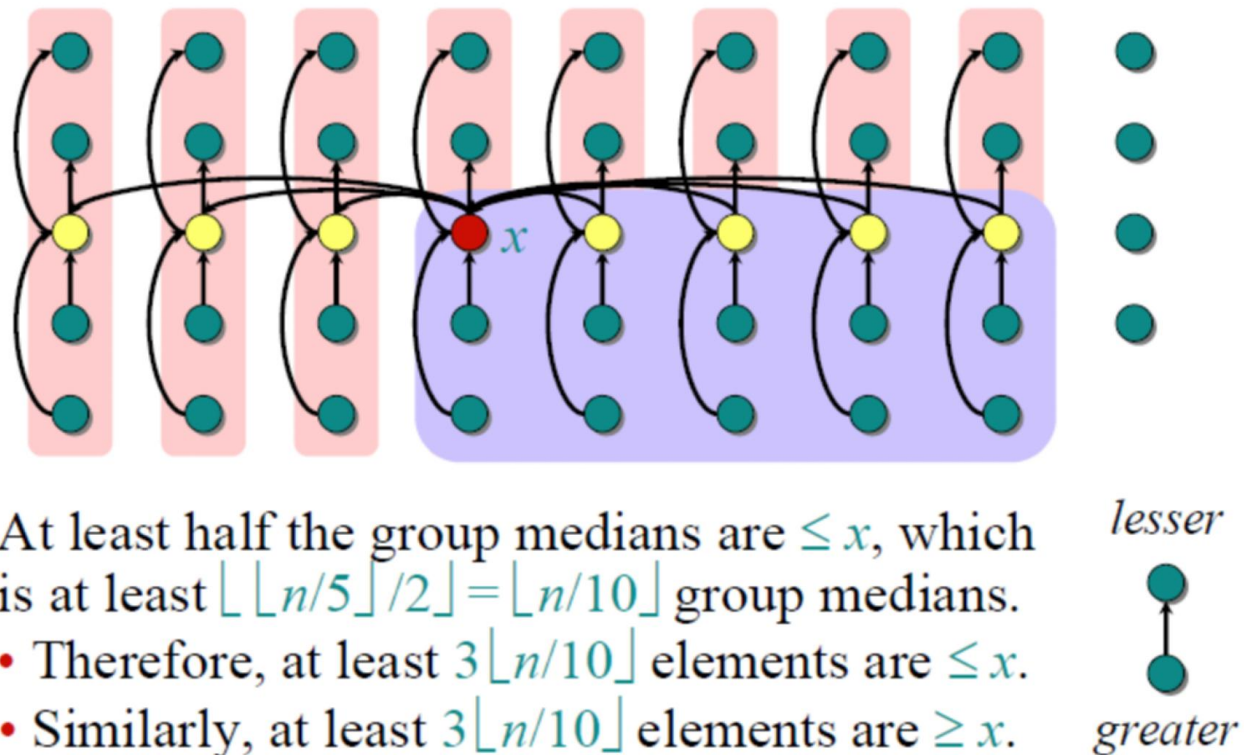
递推关系式

$$T(n) \leq T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right) + O(n)$$

结论：线性时间

通过递归树或代入法可证明，该递推式的解为 $T(n) = O(n)$ 。

虽然常数因子较高，但它保证了最坏情况下的线性性能。



实例演练：在数组中找第3小元素

以数组 `[8, 3, 2, 9, 1, 7, 5, 4, 6]` 为例，展示BFPRT算法流程。

1. 分块 `[8,3,2,9,1]` `[7,5,4,6]` → 组1: `[1,2,3,8,9]`, 组2: `[4,5,6,7]`

2. 排序 各组中位数: 3, 5

3. 选
MoM 中位数数组: `[3, 5]` → 递归选出枢轴 `pivot = 4`

4. 分区 以4为枢轴: `[1,2,3]` `[4]` `[5,6,7,8,9]` → `pivotIndex = 3`

5. 递归 因 $K=3 < \text{pivotIndex}$, 在左子数组 `[1,2,3]` 中递归找第3小元素。

实例演练与关键代码

```
1. def insertion_sort(arr):
2.     """插入排序, 用于对小数组进行排序"""
3.     for i in range(1, len(arr)):
4.         key = arr[i]
5.         j = i - 1
6.         while j >= 0 and key < arr[j]:
7.             arr[j + 1] = arr[j]
8.             j -= 1
9.         arr[j + 1] = key
10.    return arr
11.
12. def get_median(arr):
13.     """获取数组的中位数"""
14.     sorted_arr = insertion_sort(arr)
15.     return sorted_arr[len(arr) // 2]
16.
```

```
43. # 示例测试
44. arr = [3, 2, 1, 5, 4, 9, 8, 7, 6, 0, 12, 11, 10, 14, 13, 15]
45. k = 8
46. result = bfprt(arr, k)
47. print(f"数组中第{k}小的元素是: {result} 🎉")
```

```
17. def bfprt(arr, k):
18.     """BFPRT算法主函数, 找到数组中第k小的元素"""
19.     if len(arr) <= 5:
20.         return insertion_sort(arr)[k - 1]
21.
22.     # 分组并获取中位数数组
23.     medians = []
24.     for i in range(0, len(arr), 5):
25.         group = arr[i:i + 5]
26.         medians.append(get_median(group))
27.
28.     # 递归找到中位数的中位数作为pivot
29.     pivot = bfprt(medians, len(medians) // 2 + 1)
30.
31.     # 划分操作
32.     low = [x for x in arr if x < pivot]
33.     high = [x for x in arr if x > pivot]
34.
35.     # 递归搜索
36.     if k <= len(low):
37.         return bfprt(low, k)
38.     elif k > len(arr) - len(high):
39.         return bfprt(high, k - (len(arr) - len(high)))
40.     else:
41.         return pivot
42.
```

实战对比：快速选择 vs BFPRT

快速选择 (QuickSelect)

- ✓ 平均时间: $O(n)$
- ✗ 最坏时间: $O(n^2)$
- ✓ 常数因子: 小
- ✓ 实现难度: 简单

适用场景：对平均性能要求高，且输入数据随机性较强的场景。

BFPRT

- ✓ 最坏时间: $O(n)$ (保证)
- ✓ 稳定性: 极高
- ✗ 常数因子: 大
- ✗ 实现难度: 复杂

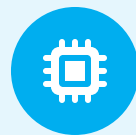
适用场景：对最坏情况性能有严格要求的实时或关键系统。

工程优化与并行化视角



常数优化

调整块大小、对小数组使用插入排序、消除尾递归。



并行化

分块和组内排序阶段可并行化，适合GPU加速。



混合策略

结合随机pivot与BFPRT，兼顾平均与最坏性能。



稳健部署

在超大规模或实时系统中，提供性能保证。

从选择到秩序：算法思想再升华

分治思想在随机与确定性两端开花结果，其核心在于 **枢轴质量** 与 **问题缩减**。



概率的力量

快速选择展示了随机化算法的强大威力。



精细的设计

BFPRT彰显了确定性算法的精妙设计。



思想的迁移

将分治与线性审视迁移至更多计算难题。

选择算法的思想在 **median filter**, **顺序统计**, **数据流** 等场景中持续发挥影响。



PART 09



分治寻邻：从一条线到一张图



最接近点对：定义与度量

在度量空间中，寻找彼此的“灵魂伴侣”

问题定义

在计算几何中，**最接近点对问题**要求我们在一个包含 **n 个点** 的集合中，找到一对点，使得它们之间的距离 **最小**。

输入：点集 $P = \{p_1, p_2, \dots, p_n\}$

输出：点对 (p_i, p_j) 使得 $d(p_i, p_j)$ 最小

距离度量

距离函数 $d(p, q)$ 必须满足非负、对称和三角不等式，为算法提供理论基础。

一维空间 (直线)

点表示为实数，距离为绝对值之差：

$$d(x_1, x_2) = |x_2 - x_1|$$

假设我们有以下一维点集：

```
1. points = [1, 5, 3, 7, 2]
```

我们需要找到距离最近的两个点。

1. 列举所有点对：

- $(1, 5)$: $|1 - 5| = 4$
- $(1, 3)$: $|1 - 3| = 2$
- $(1, 7)$: $|1 - 7| = 6$
- $(1, 2)$: $|1 - 2| = 1$
- $(5, 3)$: $|5 - 3| = 2$
- $(5, 7)$: $|5 - 7| = 2$
- $(5, 2)$: $|5 - 2| = 3$
- $(3, 7)$: $|3 - 7| = 4$
- $(3, 2)$: $|3 - 2| = 1$
- $(7, 2)$: $|7 - 2| = 5$

2. 找出最小距离：

- 最小距离为 **1**，对应的点对有 **$(1, 2)$** 和 **$(3, 2)$** 。

时间复杂度为 $O(n^2)$ ，无法处理大规模数据。

一维排序法: $O(n \log n)$ 的优雅模板

排序, 让最近邻无所遁形



1. 排序

将n个点按坐标值从小到大排序。



2. 扫描

线性扫描排序后的数组, 比较相邻两点的距离。



3. 结论

遍历过程中的最小距离即为答案。

核心洞察: 排序后, 最近点对必然相邻。此算法优雅高效, 但核心思想无法直接迁移到二维, 因为平面上的点没有天然的全序关系。

分治框架：平分 · 递归 · 合并

将归并排序的思想迁移到一维最近点问题

1

平分

找到坐标的中位数 m ，将直线分成左右两半。

2

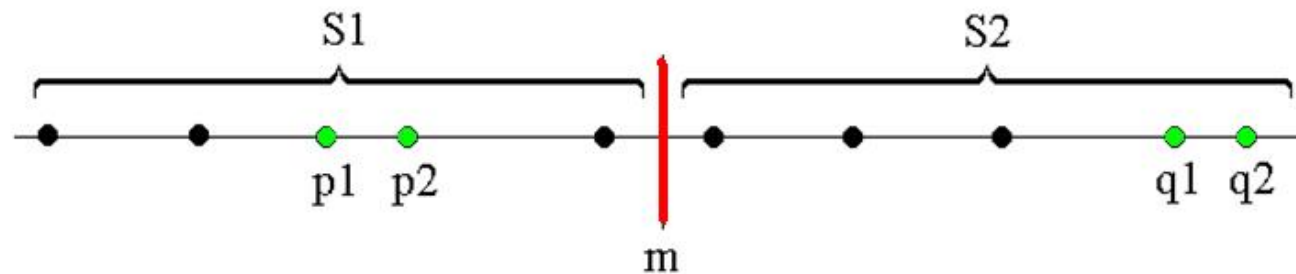
递归

递归地在左右两半求解最近点对距离 δ_1 和 δ_2 。

3

合并

检查跨越切分点的点偶，更新最小距离 δ 。



关键优化：合并时，只需考察距切分点不超过当前最小距 δ 的点，即可将合并代价维持在 $O(n)$ 。

合并关键：跨越区间才需比

通过剪枝，将合并的代价从 $O(n^2)$ 降至 $O(n)$

1. 确定考察范围

设左右递归求得的最小距离为 δ 。跨越切分线 m 的点
对，其距离才可能小于 δ 。

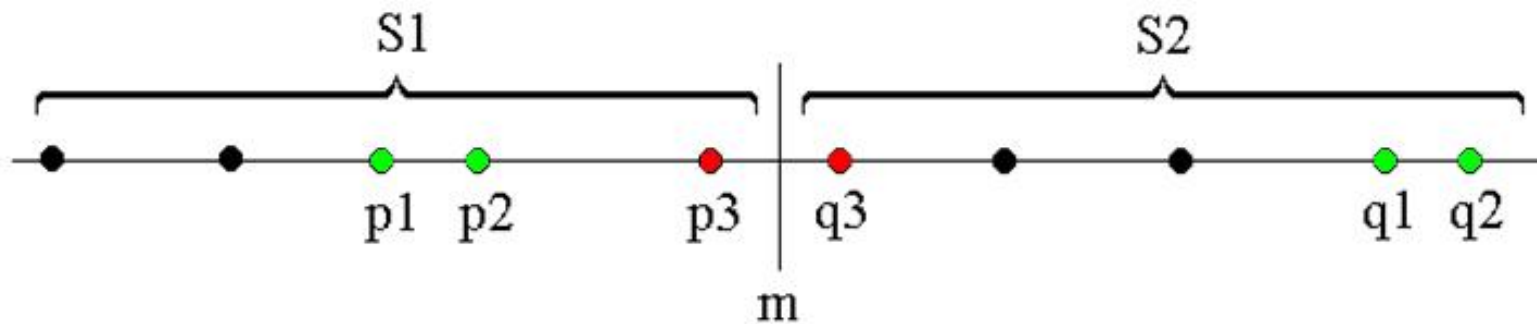
因此，只需收集位于 $[m-\delta, m+\delta]$ 区间内的点。



2. 线性扫描定乾坤

将窄带内的点按坐标排序。对于带内任意一点，只需与
常数个后继点比较。

因为窄带内不可能密集分布，保证了合并耗时为 $O(n)$ 。



一维最接近点对的分治算法closestPair1D

输入：点集S, 最近距离d

输出：找到与否，最小距离

```
1 bool closestPair1D(S,d)
2     n ← |S|
3     if n < 2:
4         d ← ∞
5         return false, d
6     // 找到S的中位数点m
7     m ← median(S)
8     S1 ← {x ∈ S | x ≤ m}, S2 ← {x ∈ S | x > m}
9     closestPair1D(S1,d1) // 递归找到S1中最小距离
10    closestPair1D(S2,d2) // 递归找到S2中最小距离
11    p ← max(S1)
12    q ← min(S2)
13    d ← min(d1, d2, q-p)
14    return true, d
```

复杂度分析

$$T(n) = \begin{cases} O(1) & n < 4 \\ 2T(n/2) + O(n) & n \geq 4 \end{cases}$$

$$T(n) = O(n \log n)$$

为防止退化现象可以通过分治法中“平衡子问题”的方法加以解决。即通过适当选择分割点，使S1和S2中有大致相等个数的点。自然地，我们会想到用S的n个点的坐标的中位数来作分割点。此时，算法的时间复杂度为 $T(n)=T(n/2)+O(n)$ ，可解得为 $T(n)=O(n \log n)$ 。

平面距离：欧氏公式

进入二维世界，距离的计算从绝对值升级为勾股定理。

距离公式

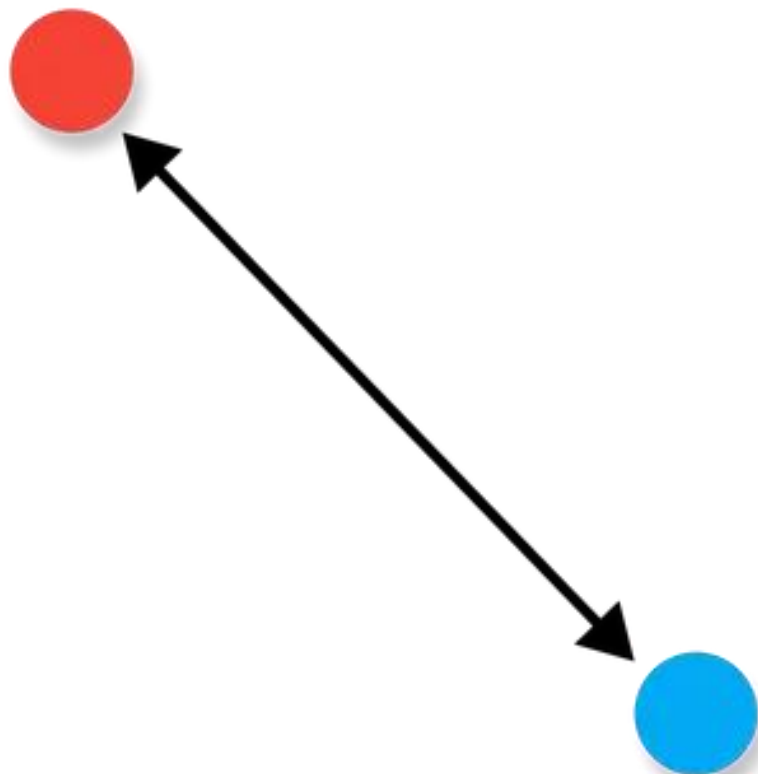
对于点 $p(x_1, y_1)$ 和 $q(x_2, y_2)$ ，其欧氏距离为：

$$d(p, q) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

优化技巧

为加速计算，比较距离大小时可省略开方，直接比较平方值，结果不变。

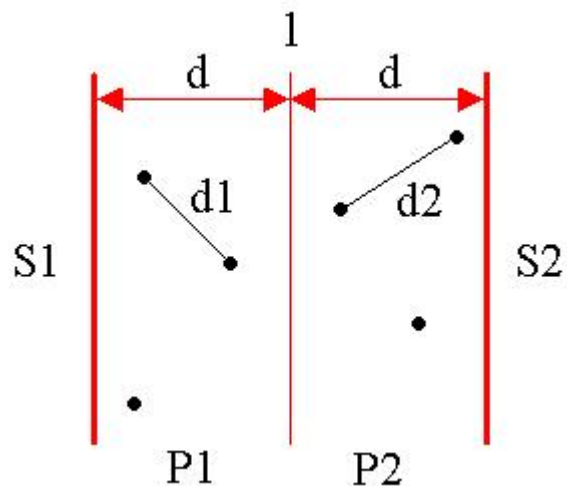
Euclidean



二维升级：复杂度瓶颈

暴力两两对比： $O(n^2)$ 的代价

当数据规模增长，平方级的复杂度将带来灾难性的性能瓶颈。

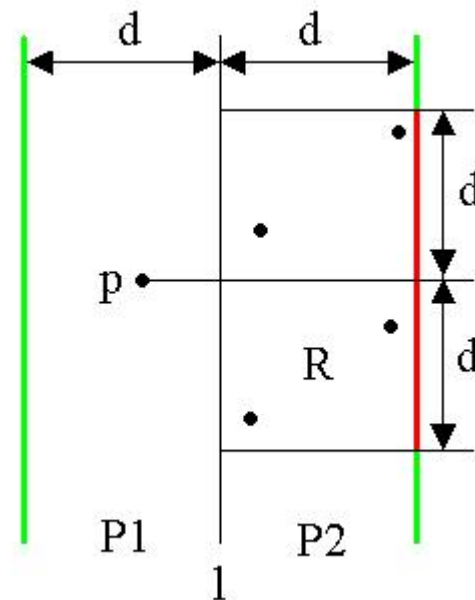


算法描述

枚举所有 $C(n, 2)$ 对点，计算并比较距离。

时间复杂度为 $O(n^2)$ ，无法处理大规模数据。

VS



突破性能瓶颈

时间复杂度为 $O(n \log n)$ ，可适用于处理大规模数据。

例如，对于 10万个点，需要进行约 50亿次距离计算和比较。这宣告了暴力法的不可行性，我们必须寻求更优策略。

平面分治：按 x 排序与中位切分

将一维的分治思想推广到二维，第一步是找到合适的划分方式。

预处理：全局排序

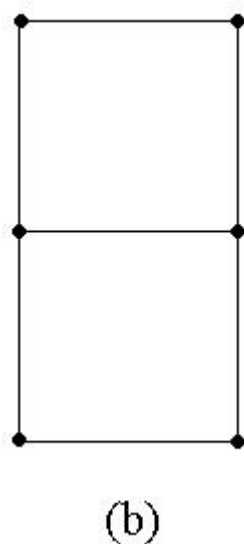
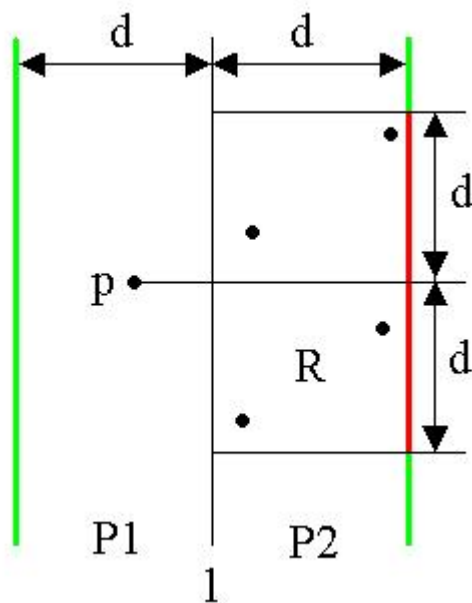
在进入递归前，对所有点按x 坐标进行一次性预排序，并建立索引。

此举避免了在递归的每一层都进行排序，将潜在的 $O(\log^2 n)$ 开销降为 $O(\log n)$ 。

划分：中位垂线

找到 x 坐标的中位数，并作一条垂直线 $x = m$ 作为分割线，将平面划分为左右两个点集大致相等的半平面。

递归求解左右两半的最近点对距离，记为 δ 。



◆考虑P1中任意一点p，它若与P2中的点q构成最近点对的候选者，则必有 $\text{distance}(p, q) < d$ 。满足这个条件的P2中的点一定落在一个 $d \times 2d$ 的矩形R中

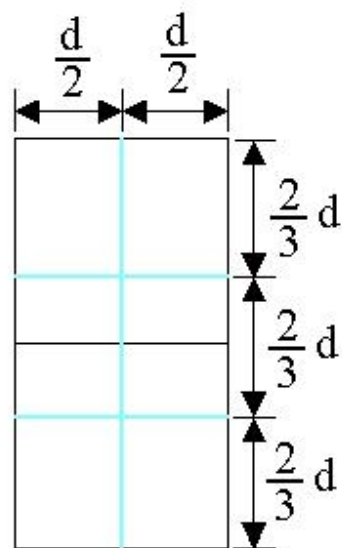
◆由d的意义可知，P2中任何2个S中的点的距离都不小于d。由此可以推出矩形R中最多只有6个S中的点。

◆因此，在分治法的合并步骤中最多只需要检查 $n/2 \times 6 = 3n$ 个候选者

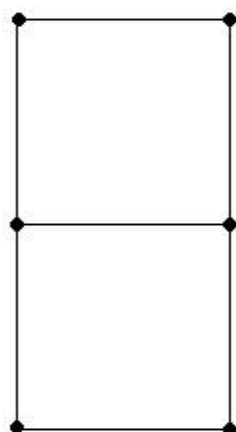
六邻定理：常数窗口的正确性

为什么只需检查6个邻居？这是保证算法线性复杂度的核心。

鸽巢原理的应用



(a)



(b)

证明：

将矩形R的长为 $2d$ 的边3等分，将它的长为 d 的边2等分，由此导出6个 $(d/2) \times (2d/3)$ 的矩形。若矩形R中有多于6个S中的点，则由鸽舍原理易知至少有一个 $(d/2) \times (2d/3)$ 的小矩形中有2个以上S中的点。设 u, v 是位于同一小矩形中的2个点，则

$$(x(u) - x(v))^2 + (y(u) - y(v))^2 \leq (d/2)^2 + (2d/3)^2 = \frac{25}{36} d^2$$

$\text{distance}(u, v) < d$ 。这与 d 的意义相矛盾。

最接近点对问题： 二维Strip

实施步骤

现在，让我们将这些策略整合到实施步骤中：

1. 预处理

- 按照x坐标对点进行排序。
- 将点划分到不同的Strip区域中（如果需要的话，但在此讲中我们主要关注单个Strip区域内的筛选）。
- 按照y坐标对Strip区域内的点进行排序。

2. 初始化

- 初始化一个空列表来存储候选点对。
- 初始化一个变量来记录当前的最小距离 δ （可以从无穷大开始，或者从其他部分算法中获得）。
- 初始化一个“窗口”或类似的数据结构来动态地存储和更新在y坐标上与之相差不超过 δ 的点。

3. 遍历与比较

- 遍历Strip区域内的每一个点。
- 对于当前点，使用“窗口”来找到所有在y坐标上与之相差不超过 δ 的点。
- 将这些点与当前点进行比较，如果找到更小的距离，则更新 δ 。
- 更新“窗口”以包含或排除相应的点（取决于具体的实现方式）。

4. 输出结果

- 遍历完成后，输出找到的最小距离 δ 以及对应的点对（如果需要的话）。

输入：点集S, 最近距离d

输出：最小距离

```
function closestPair2D (S,d)

1: n  $\leftarrow$  |S|;

2: if n < 2 :

3:         d  $\leftarrow$   $\infty$ ;

4:         return d;

5: // 找到S的中位数点m

6: m  $\leftarrow$  median(S)

7: S1  $\leftarrow$  {x  $\in$  S|x $\leq$ m}

8: S2  $\leftarrow$  {x  $\in$  S|x>m}

9: // 递归求解其内部的最小距离

10:         d1  $\leftarrow$  closestPair2D (S1,d1);

11:         d2  $\leftarrow$  closestPair2D (S2,d2);

12:         md1  $\leftarrow$  min(d1, d2)

13:         // 同时， 需要判断跨分割线的点的最小距离

14:         md2  $\leftarrow$  closestPairAcross2D(m, S1, S2, md1)

15:         distance  $\leftarrow$  min(md1 , md2);

16:         return distance;

17:

18: // 检查跨区域最小值时， 仅仅需要比较一个带状区域中的点以确定最小值

19: function closestPairAcross2D(m, S1, S2, md1)

20:         // 找到可能存在最小距离点的带状区域， 他们的x值距离m小于最小距离

21:         strip  $\leftarrow$ 

22:         // 寻找该区域中的最小距离并返回

23:         md2  $\leftarrow$  带状区域中的点对最小距离

24:         return md2
```

合并窄带: strip 内 y 有序扫描

合并阶段的核心在于高效检查“跨区”的点，关键在于构建一个“窄带”(strip)。

1

构建窄带 (Strip)

收集所有与分割线 $x = m$ 的水平距离在 δ 之内的点。

2

y 坐标排序

将窄带内的所有点，按 y 坐标进行排序(可利用预排序的 y 数组)。

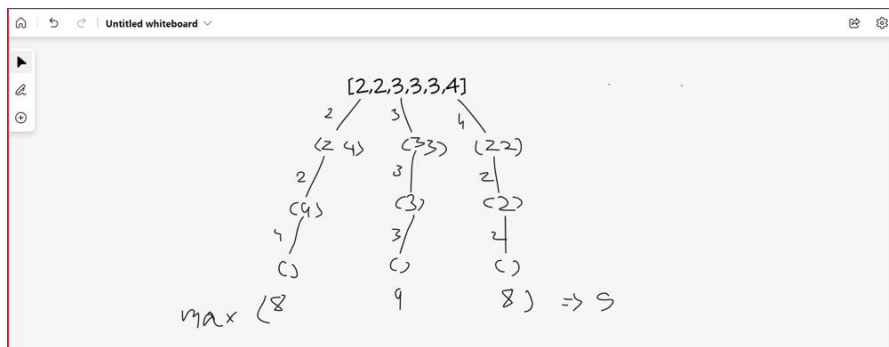
3

线性扫描

对排序后的每个点，只需检查其与后续常数个点的距离。

低效做法 (避免)

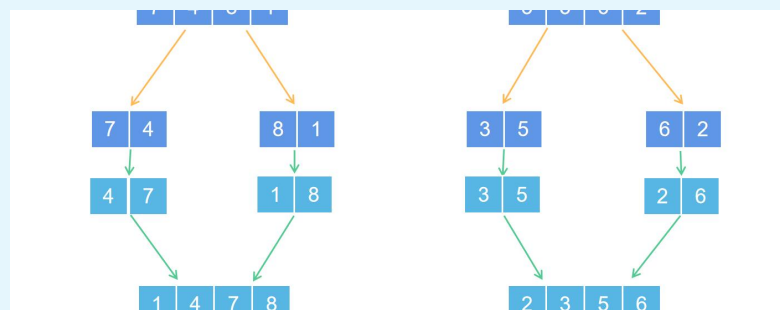
在递归的每一层，都对当前点集按 y 坐标进行排序。



这将导致总的时间复杂度升至 $O(n \log^2 n)$ 。

高效做法 (推荐)

在顶层对所有点按 y 坐标预排序，递归时传递指针或索引区间。



构建 strip 时，只需从 y 有序数组中筛选，保持线性合并。

一次预排序: 免递归重排 一个关键的工程优化，将复杂度中的 $\log^2 n$ 因子降为 $\log n$ 。

复杂度方程: $T(n) = 2T(n/2) + O(n)$

分治策略的完美体现, 最终成就了 $O(n \log n)$ 的高效算法。

递归式分析

 $2T(n/2)$: 将问题分解为两个规模为 $n/2$ 的子问题。

 $O(n)$: 合并步骤的代价, 包括构建 strip 和线性扫描。

主定理求解

该递归式符合主定理 (Master Theorem) 的情况 2:

$$T(n) = aT(n/b) + f(n)$$

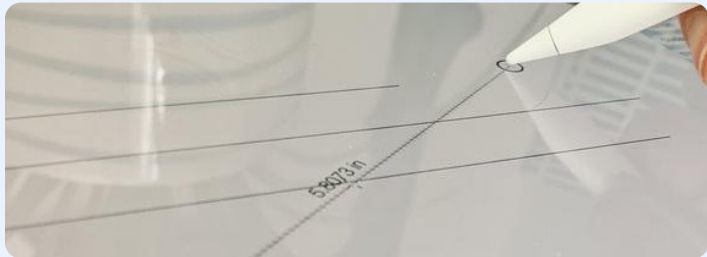
其中 $a=2$, $b=2$, $f(n)=O(n)$ 。

因为 $f(n) = \Theta(n^{\log_2 2}) = \Theta(n)$,

所以 $T(n) = \Theta(n \log n)$ 。

从平面到空间：思想再迁移

“平分-递归-剪枝-合并”的分治范式，是跨越维度的通用解题钥匙。



一维

排序后，最近邻必相邻。

$O(n \log n)$

oints on the 2-dimensional vertical plane are located such that B is 8m vertically above point C is 4m to the right of B. An inclined force creates a moment about A of 480 kN.m clockwise of 288 kN.m counter clockwise about point B and zero moment about point C. Determine the value of the force.

a) 36 b) 36 c) 120 d) 200

二维

构建窄带 (strip)，检查6个邻居。

$O(n \log n)$



三维

构建厚板 (slab)，检查26个邻居。

$O(n \log n)$

核心思想不变：通过**预排序**和**空间剪枝**，将合并的代价控制在**线性**，从而保证整体效率。



PART 10



用分治法实现循环赛日程表



循环赛三约束与 2^k 规模

将 **排列爆炸** 的日程规划，转化为 **优雅的分治蓝图**



两两交锋

$n=2^k$ 位选手，需与其他 $n-1$ 人各赛一次。



每日一赛

每位选手每天最多进行一场比赛。



$n-1$ 天完赛

整个循环赛必须在 $n-1$ 天内完成。

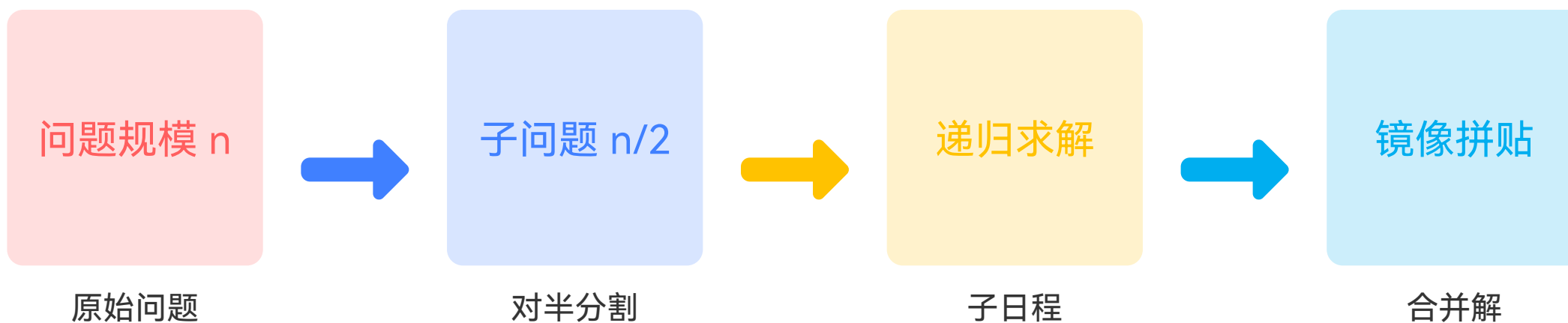


挑战：排列爆炸

直接构造日程面临组合爆炸，暴力枚举不可行。这引出了用**分治法**将问题规模减半的核心思路。

分治策略：一分为二填两边

将 $O(n^2)$ 的直接构造，转化为 $O(n \log n)$ 的递归复制



核心思想：将选手均分成两半，先各自排布子日程，再通过**镜像旋转**映射到另一半，完成合并。

递归构造与合并

基础情形

$k=1$ 的 1×2 日程

这是递归的原子解，也是构建一切的基石。

- ✓ 选手数: $n = 2^1 = 2$
- ✓ 天数: $n-1 = 1$
- ✓ 对阵: 唯一一场比赛

比赛日程表

Day 1

-

1 vs 2

-

子表复制：左上驱动右上

将左上半区的子日程，平移至右上半区，选手编号整体加 $n/2$

左上半区 (子日程)

1 vs 2	3 vs 4
1 vs 3	2 vs 4
1 vs 4	2 vs 3



右上半区 (复制后)

5 vs 6	7 vs 8
5 vs 7	6 vs 8
5 vs 8	6 vs 7

通过索引映射，无需重新计算，即可维持“每天每人一次”的约束。

镜像映射：左下右下补全

将左上、右上子表行列转置，分别填入左下、右下区域

左上 (子日程)

1vs2, 3vs4 ...

右上 (复制)

5vs6, 7vs8 ...

左下 (左上转置)

2vs1, 4vs3 ...

右下 (右上转置)

6vs5, 8vs7 ...

此步骤确保新区域的对阵与已排区域互补，为每位选手生成新对手，避免重复。

Invariant: 三约束全程守恒

1	2	3	4	5	6	7	8
2	1	4	3	6	5	8	7
3	4	1	2	7	8	5	6
4	3	2	1	8	7	6	5
5	6	7	8	1	2	3	4
6	5	8	7	2	1	4	3
7	8	5	6	3	4	1	2
8	7	6	5	4	3	2	1

递归合并过程是约束的“守恒定律”。

- ✓ 每天每人一次：复制与转置操作保证了每日每行仅一场比赛。
- ✓ 两两交锋一次：镜像映射确保对阵集合互异，无重复。
- ✓ 总天数 $n-1$ ：递归定义确保了日程规模的正确性。

最终表格构成一个拉丁矩形，是问题的一个可行解。

复杂度分析



时间复杂度
 $O(n \log n)$

递归深度 $\log_2 n$ ，每层 $O(n)$ 填充代价。

VS



空间复杂度
 $O(n^2)$

存储完整表格需 $n \times (n-1)$ 条目。

相比暴力排列的 $O(n!)$ ，分治在时间与实现复杂度上拥有双重优势，且已接近理论下限。

实例推演：k=3 八人日程逐日展开

Day 1

1 vs 2

3 vs 4

5 vs 6

7 vs 8

Day 2

1 vs 3

2 vs 4

5 vs 7

6 vs 8

Day 3

1 vs 4

2 vs 3

5 vs 8

6 vs 7

通过跟踪1号选手的每日对手(2, 3, 4, 5, 6, 7, 8)，可直观感受“与其他7人各一次”的覆盖过程。

自顶向下拼表

步骤1: 初始化日程表

创建一个二维数组或表格，用于存储每支队伍在每一天的比赛安排。行代表队伍，列代表天数。初始时，所有比赛安排为空。

步骤2: 安排第一轮比赛

将队伍分成两半，例如前半部分和后半部分。

安排第一轮比赛，使得前半部分的每支队伍与后半部分的对位队伍比赛。

例如：

队伍1 vs 队伍5
队伍2 vs 队伍6
队伍3 vs 队伍7
队伍4 vs 队伍8

步骤3: 填充剩余轮次

- 对于剩余的每一天，采用循环移位的方法来安排新的比赛。
- 保持前半部分队伍的位置不变，将后半部分队伍的顺序循环移动一位，以安排新的对手。

例如，第二天：

队伍1 vs 队伍6
队伍2 vs 队伍7
队伍3 vs 队伍8
队伍4 vs 队伍5（此时后半部分循环移动）

步骤4: 确保所有比赛都被安排

- 重复上述过程，直到所有可能的比赛组合都被安排。
- 检查日程表，确保每支队伍都与其他每支队伍比赛了一次，且没有重复或遗漏。

```
1. def schedule_round_robin(teams):
2.     n = len(teams)
3.     schedule = []
4.     # 复制队伍列表，用于模拟旋转
5.     teams = teams[:] # 避免修改原列表
6.     for round in range(n-1):
7.         round_matches = []
8.         # 第一支队伍固定，其他队伍旋转
9.         # 或者采用其他标准方法
10.        # 示例：将第一支队伍与所有其他队伍配对，然后旋转
11.        # 这里展示一个简化的正确逻辑
12.        for i in range(n//2):
13.            home = teams[i]
14.            away = teams[n-1-i]
15.            if i != n-1-i: # 避免同一队伍比赛
16.                round_matches.append((home, away))
17.            schedule.append(round_matches)
18.            # 旋转队伍列表，除了第一个
19.            teams = [teams[0]] + teams[-1:] + teams[1:-1] # 示例旋转，可能需调整
20.        return schedule
21.
22. # 示例运行
23. teams = [1, 2, 3, 4]
24. print(schedule_round_robin(teams))
25. # 输出将展示每轮的比赛安排
```

总结与延伸：分治思想再抽象

“对半分割—子解复制—互补映射”是一种强大的通用范式。



思想迁移

可推广至任意 2^k 规模的会议配对、资源调度等场景。



规模适配

当 $n \neq 2^k$ 时，可通过虚拟选手补全，赛后删除对应场次。



思维奠基

为学习FFT、Strassen等高级分治算法奠定思维基础。

核心收获：掌握将复杂问题递归分解，并通过模式化操作合并解的能力。



THANKS!

